MET's Institute of Engineering

Bhujbal Knowledge City, Adgaon, Nashik.

Department of Computer Engineering

"Stack, Queue and Linked List"

"

Prepared By

Prof. Anand N. Gharu

(Assistant Professor)

Computer Dept.

CLASS : SE COMPUTER 2019

04 August 2025

SUBJECT : DS (SEM-I)

UNIT : III

Note: The material to prepare this presentation has been taken from internet and are generated only for students reference and not for commercial use.

Study material provided by: Vishwajeet Londhe

Join Community by clicking below links

Telegram Channel

https://t.me/SPPU_TE_BE_COMP

(for all engineering Resources)

WhatsApp Channel

(for all Engg & tech updates)

https://whatsapp.com/channel/0029ValjFriICVfpcV9HFc3b

Insta Page

(for all Engg & tech updates)

https://www.instagram.com/sppu_engineering_update

Syllabus

- **Stacks:** Stack operations, Multiple Stacks, Applications of Stack for Expression Conversion [infix, prefix and postfix], Postfix expression evaluation
- **Queues:** Queue Operations, Circular Queue, Priority Queue and its advantages and applications
- **Linked list:** Introduction of Linked Lists, Primitive Operations on Linked List- Create, Traverse, Search, Insert, Delete, Sort, and Concatenate. Types of Linked List: Singly linked, linear and Circular Linked Lists, Doubly Linked List,
- **Case study:** Implementation of Stack and Queue operations using Linked lists

STACK

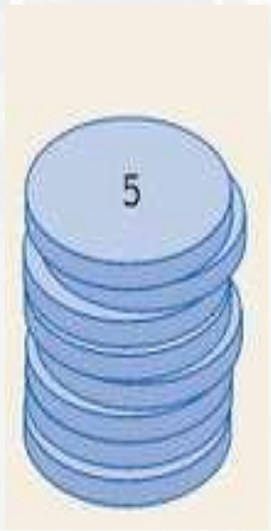
CONCEPT OF STACKs

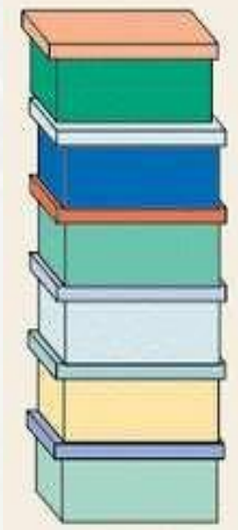
Definition :

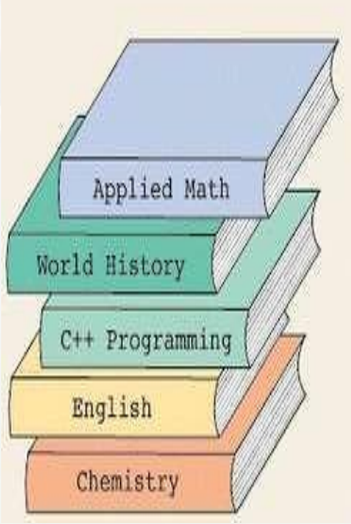
“ Stack is data structure in which addition and removal of an element is allowed at the same end is called as top of stack.”

- Stack is also called as Last In First Out(LIFO) list.
- It means element which get added at last will be removed first.

e.g.



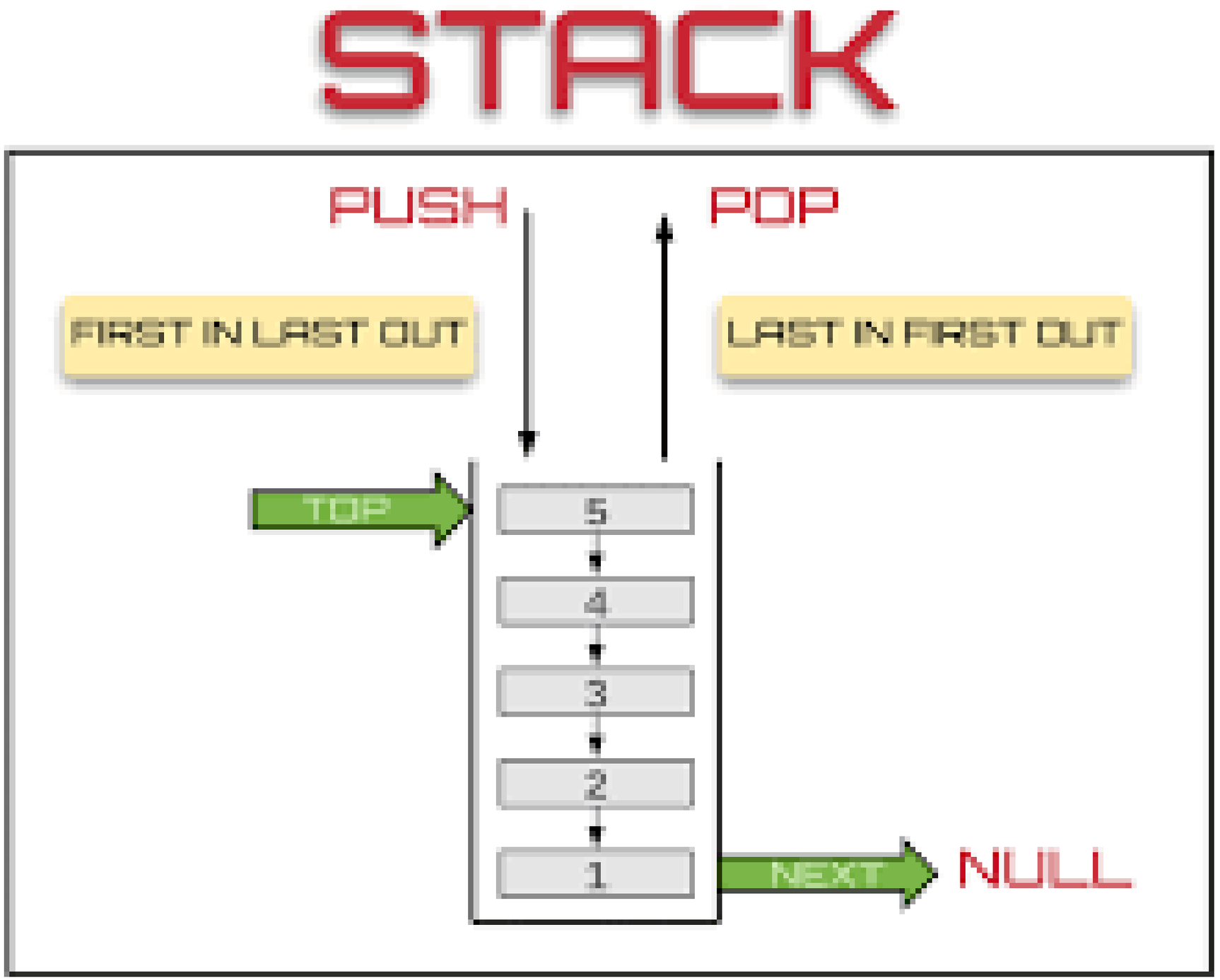




- *Example : https://youtu.be/-KQpk-dlA8s*

CONCEPT OF STACKs

Example :



Components of Stack

- ***Top*** *is a variable which refers to last position in stack.*
- ***Element*** *is component which has data.*
- ***MaxStack*** *is variable that describes maximum number of elements in a stack.*

Main Operation

PUSH

Add data to element in stack.

POP

Take data from element in stack

Kinds of Operation

- *Stack Operation in array form*
- *Stack Operation in Linked list form*

STACK AS AN ADT

Stack is an abstract data type which is defined by the following structure and operation.

Stack operation :

1. createstack( )
2. Push( )
3. Pop( )
4. Peek( )
5. IsEmpty( )
6. IsFull( )
7. Size( )

STACK AS AN ADT

Createstack - it create new empty stack.

push( ) – Pushing (storing) an element on the stack.

pop( ) – Removing an element from the stack.

peek( ) – get the top data element of the stack, without removing it.

isFull( ) – check if stack is full.

isEmpty( ) – check if stack is empty.

Size() – return the number of item in the stack.

- *Example Stack as ADT : https://youtu.be/0jhUesQToOc*

STACK AS AN ADT

1. Initializing stack :

```
void initstack ()
{
    stack[top] = -1 ;
}
```

STACK AS AN ADT

2. IsFull( ) stack :

check stack is full or not?

```
int is_full()
{
  if(top == SIZE-1)
        return(1);
    else
     return(0);
}
```

STACK AS AN ADT

3. IsEmpty( ) stack :

check stack is empty or not?

```
int is_empty()
{
if(top == -1)
        return(1);
 else
    return(0);
}
```

STACK AS AN ADT

4. Push( ) stack : add element in stack

```
void push(int val) {
    if(top>=n-1)
    cout<<"Stack Overflow"<<endl;
    else {
        top++;
        stack[top]=val;
    }
}
```

STACK AS AN ADT

5. pop( ) stack : remove element from stack

```
void pop() {
    if(top<=-1)
    cout<<"Stack Underflow"<<endl;
    else {
        cout<<"The popped element is "<< stack[top] <<endl;
        top--;
    }
}
```

STACK AS AN ADT

6. display( ) stack :displaying stack

```
void display() {
    if(top>=0) {
        cout<<"Stack elements are:";
        for(int i=top; i>=0; i--)
        cout<<stack[i]<<" ";
        cout<<endl;
    } else
    cout<<"Stack is empty";
}
```

ALGORITHM TO IMPLEMENT STACK USING ARRAY

Step 1 : start

Step 2 : Display Menu : 1. push 2. pop 3. display
4. exit.

Step 3 : read choice

Step 4 : if choice 1 then call push ( )
if choice 2 then call pop ( )
if choice 3 the call display ( )
if choice 4 then call exit ( )
default : Invalid choice

Step 5 : read choice again

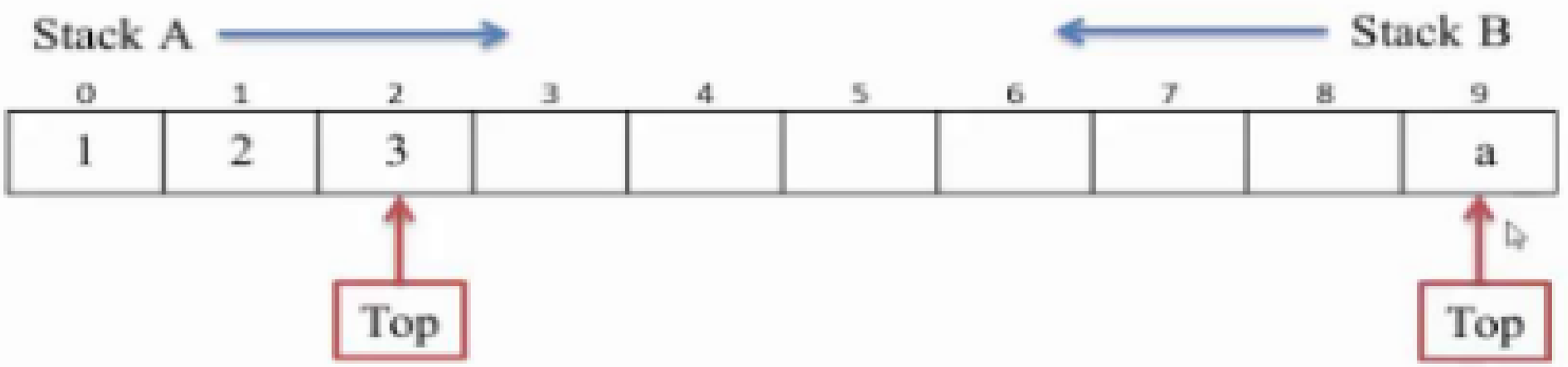
Step 6 : If choice between 1- 3
repeat step 4 else stop

MULTIPLE STACKs

“When a stack is created using single array, we can not able to store large amount of data, thus this problem is rectified using more than one stack in the same array of sufficient array. This technique is called as **Multiple Stack**”

MULTIPLE STACKs

Example : When an array of STACK[n] is used to represent two stacks, say Stack A and Stack B. Then the value of n is such that the combined size of both the Stack[A] and Stack[B] will never exceed n. Stack[A] will grow from left to right, whereas Stack[B] will grow in opposite direction i.e. right to left.



APPLICATION OF STACKs

- Convert infix expression to postfix and prefix expressions
- Evaluate the postfix expression
- Reverse a string
- Check well-formed (nested) parenthesis
- Process subprogram function calls
- Parse (analyze the structure) of computer programs
- Simulate recursion
- In computations like decimal to binary conversion
- In Backtracking algorithms (often used in optimizations and in games)

REVERSING OF STACKs

Algorithms :

Step 1 : start

Step 2 : accept string

Step 3 : insert string into character by character using push method

Step 4 : remove character from stack one by one and print using pop method

Step 5 : stop

REVERSING OF STACKs

```
/*program For Reverse String */
#include<iostream.h>
#include<conio.h>
#include<stdlib.h>
# define MAX 10
class stack
{               char stk[MAX];
                int top;
  public:
                stack()
                 {       top=-1;         }

 void push()
{  int n,i;
    cout<<"Enter the size of string";
    cin>>n;
    if(n>MAX)
          {                 cout<<"out of size";
          }  else
          {    for(i=0;i<n;i++)
               cin>>stk[++top];
           }
          }
```

```
void reverse()
{               if(top<0)
       {   cout <<" stack empty";
                          return;
        }     for(int i=top;i>=0;i--)
                          cout <<stk[i] <<" ";
                          } };
main()
{    int ch;
    clrscr();
    stack st;
    while(1)
{  cout <<"\n1.push  2.reverse  3.exit\n Enter ur
choice";
                cin >> ch;
                switch(ch)
  {
                 case 1:  st.push();break;
                 case 2:  st.reverse();break;
                 case 3:  exit(0);
     }       }
return (0);            }
```

polish notation – Expression Evaluation and conversion

Notation is a way of writing arithmatic expression

Concepts : polish is a way of expressing arithmatic expression that avoids the use of brackets to define periorities for evaluation of operators.

There are three notation :

1. Infix notation
2. Prefix notation
3. Postfix notation

polish notation – Expression Evaluation and conversion

Infix	Prefix	Post fix
(operand) (operator) (operand)	(operator) (operand) (operand)	(operand) (operand) (operator).
(A+B)*C	*+ABC	AB+C*

The example ex...pression in various forms- infix, prefix and postfix

- *The postfix expressions can be evaluated easily hence infix expression is converted into postfix expression using stack.*

- *The following operators are written is in descending order of their precedence:*
  - *Exponentiation ^, Unary +, Unary –, and not ~*
  - ***Multiplication * and division /***
  - *Addition + and subtraction –*
  - **<, £ , =, ¹, ³, >**
  - *AND*
  - ***OR***

The Operators and priorities

Operator Arithmetic, Boolean and relational	Priority
∧, Unary +, Unary − , ~	6
* /	5
+ −	4
<, ≤ , =, ≠, ≥, >	3
AND	2
OR	1

Algorithm Infix to postfix conversion

Step 1 : The input string (infix notation) is scanned from left to right.

Step 2 : If the scanned character (ch) is space or tab, it is skipped.

Step 3 : If the scanned character (ch) is digit or alphabet, it is appended to postfix string.

Step 4 : If the scanned character (ch) is opening parenthesis '(', it is pushed to stack.

Step 5 : If the scanned character (ch) is operator :

All operators from top of the stack are popped having more or same priority than ch and appended to postfix string.

When less priority operator is found, then it is pushed in the stack again with ch.

Step 6 : If the scanned character (ch) is closing parenthesis ')', then all the operators above the opening parenthesis are appended to postfix string.

Step 7 : After evaluations of all the characters, the remaining operators from stack are appended to postfix string.

Infix to postfix conversion

- Manual algorithm for converting infix to postfix

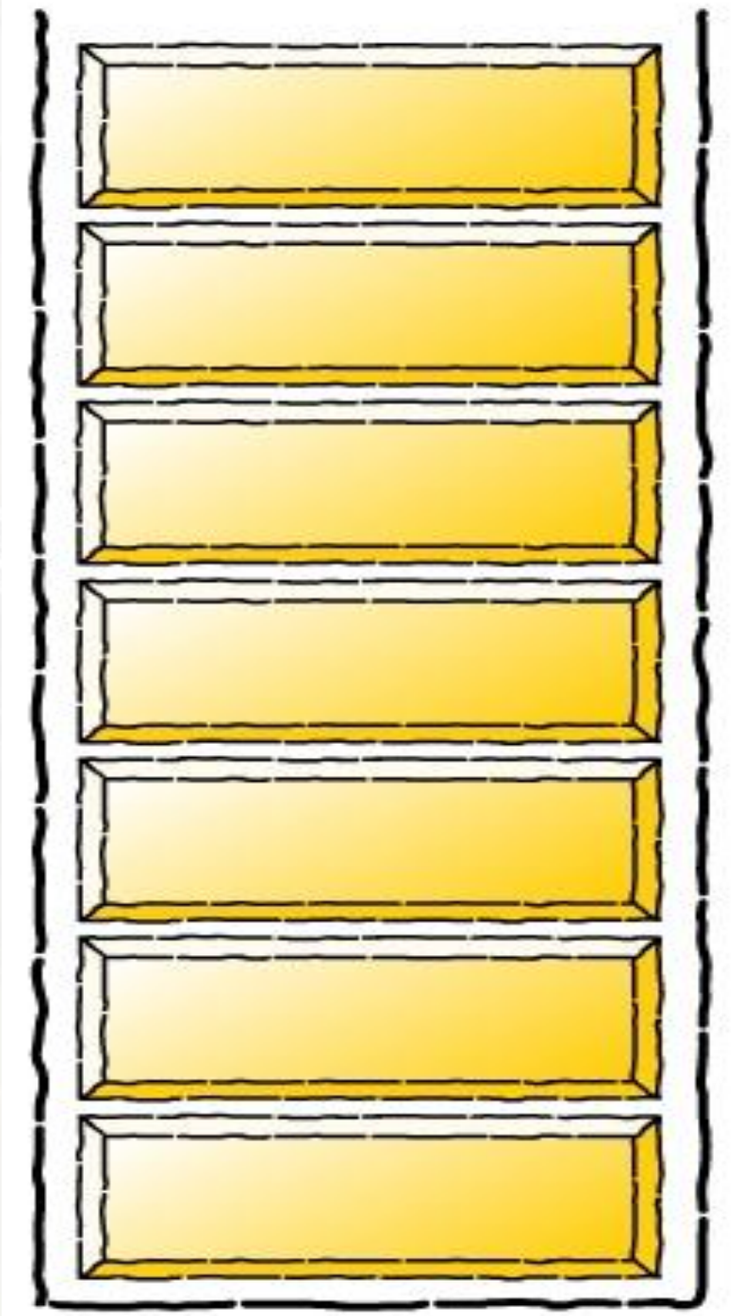
(a + b) * c

  - Write with parentheses to force correct operator precedence ((a + b) * c)

  - Move operator to right inside parentheses ((a b +) c *)

  - Remove parentheses a b + c *

Infix to postfix conversion



infixVect

(a + b - c) * d – (e + f)

postfixVect

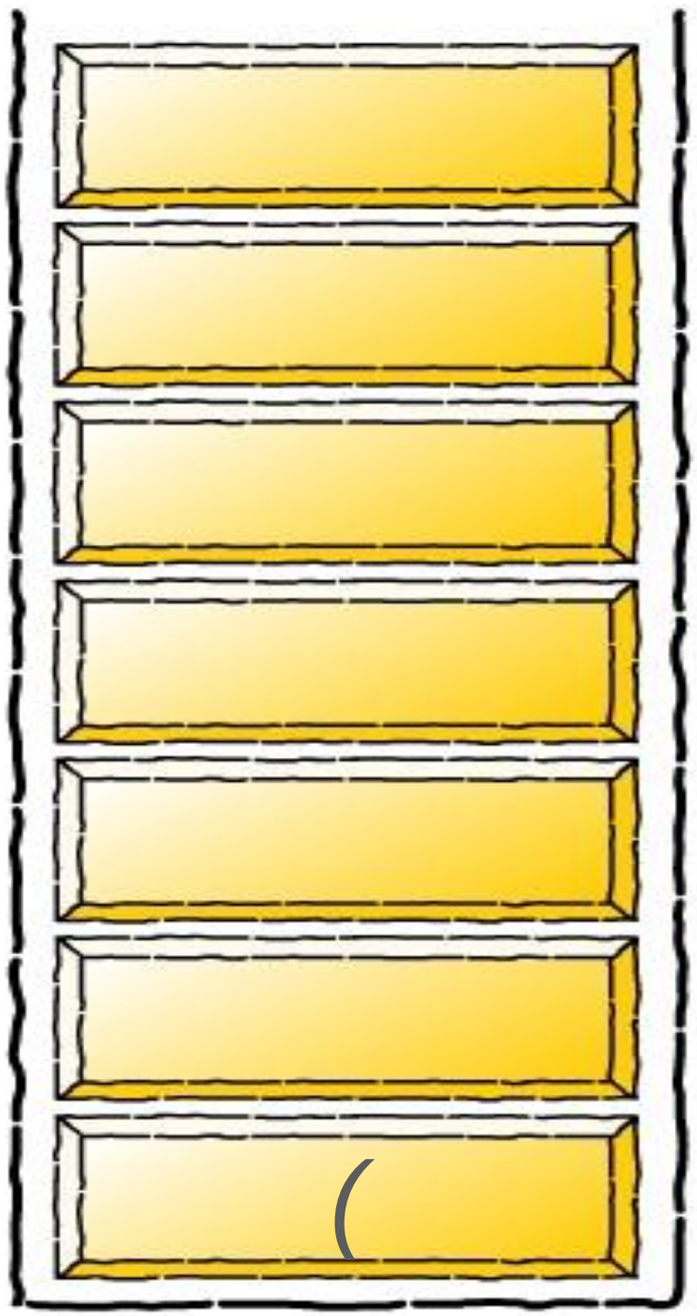
Example : https://youtu.be/OVFwgYrMShw

https://youtu.be/vq-nUF0G4fI

https://youtu.be/vXPL6UavUeA

Infix to postfix conversion

stackVect



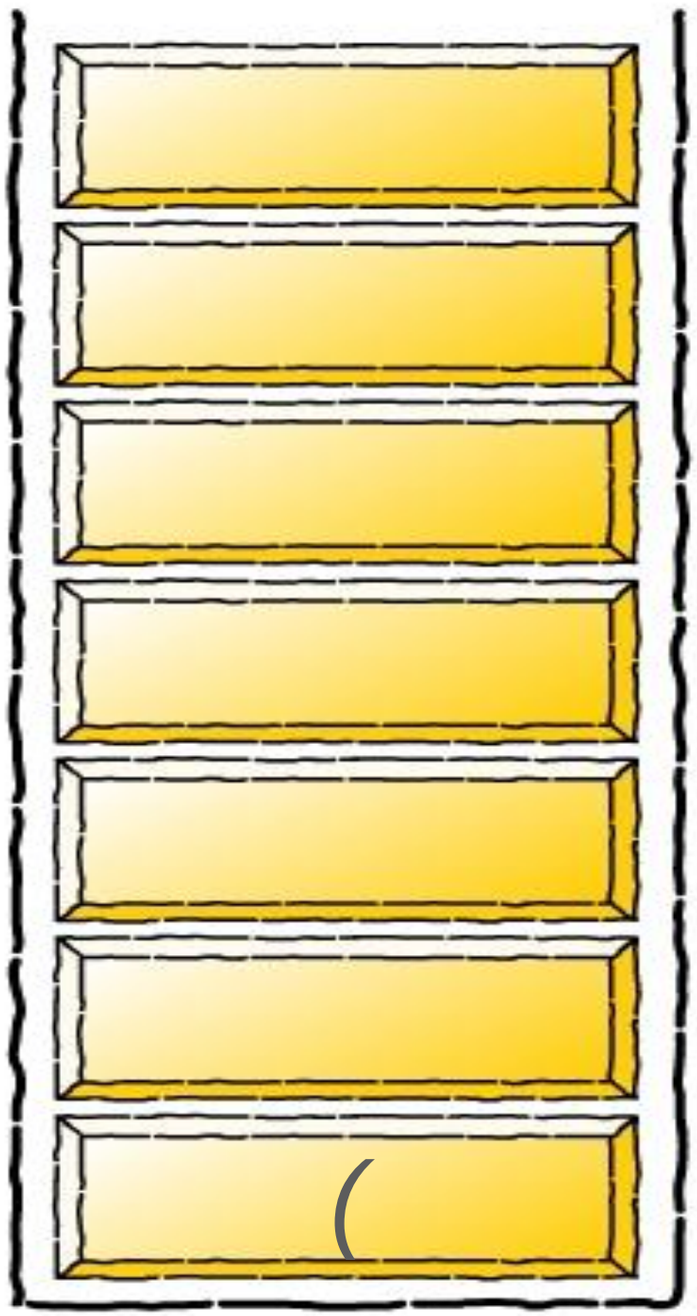
infixVect

a + b - c) * d – (e + f)

postfixVect

Infix to postfix conversion

stackVect



infixVect

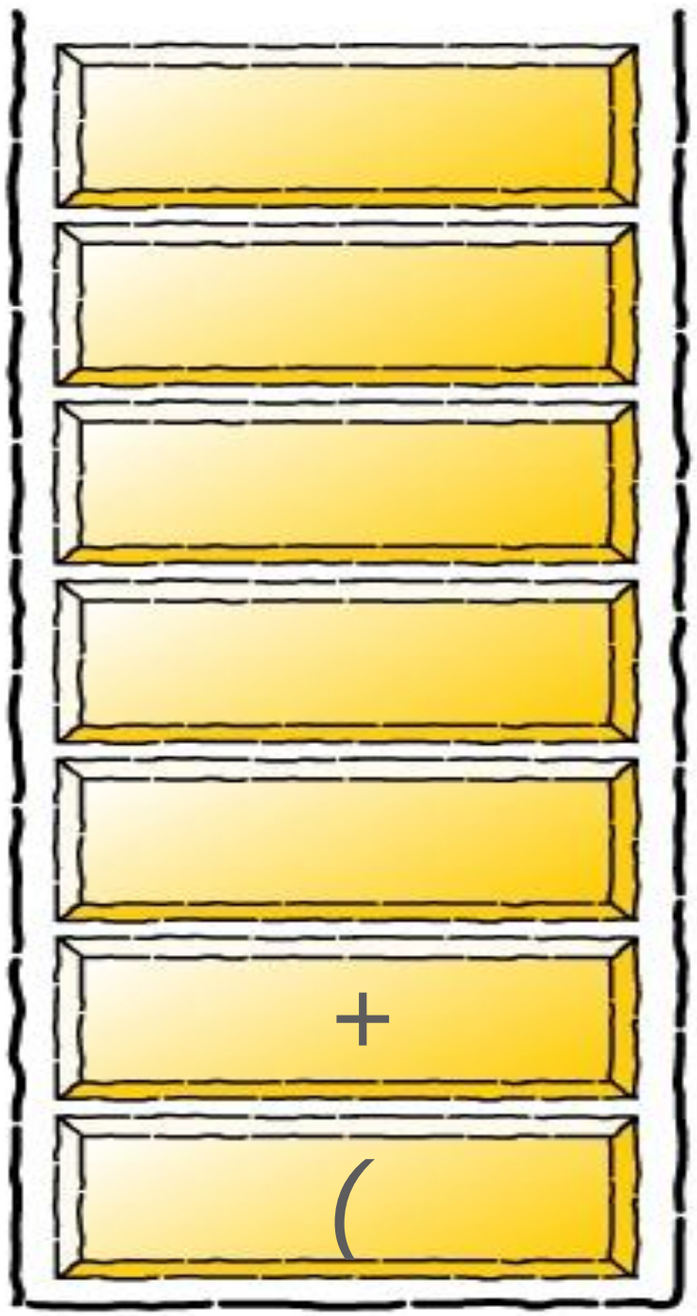
+ *b* - *c*) * *d* – (*e* + *f*)

postfixVect

a

Infix to postfix conversion

stackVect



infixVect

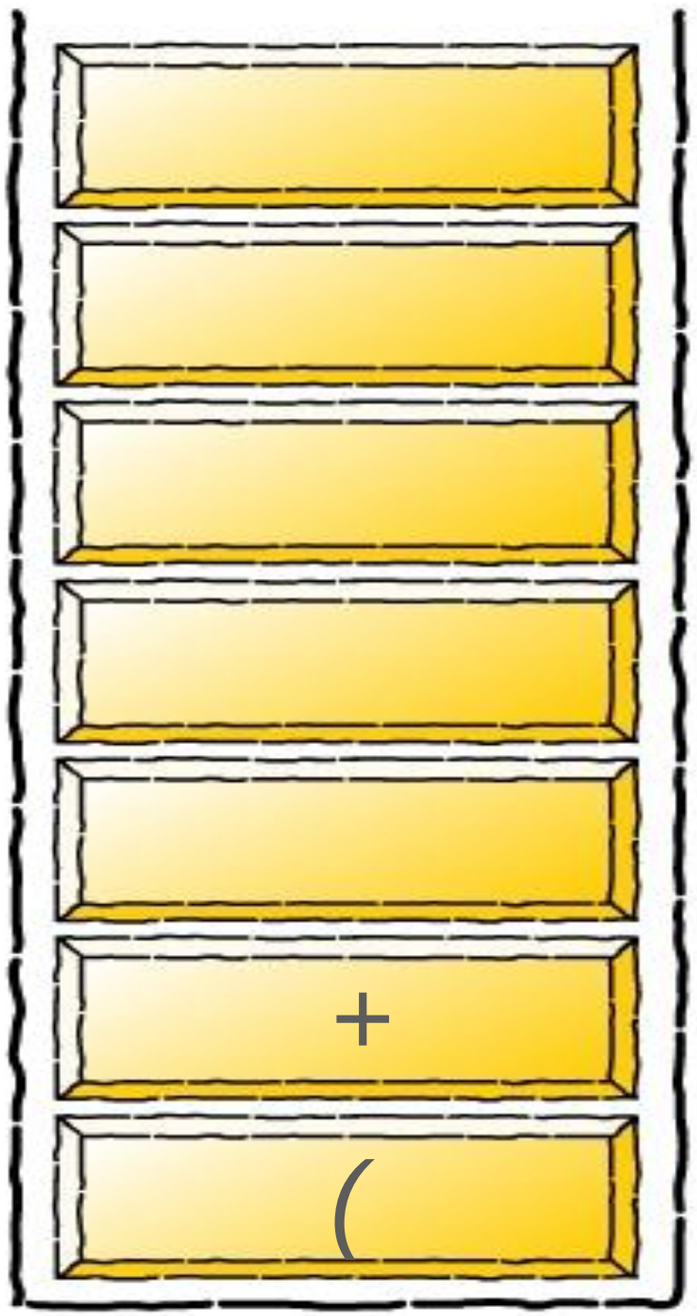
b - c) * d – (e + f)

postfixVect

a

Infix to postfix conversion

stackVect



infixVect

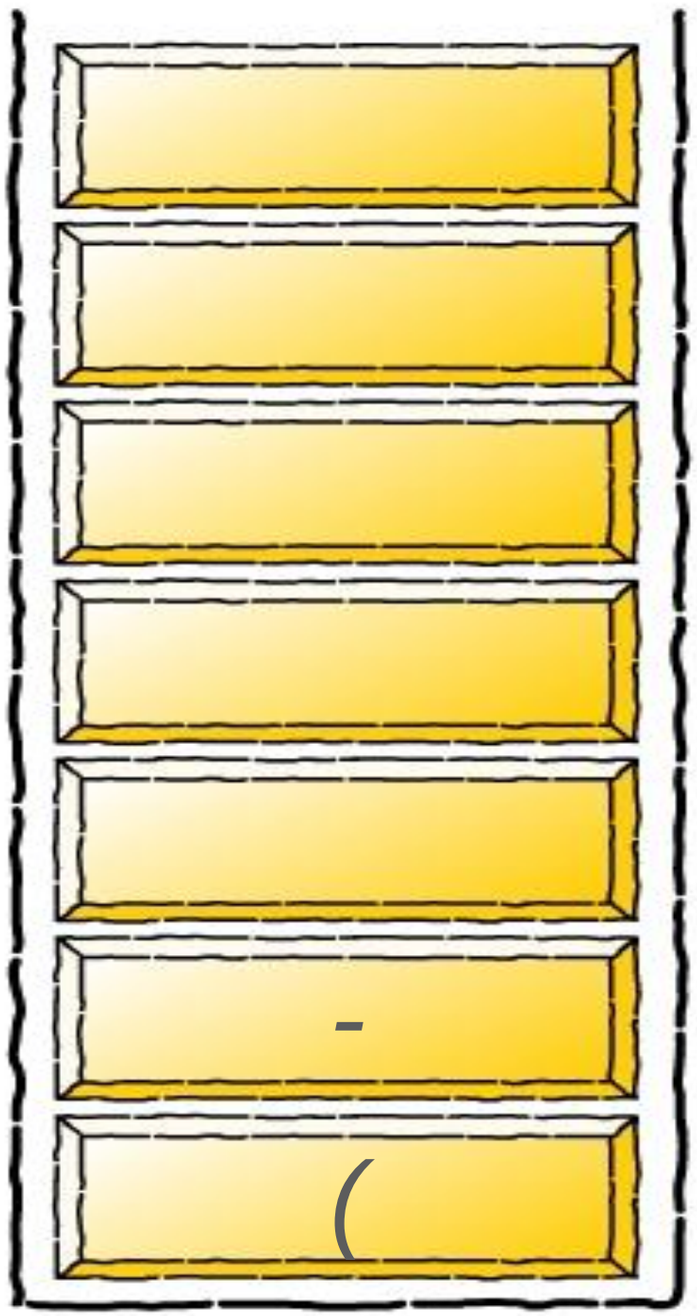
*- c) * d – (e + f)*

postfixVect

a b

Infix to postfix conversion

stackVect



infixVect

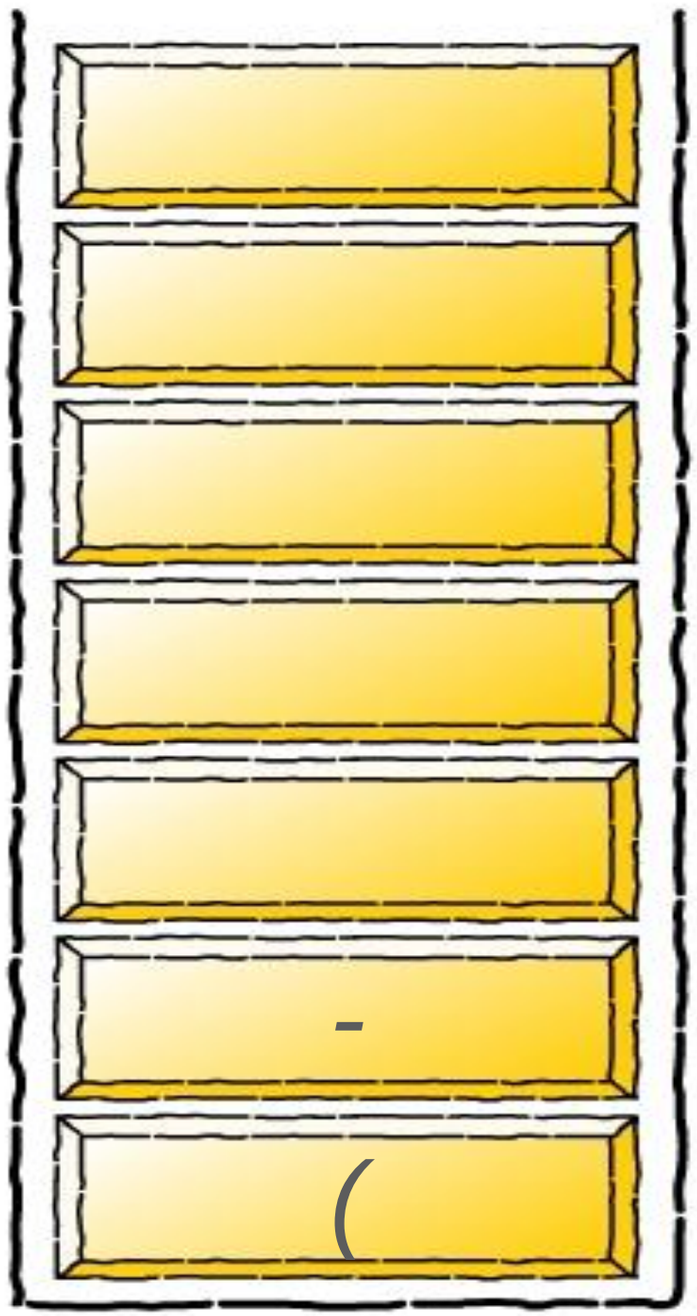
c) * d – (e + f)

postfixVect

a b +

Infix to postfix conversion

stackVect



infixVect

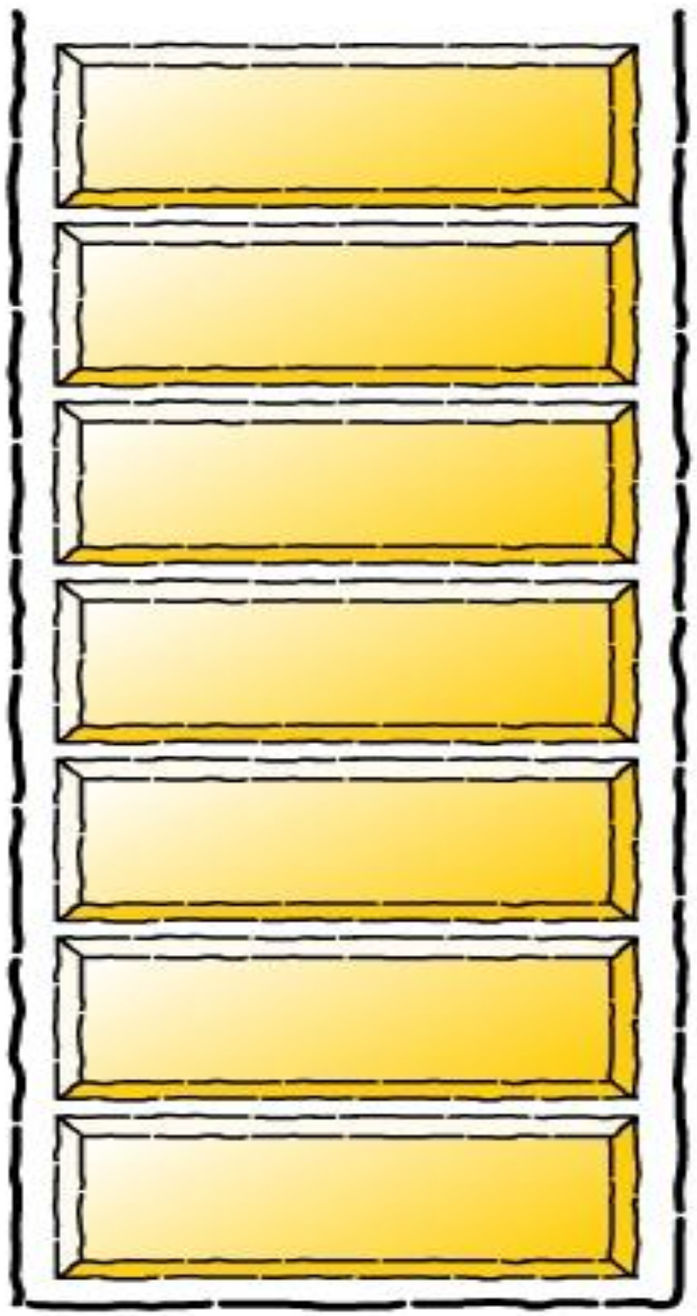
) * d – (e + f)

postfixVect

a b + c

Infix to postfix conversion

stackVect



infixVect

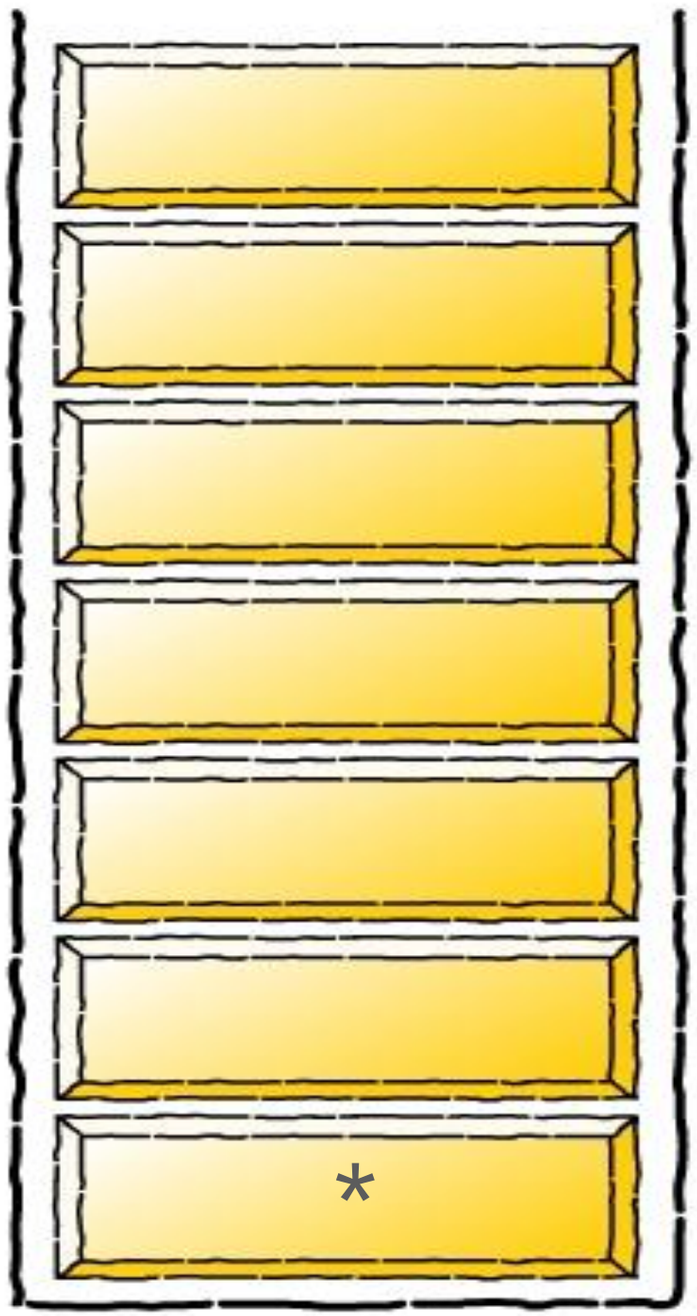
** d – (e + f)*

postfixVect

a b + c -

Infix to postfix conversion

stackVect



infixVect

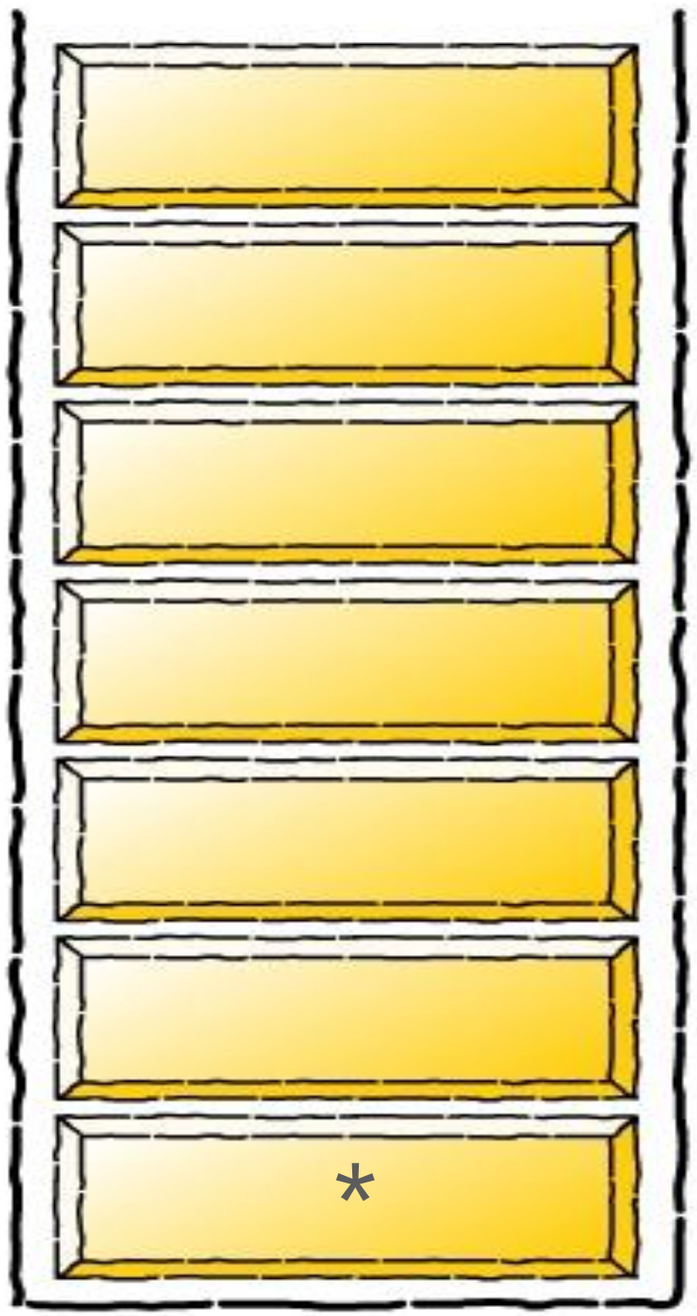
$d - (e + f)$

postfixVect

$a\ b + c -$

Infix to postfix conversion

stackVect



infixVect

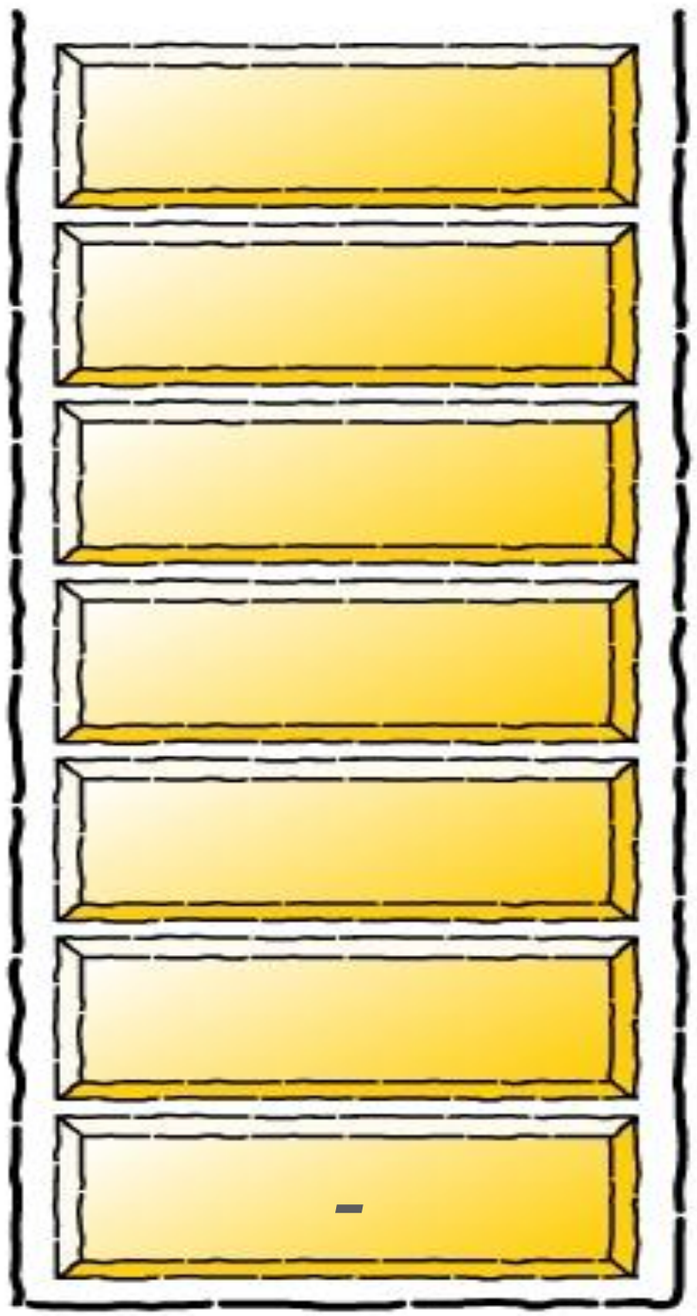
– (e + f)

postfixVect

a b + c - d

Infix to postfix conversion

stackVect



infixVect

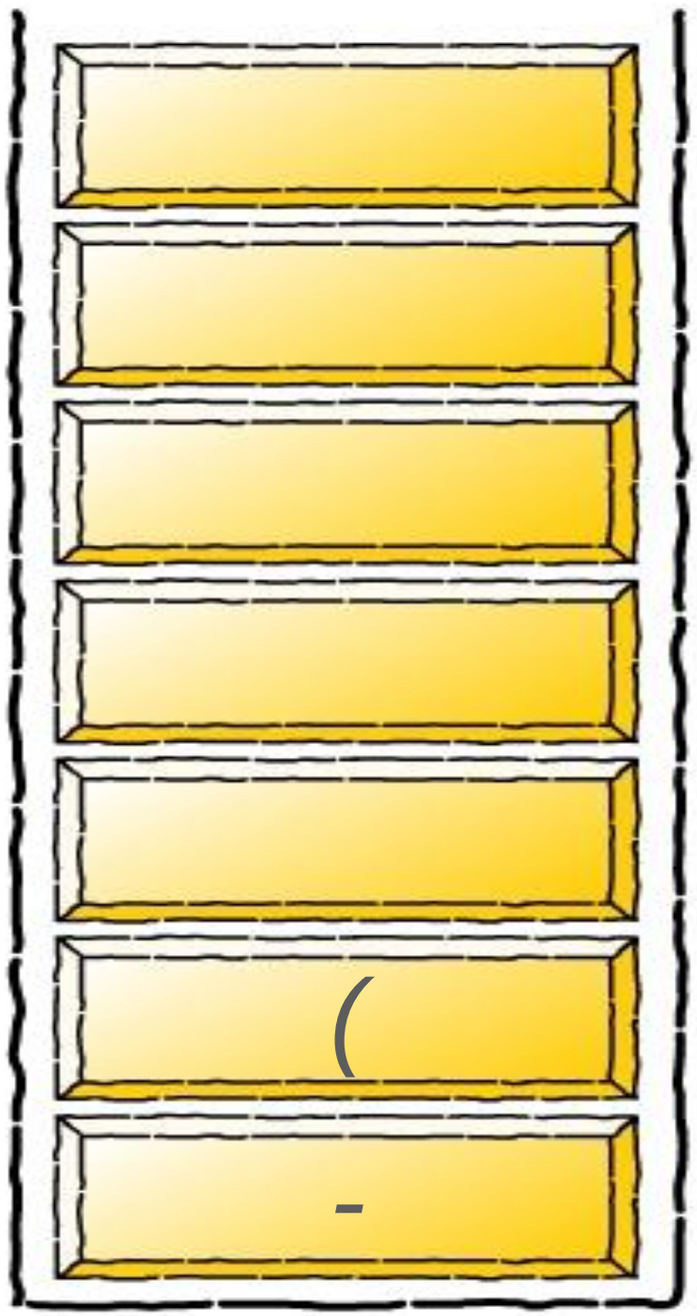
(e + f)

postfixVect

a b + c – d *

Infix to postfix conversion

stackVect



infixVect

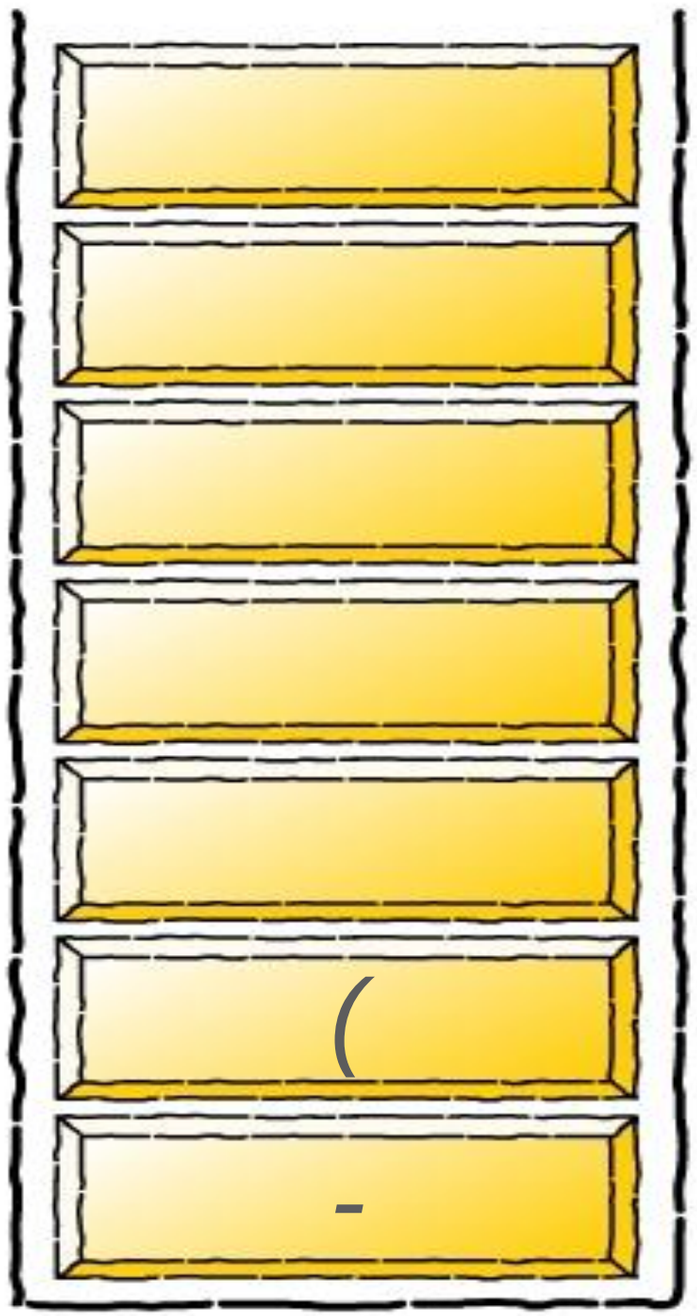
e + f)

postfixVect

*a b + c – d **

Infix to postfix conversion

stackVect



infixVect

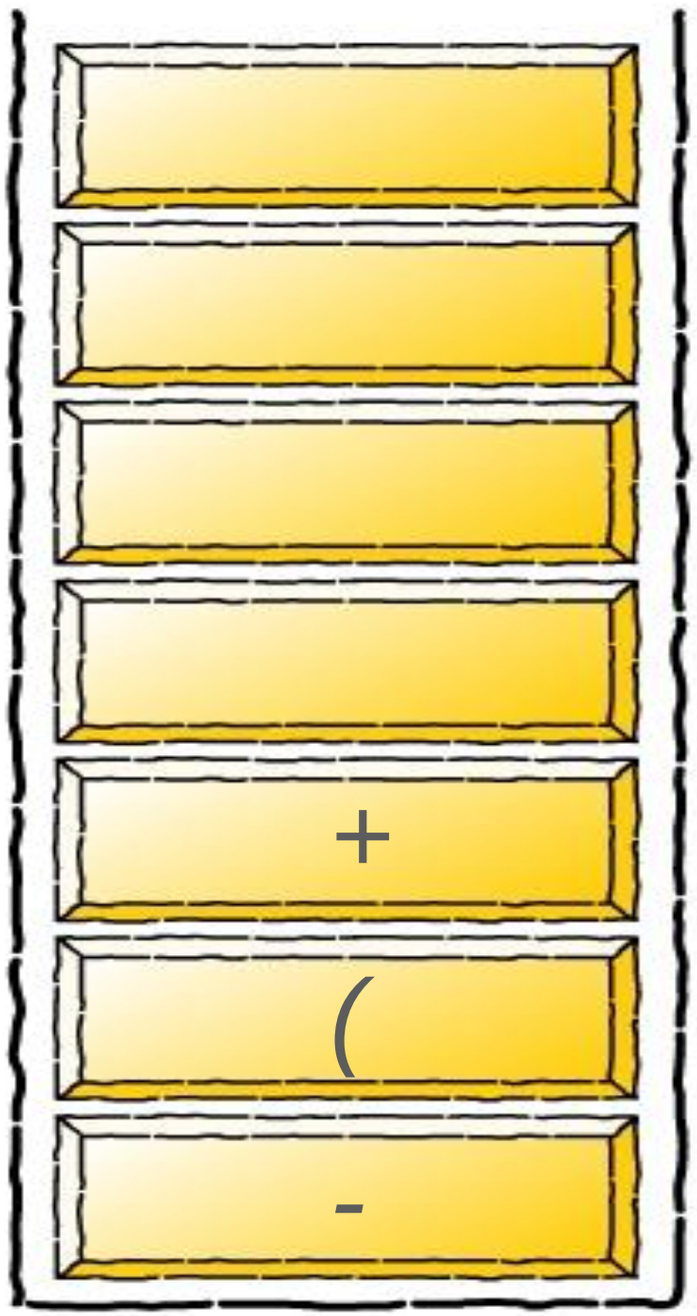
+ f)

postfixVect

*a b + c – d * e*

Infix to postfix conversion

stackVect



infixVect

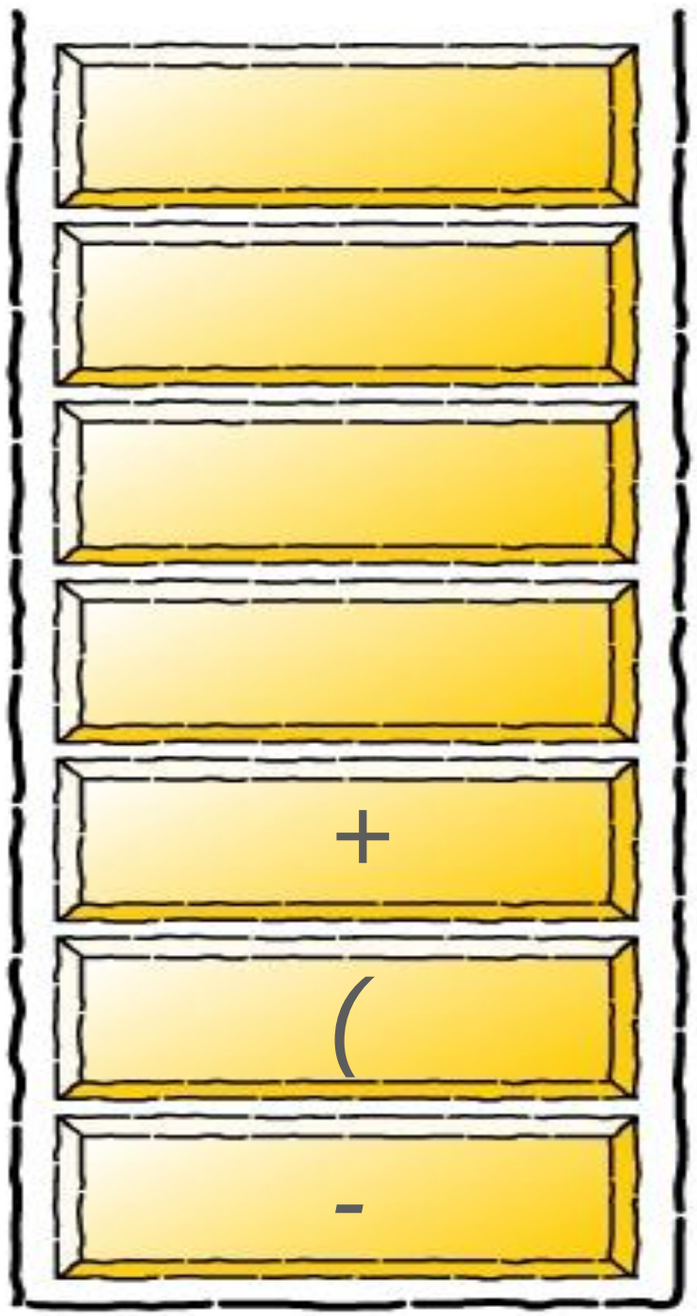
f)

postfixVect

*a b + c – d * e*

Infix to postfix conversion

stackVect



infixVect

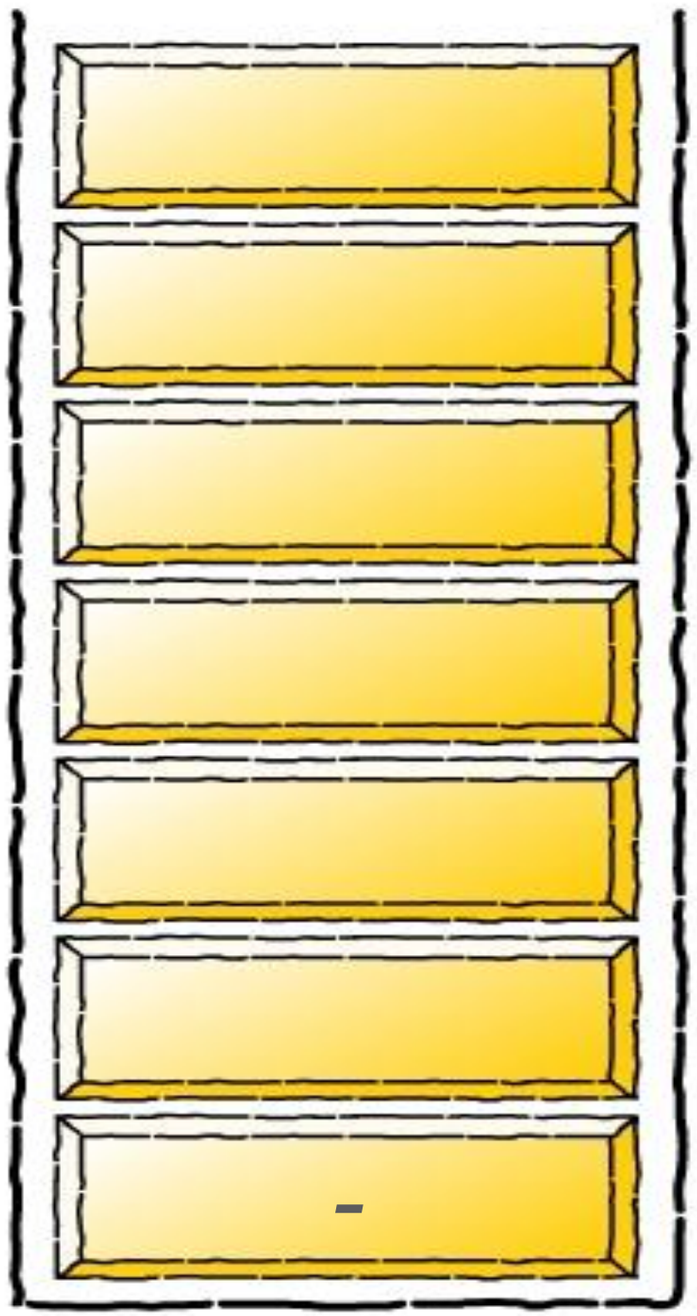
)

postfixVect

*a b + c – d * e f*

Infix to postfix conversion

stackVect



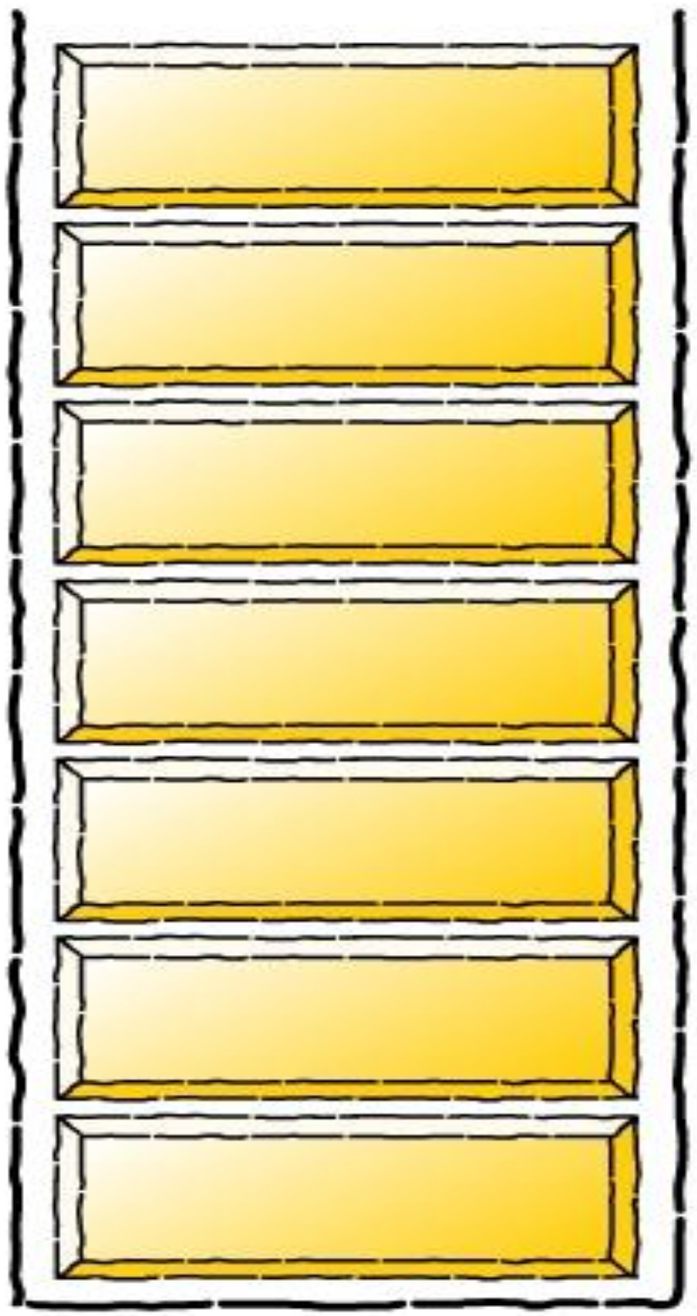
infixVect

postfixVect

*a b + c – d * e f +*

Infix to postfix conversion

stackVect



infixVect

postfixVect

*a b + c – d * e f + -*

Infix to postfix conversion

Infix Expression: a + (b*c).

Conversion of Infix to Postfix

Example to Convert Infix to Postfix using stack

a + (b*c)

Read character	Stack	Output
a	Empty	a
+	+	a
(	+(	a
b	+(	ab
*	+(*	ab
c	+(*	abc
)	+	abc*
		abc*+

9CM305.25 19

Resultant Postfix Expression: abc+*

Infix to postfix conversion

Infix Expression: A * B + C * D.

Character Scanned	Stack	Postfix)
A	–	A
*	*	A
B	*	AB
+	+	AB*
C	+	AB*C
*	+*	AB*C
D	+*	AB*CD
	Empty	AB*CD*+

Postfix expression is = AB*CD*+

Resul

Infix to postfix conversion

Input	Stack	Postfix evaluation
2*3 + 4*5	Nothing in stack	
*3+4*5	Nothing in stack	2
3+4*5	*	2
+4*5	*	23
4*5	+	23*
*5	+	23*4
5	*+	23*4
	*+	23*45
	+	23*45*
	Nothing in stack	23*45*+

Infix to postfix conversion

(2—3+4)(5+6*7)*

Input	Stack	Postfix evaluation
(2-3+4)*(5+6*7)	Nothing in stack	
2-3+4)*(5+6*7)	(	
-3+4)*(5+6*7)	(	2
3+4)*(5+6*7)	(-	2
+4)*(5+6*7)	(-	23
4)*(5+6*7)	(+	23-
)*(5+6*7)	(+	23-4
*(5+6*7)	Nothing in stack	23-4+
(5+6*7)	*	23-4+
5+6*7)	(*	23-4+
+6*7)	(*	23-4+5
6*7)	+(*	23-4+5
*7)	+(*	23-4+56
7)	*+(*	23-4+56
)	*+(*	23-4+567
	*	23-4+567*+
	Nothing in stack	23-4+567*+*

Infix to postfix conversion

((A + B) * (C – D))/E

Table 4.8.1 : Infix to Postfix Conversion

Character from Infix	Stack	Postfix Expression
(	(	
(	((	
A	((	A
+	((+	A
B	((+	AB
)	(	AB+
*	(*	AB+
(	(*(	AB+
C	(*(	AB+C
-	(*(-	AB+C
D	(*(-	AB+CD
)	(*	AB+CD-
)		AB+CD-*
/	/	AB+CD-*
E	/	AB+CD-*E
		AB+CD-*E/

*Resultant Postfix Expression: AB+CD-*E/*

Infix to postfix conversion

a ↑ b * c-d+e/f /(g + h)

Character Scanned	Stack	Postfix)
A	–	a
↑	↑	a
B	↑	ab
*	↑	ab
C	*	ab ↑c
-	–	ab ↑ c*
D	–	ab ↑ c * d
+	+	ab ↑ c * d –
E	+	ab ↑ c * d – e
/	+ /	ab ↑ c * d – e
F	+ /	ab ↑ c * d – ef
/	+/	ab ↑ c * d – ef /
(	+ / (	ab ↑ c * d – ef /
G	+ / (	ab ↑ c * d – ef / g

Character Scanned	Stack	Postfix)
+	+/(+	ab ↑ c * d – ef / g
H	+/(+	ab ↑ c * d – ef / gh
	+/	ab ↑ c * d – ef /gh +
	Empty	ab ↑ c * d – ef/ gh +/+

∴ Postfix expression is = ab ↑ c * d – ef / gh + /+

Resultant Postfix Expression:

Infix to postfix conversion

((A + B) * D)↑ (E – F)

Table 4.8.3 : Infix to Postfix Conversion

Character Scanned	Stack	Postfix
(	(	–
(	((	–
A	((	A
+	((+	A
B	((+	AB
)	(	AB+
*	(*	AB+
D	(*	AB+D
)	Empty	AB+D*
↑	↑	AB+D*
(	↑(	AB+D*E
E	↑(	AB+D*E
–	↑(–	AB+D*E
F	↑(–	AB+D*EF
	↑	AB+D*EF–
	Empty	AB+D*EF–↑

∴ Postfix expression = AB+D*EF–↑

Infix to postfix conversion

((a/(b-c+d))*(e - a)*c)

Character Scanned	Stack	Postfix)
(	(	-
(	((	-
A		a
/	((/	a
(	((/(	a
B		ab
-	((/(-	
C		abc
+	((/(+	abc-
D	((/(+	abc-d
)	((/	abc-d+
)		abc-d+/
*	(*	abc-d+/
(	(*(	abc-d+/
E	(*(	abc-d+/e
-	(*(-	abc-d+/e
A	(*(-	abc-d+/ea
)	(*	abc-d+/ea-
*	(*	abc-d+/ea-*
C	(*	abc-d+/ea-*c
)	empty	abc-d+/ea-*c*

Resultant Postfix Expression: **abc-d+/ea-*c***

Infix to postfix conversion

Infix Expression: ***A+ (B*C-(D/E^F)*G)*H***, where ^ is an exponential operator.

Symbol	Scanned	STACK	Postfix Expression	Description
1.		(		Start
2.	A	(	A	
3.	+	(+	A	
4.	(	(+(	A	
5.	B	(+(	AB	
6.	*	(+(*	AB	
7.	C	(+(*	ABC	
8.	-	(+(-	ABC*	'*' is at higher precedence than '-'
9.	(	(+(-(	ABC*	
10.	D	(+(-(	ABC*D	
11.	/	(+(-(/	ABC*D	
12.	E	(+(-(/	ABC*DE	
13.	^	(+(-(/^	ABC*DE	
14.	F	(+(-(/^	ABC*DEF	
15.	)	(+(-	ABC*DEF^/	Pop from top on Stack , that's why '^' Come first
16.	*	(+(-*	ABC*DEF^/	
17.	G	(+(-*	ABC*DEF^/G	
18.	)	(+	ABC*DEF^/G*-	Pop from top on Stack , that's why '^' Come first
19.	*	(+*	ABC*DEF^/G*-	
20.	H	(+*	ABC*DEF^/G*-H	
21.	)	Empty	ABC*DEF^/G*-H*+	END

*Resultant Postfix Expression: ABC*DEF^/G*-H*+*

Infix to postfix conversion

Infix Expression: A+(B*(C-D)/E).

	RPN	Stack	Input Expression
1			A+(B*(C-D)/E)
2	A		+(B*(C-D)/E)
3	A	+	(B*(C-D)/E)
4	A	(+	B*(C-D)/E)
5	AB	(+	*(C-D)/E)
6	AB	* (+	(C-D)/E)
7	AB	(* (+	C-D)/E)
8	ABC	(* (+	-D)/E)
9	ABC	(* (+	D)/E)
10	ABCD	(* (+	)/E)
11	ABCD-	* (+	/E)
12	ABCD-*	/ (+	E)
13	ABCD-*E	/ (+	)
14	ABCD-*E/	+	
15	ABCD-*E/+		

*Resultant Postfix Expression: ABCD-*E/+*

Infix to postfix conversion

Infix Expression: A * (B + C * D) + E.

Infix to Postfix using stack ...

- Example A * (B + C * D) + E becomes A B C D * + * E +

	current symbol	operator stack	postfix string
1	A		A
2	*	*	A
3	(	*(	A
4	B	*(	A B
5	+	*(+	A B
6	C	*(+	A B C
7	*	*(+*	A B C
8	D	*(+*	A B C D
9	)	*	A B C D *+
10	+	+	A B C D *+*
11	E	+	A B C D *+*E
12			A B C D *+*E+

Resultant Postfix Expression: ABCD+*E+*

Infix to postfix conversion

Infix Expression : A+B*(C^D-E)				
Token	**Action**	**Result**	**Stack**	**Notes**
A	Add **A** to the result	A		
+	Push **+** to stack	A	+	
B	Add **B** to the result	AB	+	
*	Push ***** to stack	AB	* +	* has higher precedence than +
(	Push **(** to stack	AB	(* +	
C	Add **C** to the result	ABC	(* +	
^	Push **^** to stack	ABC	^ (* +	
D	Add **D** to the result	ABCD	^ (* +	
-	Pop **^** from stack and add to result	ABCD^	(* +	- has lower precedence than ^
	Push **-** to stack	ABCD^	- (* +	
E	Add **E** to the result	ABCD^E	- (* +	
)	Pop **-** from stack and add to result	ABCD^E-	(* +	Do process until (is popped from stack
	Pop **(** from stack	ABCD^E-	* +	
	Pop ***** from stack and add to result	ABCD^E-*	+	Given expression is iterated, do Process till stack is not Empty, It will give the final result
	Pop **+** from stack and add to result	**ABCD^E-*+**		
Postfix Expression : ABCD^E-*+				

Infix to postfix conversion

Suppose we want to convert 2*3/(2-1)+5*3 into Postfix form,

Expression	Stack	Output
2	Empty	2
*	*	2
3	*	23
/	/	23*
(	/(	23*
2	/(	23*2
-	/(-	23*2
1	/(-	23*21
)	/	23*21-
+	+	23*21-/
5	+	23*21-/5
*	+*	23*21-/53
3	+*	23*21-/53
	Empty	23*21-/53*+

So, the Postfix Expression is 23*21-/53*+

Infix to postfix conversion

Convert A * (B + C) * D
to postfix notation.

Scanned character	stack	postfix
A		A
*	*	A
(	*(	A
B	*(	AB
+	*(+	AB
C	*(+	ABC
)	*	ABC+
*	*	ABC+*
D	*	ABC+*D
	Empty	ABC+*D*

Infix to postfix conversion

(a + (b * c) / (d – e)) = abc*de-/+

SYMBOL	POSTFIX	OPSTACK
(	EMPTY	(
a	a	(
+	a	(+
(	a	(+(
b	ab	(+(
*	ab	(+(*
c	abc	(+(*
)	abc*	(+
/	abc*	(+/
(	abc*	(+/(
d	abc*d	(+/(
-	abc*d	(+/(-
e	abc*de	(+/(-
)	abc*de-	(+/
)	abc*de-/+	Empty

Postfix into Infix conversion

Algorithm:

Iterate the given expression from left to right, one character at a time

1. If a character is operand, push it to stack.

2. If a character is an operator,

- pop operand from the stack, say it's s1.
- pop operand from the stack, say it's s2.
- perform **(s2 operator s1)** and push it to stack.

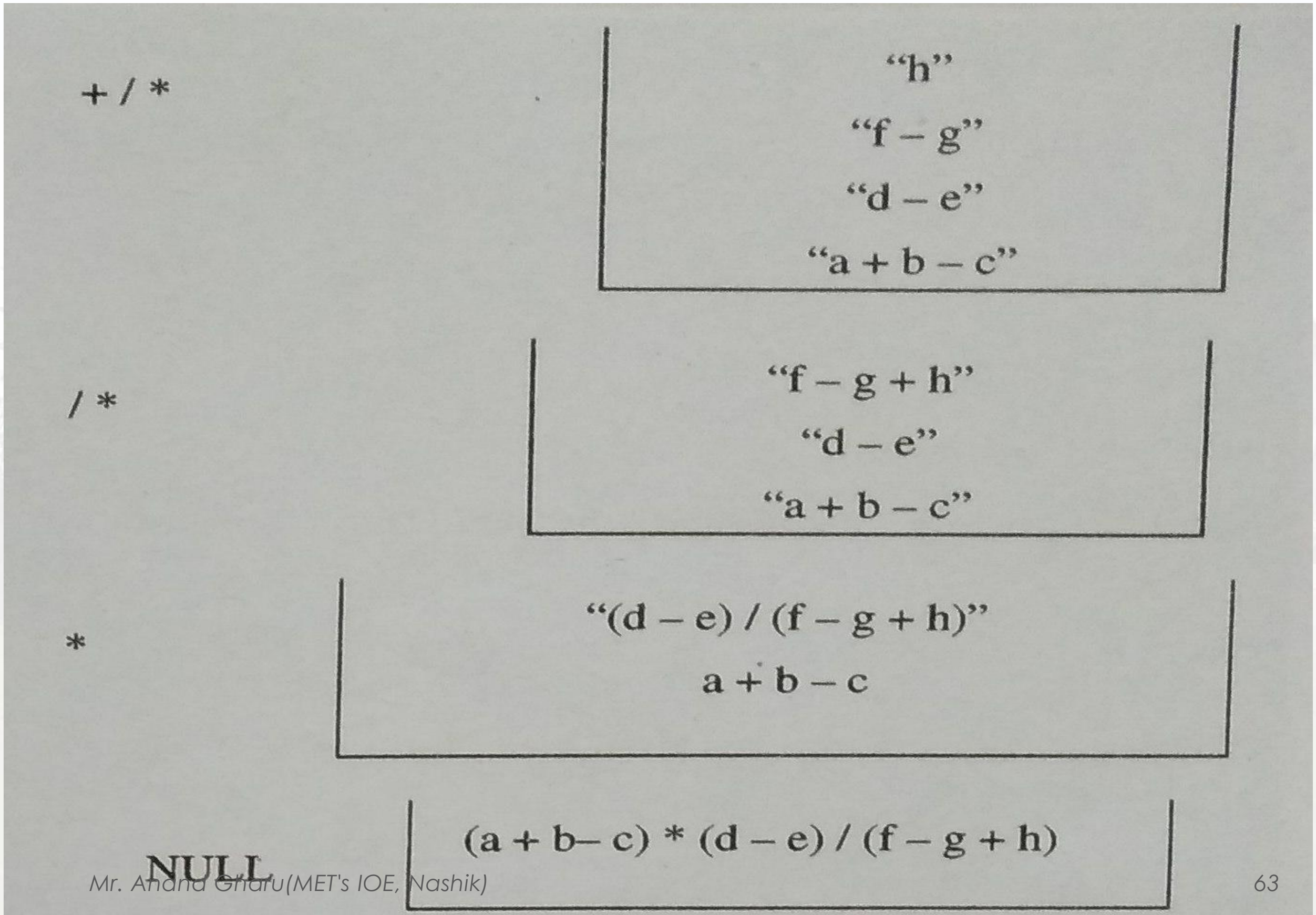
3. Once the expression iteration is completed, initialize the result string and pop out from the stack and add it to the result.

4. Return the result.

Postfix into Infix conversion

Expression	Stack (a stack of strings)
bc – – de –fg – h + / *	"a"
c – + de –fg – h + / *	"b" "a"
– + de – fg – h + / *	"c" "b" "a"
+ de – fg –h + / *	"b –c" "a"
de – fg – h + / *	"a + b – c"
e – fg – h + / *	"d" "a + b – c"
– fg – h + / *	"e" "d" "a + b – c"
fg – h + / *	"d – e" "a + b – c"
g – h + / *	"f" "d – e" "a + b – c"
– h + / *	"g" "f" "d – e" "a + b – c"
h + / *	"f – g" "d – e" "a + b – c"

Postfix into Infix conversion



Postfix into Infix conversion

Convert the following postfix expression to infix and prefix using stack. Show the contents of stack at every stage :

Given expression : abcde∧∧* –

Solution :

(i) Postfix to infix

Expression	Stack	Action
a b c d e ∧ ∧ * –	Empty	initial
b c d e ∧ ∧ * –	a	push
c d e ∧ ∧ * –	b a	push
d e ∧ ∧ * –	c b a	push
e ∧ ∧ * –	d c b a	push
∧ ∧ * –	e d c b a	push
∧ * –	(d^e) c b a	Convert
* –	c∧ (d∧e) b a	Convert
–	b*c∧ (d ∧e) a	convert
End	a–b * c∧ (d ∧e)	Convert

Postfix expression : a – b * C ∧ (d ∧ e)

Postfix into prefix conversion

(ii) Postfix to prefix

Expression	Stack	Action
a b c d e ∧ ∧ * –	Empty	initial
b c d e ∧ ∧ * –	a	push
c d e ∧ ∧ * –	b a	push
d e ∧ ∧ * –	c b a	push
e ∧ ∧ * –	d c b a	push
∧ ∧ * –	e d c b a	push
∧ * –	∧de c b a	Convert
* –	∧c∧de b a	Convert
–	*b∧c∧de a	convert
End	– a*b ∧ c∧de	Convert

Prefix expression : – a * b∧ c ∧d e

POSTFIX EXPRESSION EVALUATION

ALGORITHMS :

Step 1 : Read postfix expression from Left to Right

Step 2 : If operand is encountered, push it in Stack.

Step 3 : If operator is encountered, Pop two elements
A → Top element B → Next Top element
Evaluate B operator A

Step 4 : Push result into stack

Step 5 : Read next postfix element if not end of postfix string

Step 6 : Repeat from Step 2

Step 7 : Print the result popped from stack.

POSTFIX EXPRESSION EVALUATION

POSTFIX EVALUATE EXPRESSION : 53+82-* https://youtu.be/TeM6DdvSxvC

Reading symbol	Stack operations	Evaluated part of expression
Initially	Stack is Empty	Nothing
5	push(5)	Nothing
3	push(3)	Nothing

+	Value 1 = pop()
	Value 2 = pop()
	result = Value 2 + Value 1
	push(result)
		Value 1 = pop(); // 3
		Value 2 = pop(); // 5
		result = 5 + 3; // 8
		Push(8)
		(5 + 3)

8	push(8)	(5 + 3)
2	push(2)	(5 + 3)

–	Value 1 = pop()
	Value 2 = pop()
	result = Value 2 – Value 1
	push(result)
		Value 1 = pop(); // 2
		Value 2 = pop(); // 8
		result = 8 – 2; // 6
		Push(6)
		(8 – 2)
		(5 + 3),(8 – 2)

*	Value 1 = pop()
	Value 2 = pop()
	result = Value 2 + Value 1
	push(result)
		Value 1 = pop(); // 6
		Value 2 = pop(); // 8
		result = 8 * 6; // 48
		Push(48)
		(6 * 8)
		(5 + 3),(8 – 2)

End of expression	result = pop()	Display (result) 48 as final result

POSTFIX EXPRESSION EVALUATION

**** Evaluate the following postfix expression and show stack after every step in tabular form. Given A=5, B=6, C=2, D=12, E=4***

ABC + *DE\-

ABC+*DE/-

This will become

5 6 2 + * 12 4 / -

Table 4.9.2 : Evaluate Postfix Expression

Reading	Stack	Evaluated
5	5	
6	5,6	
2	5,6,2	
+	5,8	6 + 2 = 8
*	40	5*8 = 40
12	40,12	
4	40,12,4	
/	40,3	12 / 4 = 3
-	37	40-3 = 37

Result is 37.

POSTFIX EXPRESSION EVALUATION

- *Evaluate the following postfix expression*

  *A : 6, 2, 3, +, - , 3, 8, 2, +, +, * , 2, ^, 3, +*

Reading	Stack	Evaluated
6	6	
2	6 2	
3	6 2 3	
+	6 5	2 + 3 = 5
-	1	6 – 5
3	1 3	
8	1 3 8	
2	1 3 8 2	
+	1 3 10	8 + 2 = 10
+	1 13	3 + 10 = 13
*	13	1 * 13
2	13 2	
^	169	$13^2 = 169$
3	169 3	
+	172	169 + 3 = 172

Result 172

POSTFIX EXPRESSION EVALUATION

- ***Evaluate the following postfix expression***

  532*8+*

postfix evaluation example

postfix expression	stack	output
5 3 2 * 8 + *	5	
5 3 2 * 8 + *	5 3	
5 3 2 * 8 + *	5 3 2	
5 3 2 * 8 + *	5 6	
5 3 2 * 8 + *	5 6 8	
5 3 2 * 8 + *	5 14	
5 3 2 * 8 + *	70	
5 3 2 * 8 + *		70

POSTFIX EXPRESSION EVALUATION

- **Consider the following arithmetic expression written in postfix notation**

10, 2, * , 15, 3, / , + , 12, 3, 2, ↑ , + , + **evaluate this expression to find its val**

Reading	Stack	Evaluated
10	10	
2	10 2	
*	20	10 * 2 = 20
15	20 15	
3	20 15 3	
/	20 5	15 / 3 = 5
+	25	20 + 5
12	25 12	
3	25 12 3	
2	25 12 3 2	
↑	25 12 9	$3^2 = 9$
+	25 21	12 + 9 = 21
+	46	25 + 21 = 46

POSTFIX EXPRESSION EVALUATION

5.3 EVALUATION OF POSTFIX EXPRESSION

- For example, consider the evaluation of the following postfix expression using stacks: **abc+d*f/- where,**
- a=6, b=3, c-6, d=5, f=9 After substituting the values of a, b, c, d and f, the postfix expression becomes 636+5*9/-
- The expression is evaluated as follows:

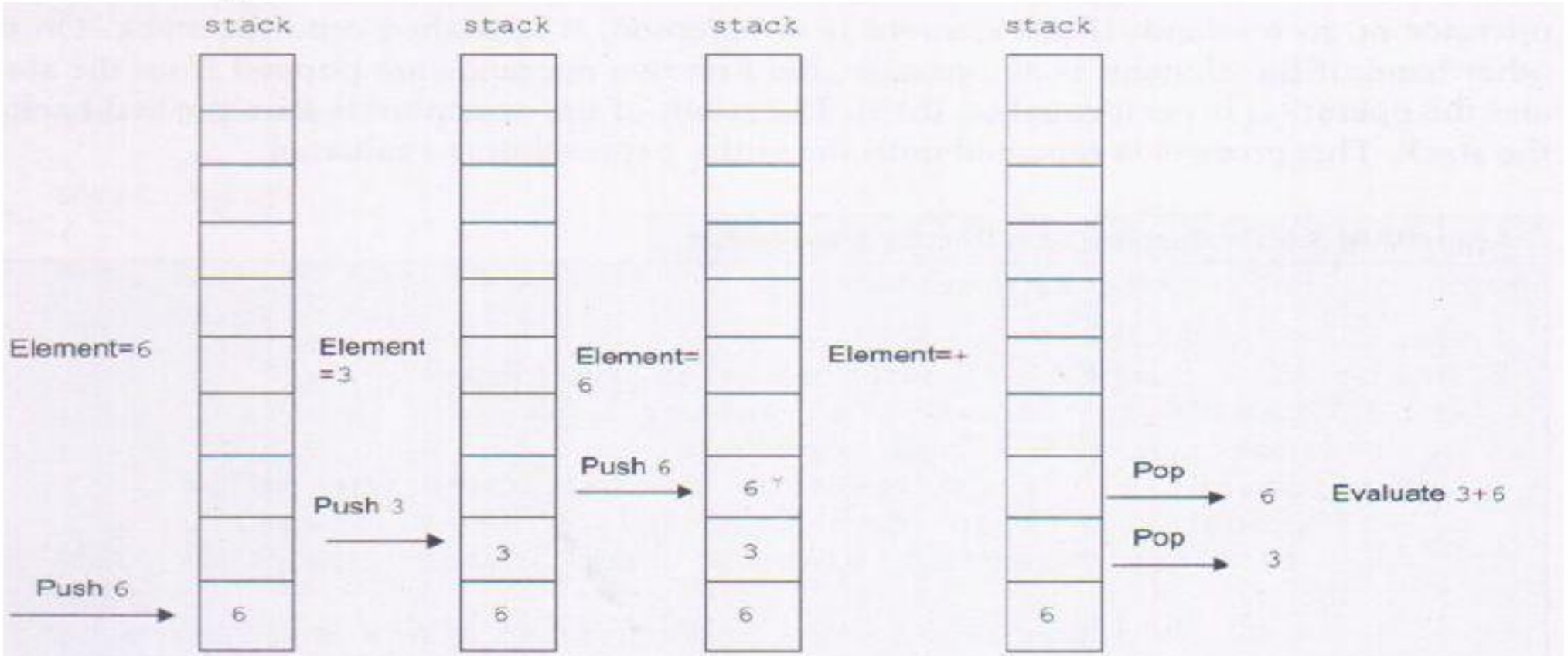


Figure 3.8 Evaluation of the Expression Using Stack

POSTFIX EXPRESSION EVALUATION

Input	Stack	Postfix evaluation
3 4 * 2 5 * +	*empty*	Push 3 into stack
4 * 2 5 * +	3	Push 4 into stack
* 2 5 * +	4 3	Pop 4 & pop 3 from stack do 3*4, Push 12
2 5 * +	12	Push 2 into stack
5 * +	2 12	Push 5 into stack
* +	5 2 12	Pop 5, Pop 2 from stack do 2*5, Push 10 into stack
+	10 12	Pop 10 & Pop 12 from stack do 12 + 10, push 22 into stack
	22	

Algorithm infix into prefix expression

Step 1 : The infix expression is first reversed.

Step 2 : The input string (infix notation) is scanned from left to right.

Step 3 : If the scanned character (ch) is space or tab, it is skipped.

Step 4 : If the scanned character (ch) is digit or alphabet, it is appended to prefix string.

Step 5 : If the scanned character (ch) is closing parenthesis ')', it is pushed to stack.

Step 6 : If the scanned character (ch) is operator :

All operators from top of the stack are popped having more priority than ch and appended to prefix string.

When less priority operator is found, then it is pushed in the stack again with ch.

Step 7 : If the scanned character (ch) is opening parenthesis '(', then all the operators above the closing parenthesis are appended to prefix string.

Step 8 : After evaluations of all the characters, the remaining operators from stack are appended to prefix string.

Convert infix into prefix expression

convert the given infix to prefix expression and show detail of stack.

(A-B/C)*(D*E-F)

First we have to reverse the string

(F – E * D) * (C/B – A)

Table 4.10.1 : Evaluation of Prefix Expression

Character scanned	Stack	Postfix
(	(	–
F	(	F
–	(–	F
E	(–	FE
*	(– *	FE
D	(– *	FED
)	Empty	FED* –
*	*	FED* –
(	* (	FED * –
C	* (	FED * – C
/	* (/	FED * – C
B	* (/	FED * – CB
–	* (–	FED * – CB/
A)	* (–	FED * – CB/A
)	*	FED * – CB/A –
End	Empty	FED * – CB/A – *

Now reverse the expression :

Prefix expression : * – A/BC – * DEF

Convert infix into prefix expression

convert infix string ((A+B) * (C-D))/(E+F) into prefix string with stack.

---→ first reverse the string (F+E)/((D-C) * (B+A))

Character Scanned	Stack	Postfix
(	(	–
F	(	F
+	(+	F
E	(+	FE
)	Empty	FE+
/	/	FE+
(	/(	FE+
(	/((	FE+
D	/((	FE+D
–	/((–	FE+D
C	/((–	FE+DC
)	/(	FE+DC–
*	/(*	FE+DC–
(	/(*(	FE+DC–
B	/(*(	FE+DC–B
+	/(*(+	FE+DC–B
A	/(*(+	FE+DC–BA
)	/(*	FE+DC-BA+
)	/	FE+DC-BA+*
	Empty	FE+DC-BA+*/

Reverse the string

Result will be : /* + AB – CD + EF

Convert infix into prefix expression

Example: infix to prefix conversion

Convert infix expression A-B/C*(C*D$E) to prefix expression.

Scan char	Prefix Stack	Opstack
)		)
E	E	)
$	E	)$
D	ED	)$
*	ED$	)*
C	ED$C	)*
(	ED$C*	
*	ED$C*	*
C	ED$C*C	*
/	ED$C*C	*/
B	ED$C*CB	*/
-	ED$C*CB/*	-
A	ED$C*CB/*A	-
ϕ	ED$C*CB/*A-	

- **The Prefix String of given infix expression is : -A*/BC*C$DE**

Convert prefix into postfix expr.

ALGORITHMS :

- Read the Prefix expression in reverse order (from right to left)
- If the symbol is an operand, then push it onto the Stack
- If the symbol is an operator, then pop two operands from the StackCreate a string by concatenating the two operands and the operator after them.string = operand1 operand2 operator And push the resultant string back to Stack.
- Repeat the above steps until end of Prefix expression.

Convert prefix into postfix expr.

Convert the following prefix expression into postfix expression

***+a-bc/-de+-fgh**

Expression scanned from right to left.

Prefix	Stack	Postfix
*+a-bc/-de+-fgh	-	–
*+a-bc/-de+-fg	h	–
*+a-bc/-de+-f	gh	–
*+a-bc/-de+-	fgh	–
*+a-bc/-de+	fg-h	fg-
*+a-bc/-de	fg-h+	fg-h+
*+a-bc/-d	e fg-h+	fg-h+
*+a-bc/-	d e fg-h+	fg-h+
*+a-bc/	de- fg-h+	fg-h+

Mr. Anand Gharu(MET's IOE, Nashik)

Prefix	Stack	Postfix
*+a-bc	de-fg-h+/	de-fg-h+/
*+a-b	c de-fg-h+/	de-fg-h+/
*+a-	b c de-fg-h+/	de-fg-h+/
*+a	bc- de-fg-h+/	de-fg-h+/
*+	a bc- de-fg-h+/	de-fg-h+/
*	abc-+ de-fg-h+/	de-fg-h+/
Empty	Empty	abc-+ de-fg-h+/*

∴ Postfix expression = AB+D*EF–↑

Convert prefix into infix expr.

Example : * + a – bc / – d e + – fgh

Prefix expression	Stack
* + a – bc / – d e + – f g h	
* + a – bc / – d e + – f g	h
* + a – bc / – d e + – f	g h
* + a – bc / – d e + –	f g h
* + a – bc / – d e +	f – g h
* + a – bc / – d e	f – g + h
* + a – bc / – d	e f – g + h
* + a – bc / –	d e f – g + h
* + a – bc /	d – e f – g + h
* + a – bc	(d–e) / (f – g + h)
* + a – b	c (d–e) / (f – g + h)
* + a –	b c (d–e) / (f – g + h)
* + a	b – c (d–e) / (f – g + h)
* +	a b – c (d–e) / (f – g + h)
*	a + b – c (d–e) / (f – g + h)
End	(a + b – c) * (d–e) / (f – g + h)

Convert following into postfix expr.

Convert the following expression into postfix expression.

(i) ((a + b) − (c * d)) / (e / (f ∧ 2))

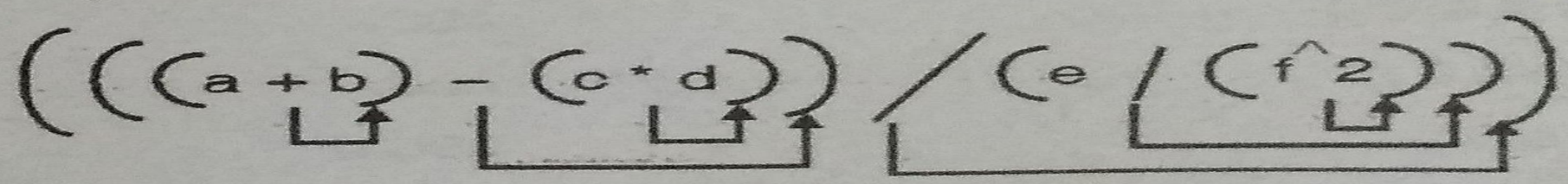
(ii) (a + b) * d + e / (f + a * d) + c

Solution :

(i) Given expression :

((a + b) − (c * d)) /(e/ (f ∧ 2))

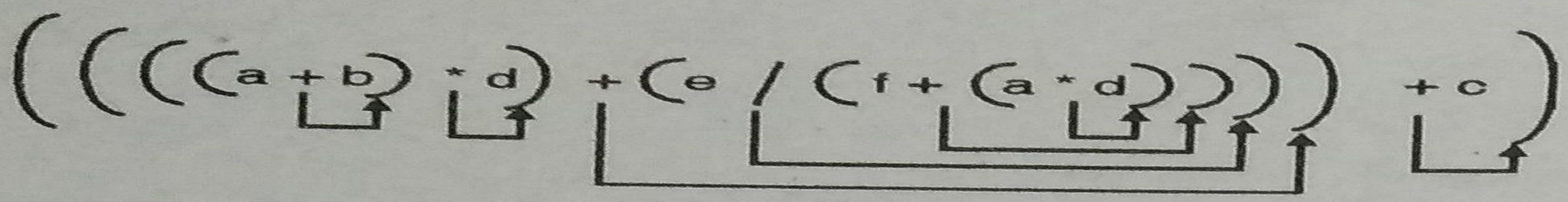
Fully parenthesizing the expression and moving the operator to its corresponding right parenthesis.



Postfix expression = ab + cd * − e f 2 ∧ //

(ii) (a + b) * d + e/ (f + a + d) + c

Fully parenthesizing the expression and moving the operator to its corresponding right parenthesis.



(dsa10.13(b))**Fig. Ex. 5.5.14(b)**

Postfix expression = ab + d * e f a d * + / + c +

Convert fully parenthesis into prefix expr.

Input	Operator stack	Output stack
(((a ∧ b) * c) + d)		
(((a ∧ b) * c) + d		
(((a ∧ b) * c) +		d
(((a ∧ b) * c)	+	d
(((a ∧ b) * c	+	d
(((a ∧ b) *	+	c d
(((a ∧ b)	* +	c d
(((a ∧ b	* +	c d
(((a ∧	* +	b c d
(((a	∧ * +	b c d
(((	∧ * +	a b c d
((	* +	∧ a b c d
(	+	* ∧ a b c d
End		+ * ∧ a b c d

Prefix expression : + * ∧ abcd

Convert fully parenthesis into postfix expr.

Input	Stack	Output
(((a ∧ b) * c) + (d * e/f)))		–
((a ∧ b) * c) + (d * e/f)))		–
(a ∧ b) * c) + (d * e/f)))		–
a ∧ b) * c) + (d * e/f)))		–
∧ b) * c) + (d * e/f)))		a
b) * c) + (d * e/f)))		a

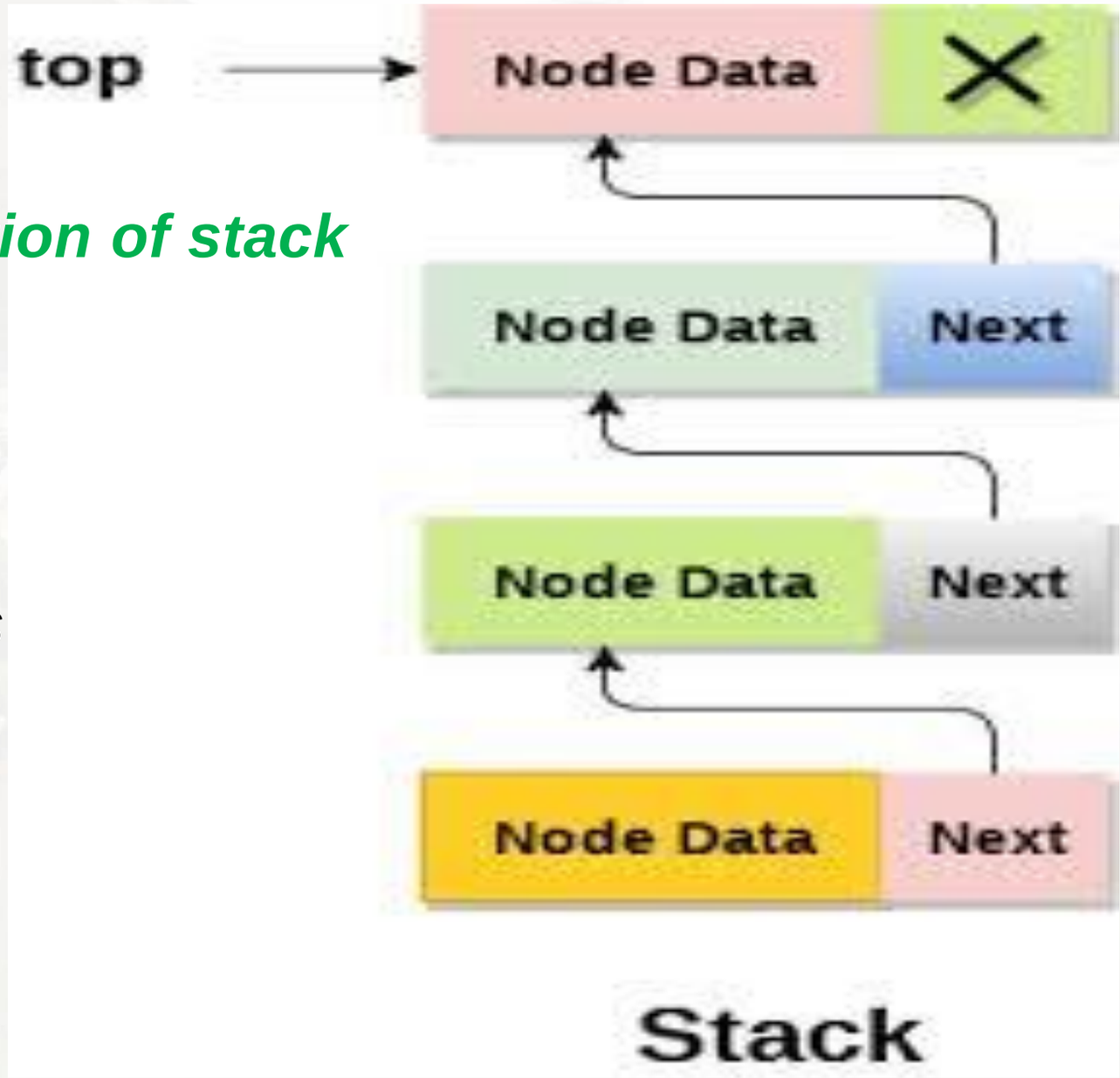
Input	Stack	Output
) * c) + (d * e/f)))	∧	a
* c) + (d * e/f)))		ab
c) + (d * e/f)))	*	ab∧
) + (d * e/f)))	*	ab∧
+ (d * e/f)))		ab∧c*
(d * e/f)))	+	ab∧c *

Convert fully parenthesis into postfix expr.

d * e/f)))	+	ab∧c *
* e/f)))	+	ab∧c *d
e/f)))	* +	ab∧c *d
/f)))	* +	ab^c * de
f)))	/ * +	ab∧c * de
)))	/ * +	ab∧c * def
)	* +	ab∧c * def /
End		ab∧c * def / *

Linked stack operation

In stack elements are placed one above other. In the same manner in stack as linked list, we place node one above other.



Advantages of dynamic implementation of stack :

1. *No memory wastage*
2. *No memory shortage*
3. *No limitation on number of elements*

Stack operation using linked list

```
#include <iostream>
using namespace std;

struct Node {
   int data;
   struct Node *next;
};
struct Node* top = NULL;

void push(int val) {
   struct Node* newnode = (struct Node*) malloc(sizeof(struct Node));
   newnode->data = val;
   newnode->next = top;
   top = newnode;
}
```

Stack operation using linked list

```
void pop() {
   if(top==NULL)
      cout<<"Stack Underflow"<<endl;
   else {
      cout<<"The popped element is "<< top->data <<endl;
      top = top->next;
   }
}
```

Stack operation using linked list

```
void display() {
   struct Node* ptr;
   if(top==NULL)
      cout<<"stack is empty";
   else {
      ptr = top;
      cout<<"Stack elements are: ";
      while (ptr != NULL) {
         cout<< ptr->data <<" ";
         ptr = ptr->next;
      }
   }
   cout<<endl;
}
```

Stack operation using linked list

```
int main() {
   int ch, val;
   cout<<"1) Push in stack"<<endl;
   cout<<"2) Pop from stack"<<endl;
   cout<<"3) Display stack"<<endl;
   cout<<"4) Exit"<<endl;
   do {
      cout<<"Enter choice: "<<endl;
      cin>>ch;
      switch(ch) {
         case 1: {
            cout<<"Enter value to be pushed:"<<endl;
            cin>>val;
            push(val);
            break;
```

Stack operation using linked list

```
case 2: {
        pop();
        break;
    }
    case 3: {
        display();
        break;
    }
    case 4: {
        cout<<"Exit"<<endl;
        break;
    }
    default: {
            cout<<"Invalid Choice"<<endl;
        }
    }
  }while(ch!=4);
    return 0;
}
```

RECURSION IN STACK

"Calling function inside itself is called as recursion. Such function is called as recursive function"

How recursion works?

How recursion works ?

```
void recurse()
{
   ...  ..  ...
   recurse();
   ...  ..  ...
}

int main()
{
   ...  ..  ...
   recurse();
   ...  ..  ...
}
```

recursive call

- The execution of recursion continues unless and until some specific condition is met to prevent its repetition.
- For the purpose of preventing infinite recursion, if...else statement (or similar approach) can be used in which one branch go for the recursive call while other doesn't.

RECURSION IN STACK

Advantages :

1. It helps to reduce size of program

2. Easy to maintain function calling

3. Evaluation of stack can be through recursion

Disadvantages :

1. It takes more time bcz of stack overlapping

2. Stack overflow may ocuur

3. Memory requirement is more

4. Efficiency is less

Backtracking algorithm strategy:

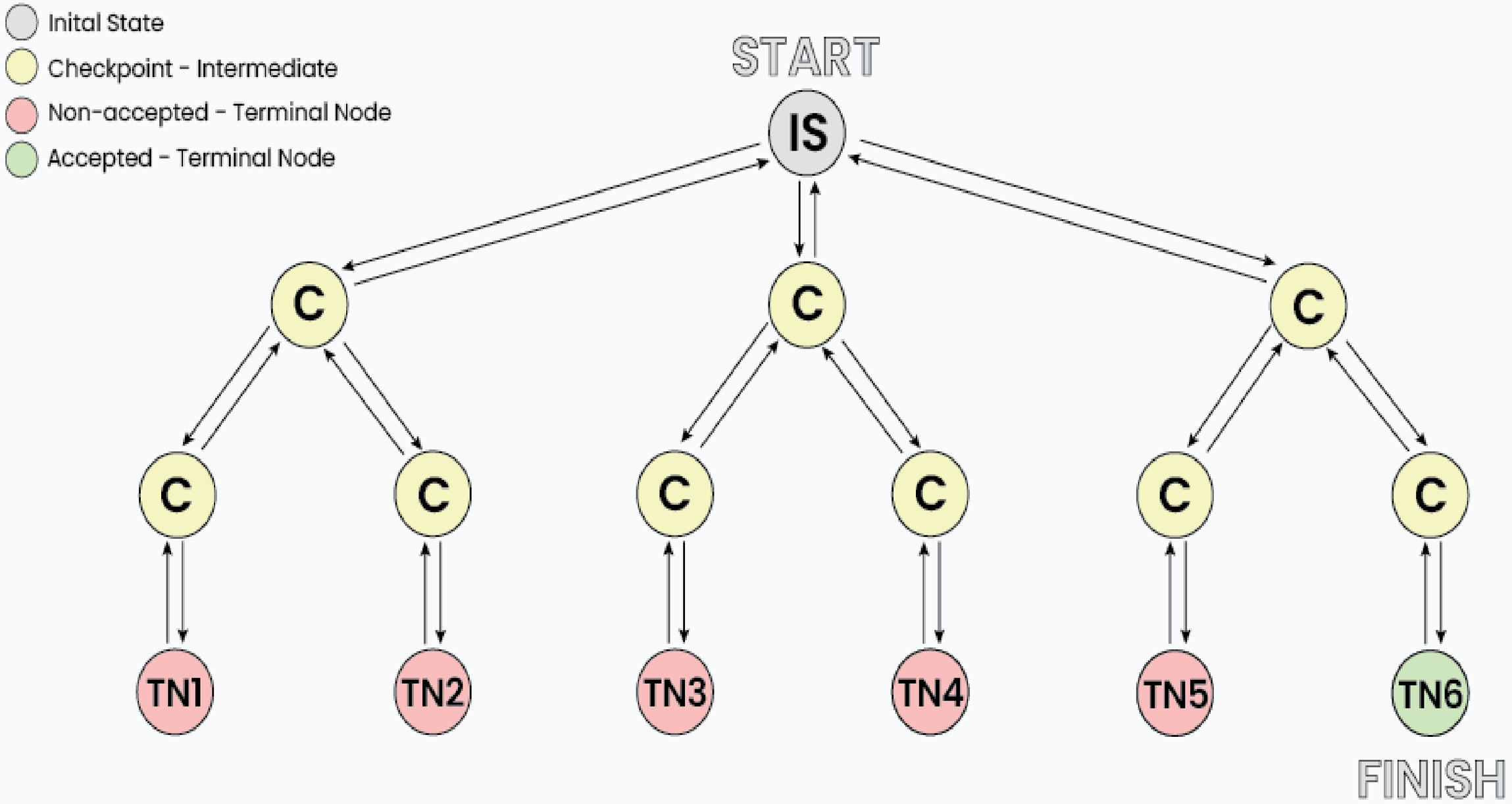
“Backtracking can be defined as a general algorithmic technique that considers searching every possible combination in order to solve a computational problem.”

Backtracking is a problem-solving algorithmic technique that involves finding a solution incrementally by trying different options and undoing them if they lead to a dead end. It is commonly used in situations where you need to explore multiple possibilities to solve a problem, like searching for a path in a maze or solving puzzles like Sudoku. When a dead end is reached, the algorithm backtracks to the previous decision point and explores a different path until a solution is found or all possibilities have been exhausted

Backtracking algorithm strategy:

How does Backtracking works?

As we know backtracking algorithm explores each and every possible path in order to find a valid solution, this exploration of path can be easily understood via given images:



Backtracking algorithm strategy:

Complexity Analysis of Backtracking

Since backtracking algorithm is purely brute force therefore in terms of time complexity, it performs very poorly. Generally backtracking can be seen having below mentioned time complexities:

Exponential (O(K^N))

Factorial (O(N!))

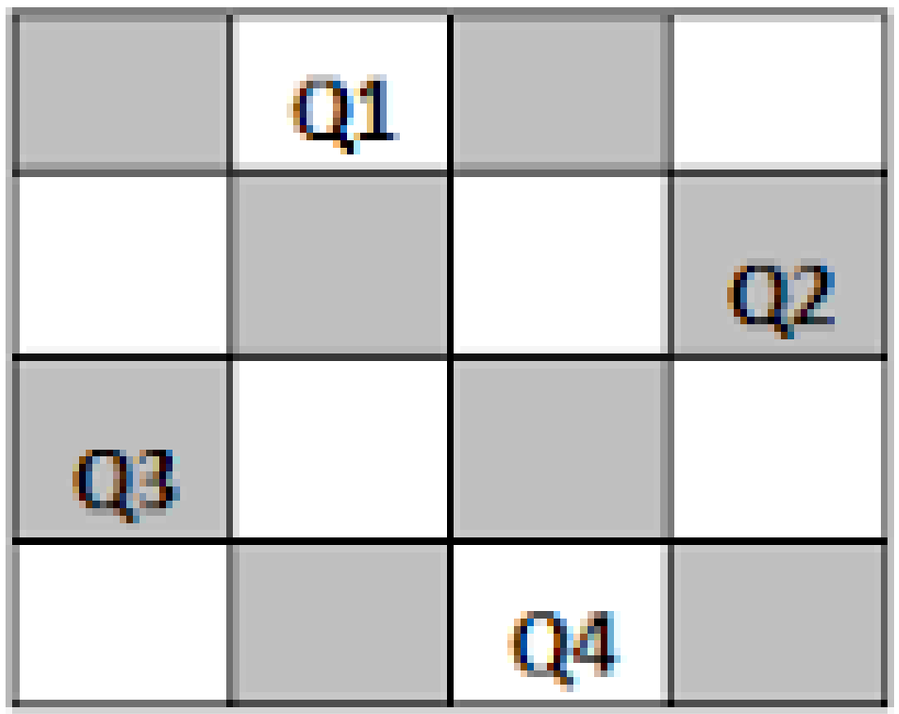
Backtracking algorithm strategy:

Applications of Backtracking :

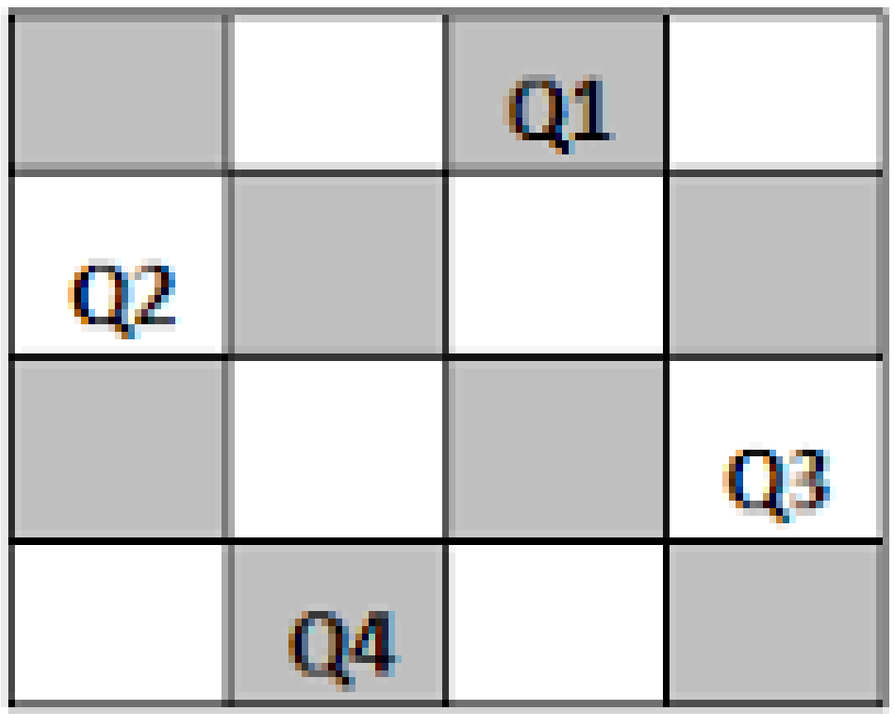
1. Creating smart bots to play Board Games such as Chess.
2. Solving mazes and puzzles such as N-Queen problem.
3. Network Routing and Congestion Control.
4. Decryption
5. Text Justification
6. Prim’s and Kruskal’s Algorithms

4-QUEEN PROBLEM

The N Queen is the problem of placing N chess queens on an N×N chessboard so that no two queens attack each other.



Solution 1



Solution 2

Recursion Vs Backtracking

Recursion	Backtracking
Recursion does not always need backtracking	Backtracking always uses recursion to solve problems
Solving problems by breaking them into smaller, similar subproblems and solving them recursively.	Solving problems with multiple choices and exploring options systematically, backtracking when needed.
Controlled by function calls and call stack.	Managed explicitly with loops and state.
Applications of Recursion: Tree and Graph Traversal, Towers of Hanoi, Divide and Conquer Algorithms, Merge Sort, Quick Sort, and Binary Search.	**Application of Backtracking:** N Queen problem, Prim's Algorithms, Kruskal's Algorithms etc.

QUEUE

Queue

- Basic concept
- Queue as Abstract Data Type
- Representation of Queue using Sequential Organization
- Queue Operations
- Circular Queue and its advantages
- Multi-queues,Linked Queue and Operations
- Deque-Basic concept, types (Input restricted and Output restricted)
- Priority Queue-Basic concept, types (Ascending and Descending).

Introduction of Queue

- "A queue is **an ordered list** in which all insertions are done at one end, called rear and deletions at another end called front.
- Queue is referred to be as First In First Out list.
- For example, people waiting in line for a rail ticket form a queue.
- Queue when implemented using arrays has some drawbacks which can be avoided by circular queue
- Queue is used in many applications such **as *as simulation, priority queue, job queue etc***

Introduction of Queue

- One of the most **common data processing structures**
- Frequently used in most of the system software's like operating systems, Network and Database implementations and in many more other areas

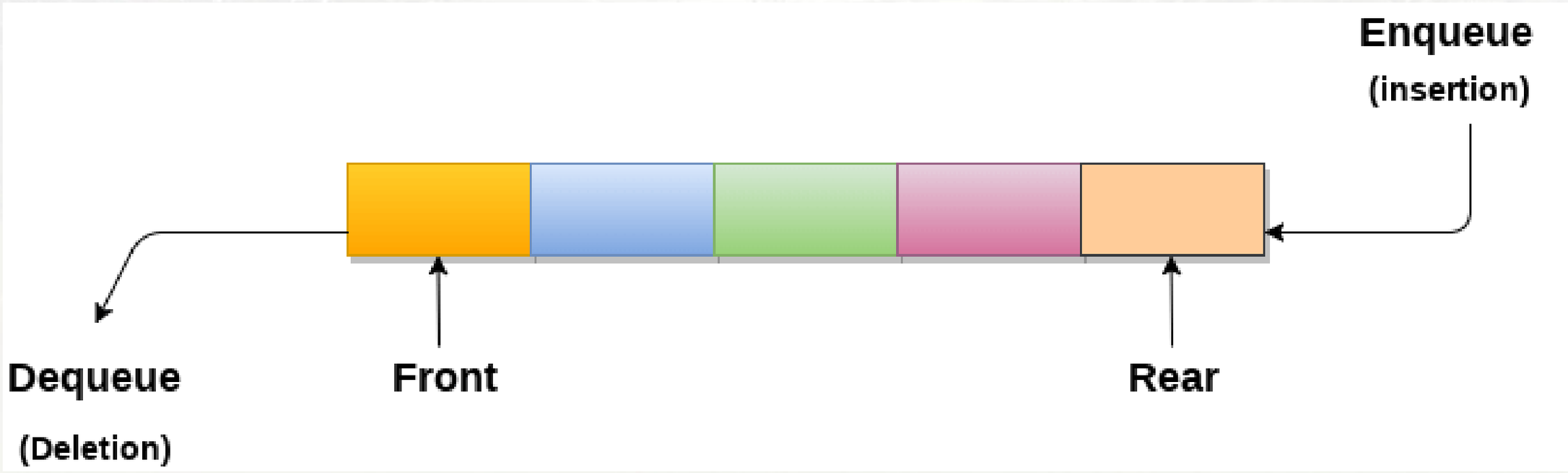


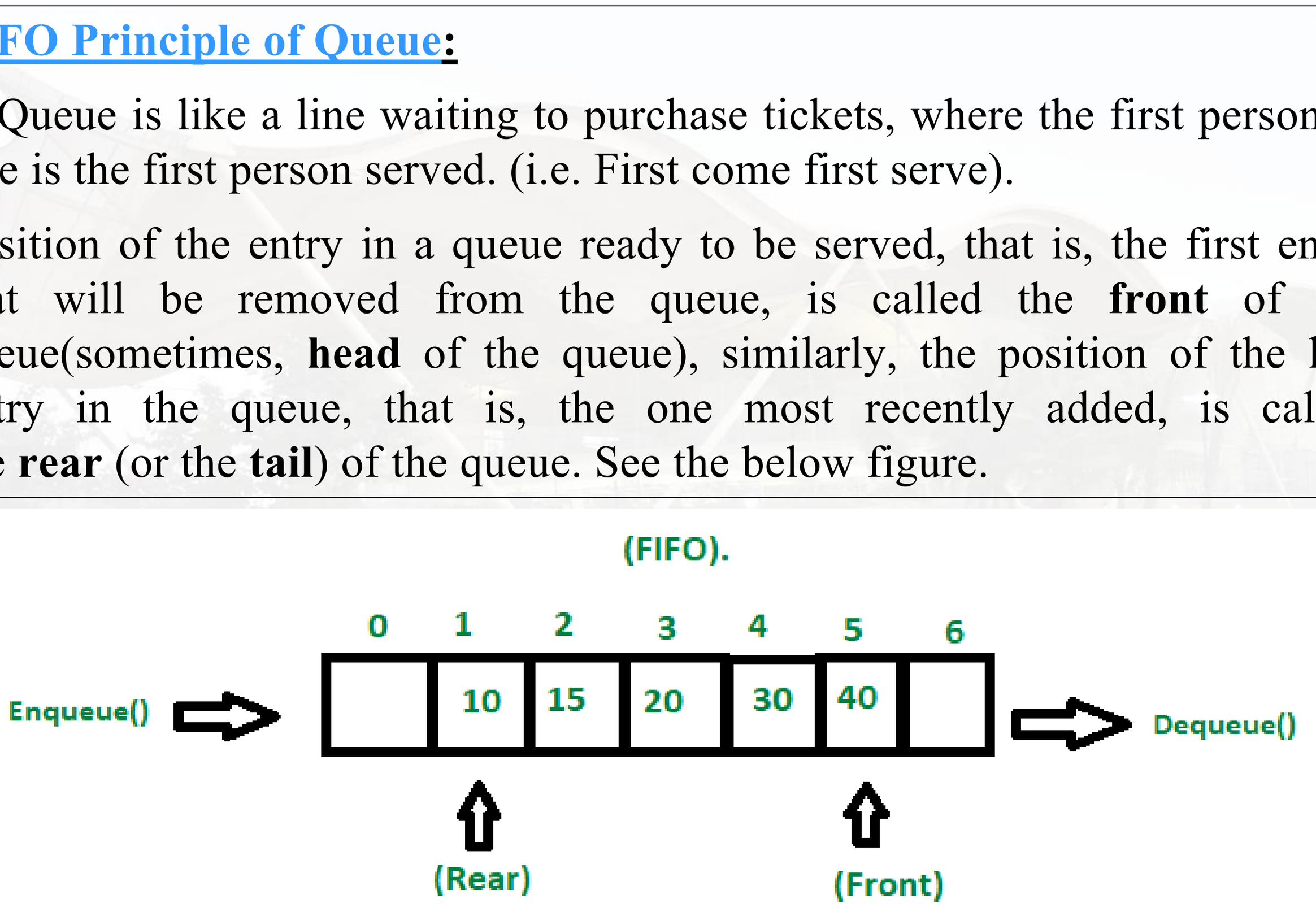
Fig : Normal/Regular Queue presentation

Introduction of Queue

FIFO Principle of Queue:

A Queue is like a line waiting to purchase tickets, where the first person in line is the first person served. (i.e. First come first serve).

Position of the entry in a queue ready to be served, that is, the first entry that will be removed from the queue, is called the **front** of the queue(sometimes, **head** of the queue), similarly, the position of the last entry in the queue, that is, the one most recently added, is called the **rear** (or the **tail**) of the queue. See the below figure.



Basic Operations of Queue

Queue operations work as follows:

- Two pointers FRONT and REAR
- FRONT track the first element of the queue
- REAR track the last element of the queue
- initially, set value of FRONT and REAR to -1

Operations performed on Queue

1. **Enqueue()** – Adds (or stores) an element to the end of the queue..
2. **Dequeue()** – Removal of elements from the queue.
3. **Peek() or front()-** Acquires the data element available at the front node of the queue without deleting it.
4. **rear()** – This operation returns the element at the rear end without removing it.
5. **isFull()** – Validates if the queue is full.
6. **isNull()** – Checks if the queue is empty.

Representation of Queue

- **Ways to implement the queue**
- **There are two ways of implementing the Queue:**

1. **Implementation using array:** The sequential allocation in a Queue can be implemented using an array.

2. **Implementation using Linked list:** The linked list allocation in a Queue can be implemented using a linked list.

Queue using Array Representation

1. Array Representation of Queue:

- Like stacks, Queues can also be represented in an array: In this representation, the Queue is implemented using the array. Variables used in this case are

- **Queue:** the name of the array storing queue elements.
- **Front:** the index where the first element is stored in the array representing the queue.
- **Rear:** the index where the last element is stored in an array representing the queue.

Operation on queue implemented using array

```
void Insert() {
    int val;
    if (rear == n - 1)
    cout<<"Queue Overflow"<<endl;
    else {
        if (front == - 1)
        front = 0;
        cout<<"Insert the element in queue : "<<endl;
        cin>>val;
        rear++;
        queue[rear] = val;
    }
}
```

Operation on queue implemented using array

```
void Delete() {
    if (front == - 1 || front > rear) {
        cout<<"Queue Underflow ";
        return ;
    }
    else {
        cout<<"Element deleted from queue is : "<< queue[front] <<endl;
        front++;;
    }
}
```

Operation on queue implemented using array

```
void Display() {
    if (front == - 1)
    cout<<"Queue is empty"<<endl;
    else {
        cout<<"Queue elements are : ";
        for (int i = front; i <= rear; i++)
        cout<<queue[i]<<" ";
        cout<<endl;
    }
}
```

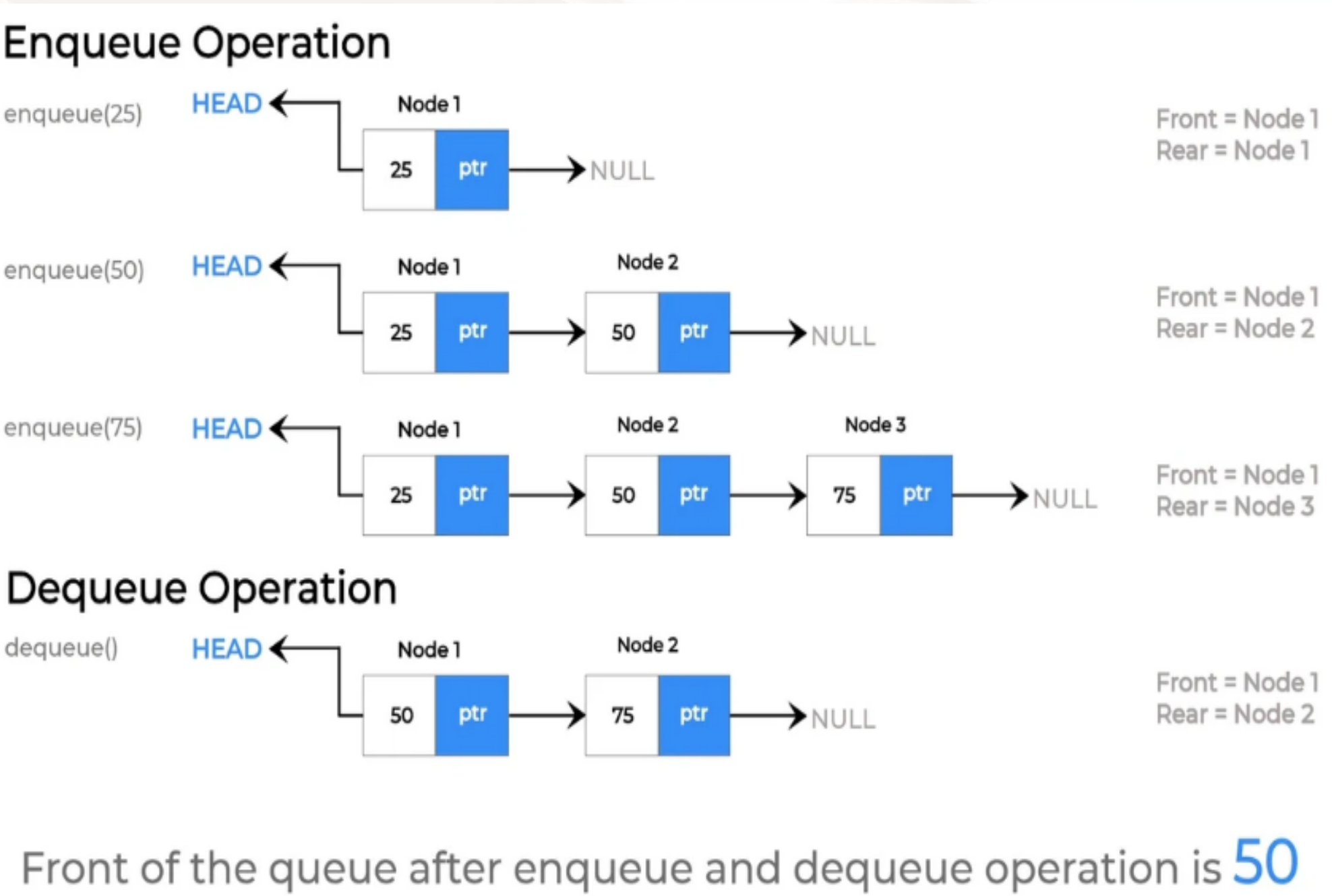
https://www.tutorialspoint.com/cplusplus-program-to-implement-queue-using-array

Queue using Linked Representation

Linked List Representation of Queue:

A queue can also be represented using following entities:

- Linked-lists,
- Pointers, and
- Structures.



Queue using Linked Representation

Algorithm for Enqueue:-

- Return Queue full
- Else
- Rear+Rear+1
- Queue[Rear]=Data
- If(Front== -1)
- Front= 0

Algorithm for Dequeue:-

1. If(Front==-1||Front==Rear+1)
2. Return
3. Else
4. Queue[Front]=0
5. Front=Front+1

Queue Implementation using Linked List

```
//function to enter elements in queue
void enqueue ( int value )
{
 Node *ptr = new Node();
 ptr->data= value;
 ptr->link = NULL;

 //If inserting the first element/node
 if( front == NULL )
 {
  front = ptr;
  rear = ptr;
 }
 else
 {
  rear ->link = ptr;
  rear = ptr;
 }
}
```

Queue Implementation using Linked List

```
//function to delete/remove element from queue
void dequeue ( )
{
 if( isempty() )
 cout<<"Queue is empty\n";
 else
 //only one element/node in queue.
 if( front == rear)
 {
  free(front);
  front = rear = NULL;
 }
else
 {
  Node *ptr = front;
  front = front->link;
  free(ptr);
 }
}
```

Queue Implementation using Linked List

```
//function to display queue
void displayQueue()
{
 if (isempty())
  cout<<"Queue is empty\n";
 else
 {
 Node *ptr = front;
 while( ptr !=NULL)
 {
  cout<<ptr->data<<" ";
  ptr= ptr->link;
 }
}
}
```

Applications of Queue

1. Queues are widely used as waiting lists for a single shared resource like printer, disk, CPU.
2. Queues are used in asynchronous transfer of data (where data is not being transferred at the same rate between two processes) for eg. pipes, file IO, sockets.
3. Queues are used as buffers in most of the applications like MP3 media player, CD player, etc.
4. Queue are used to maintain the play list in media players in order to add and remove the songs from the play-list.
5. Queues are used in operating systems for handling interrupts.

Types of Queue

A queue is a useful data structure in programming. It is similar to the ticket queue outside a cinema hall, where the first person entering the queue is the first person who gets the ticket.

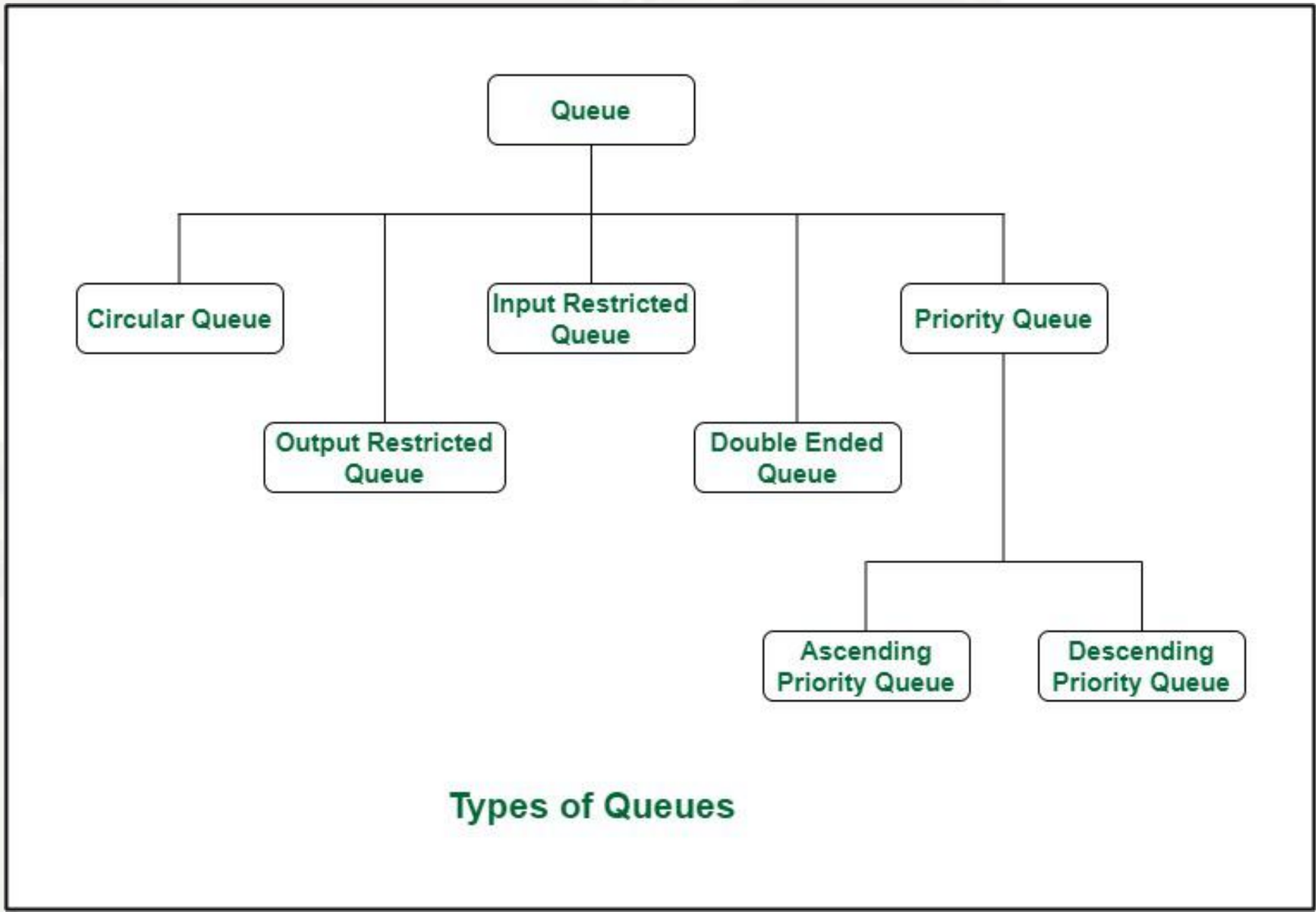
There are four different types of queues:

Simple Queue

Circular Queue

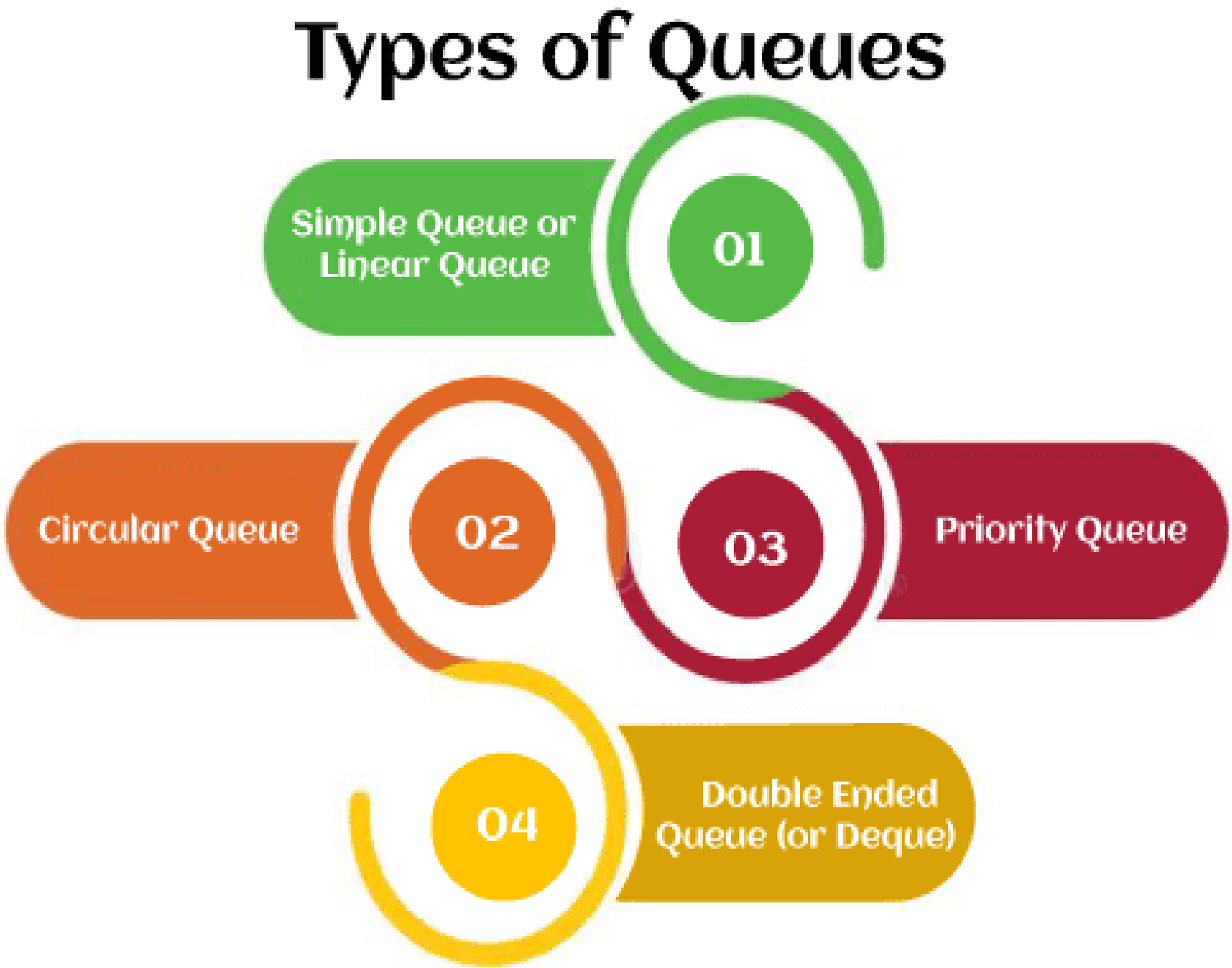
Priority Queue

Double Ended Queue



Types of Queues

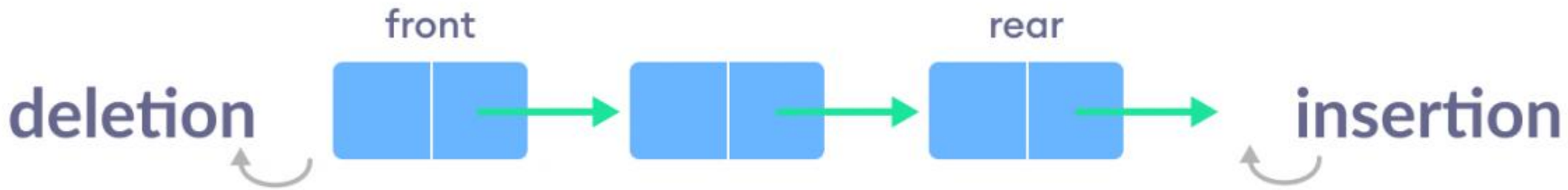
Types of Queue



Types of Queue

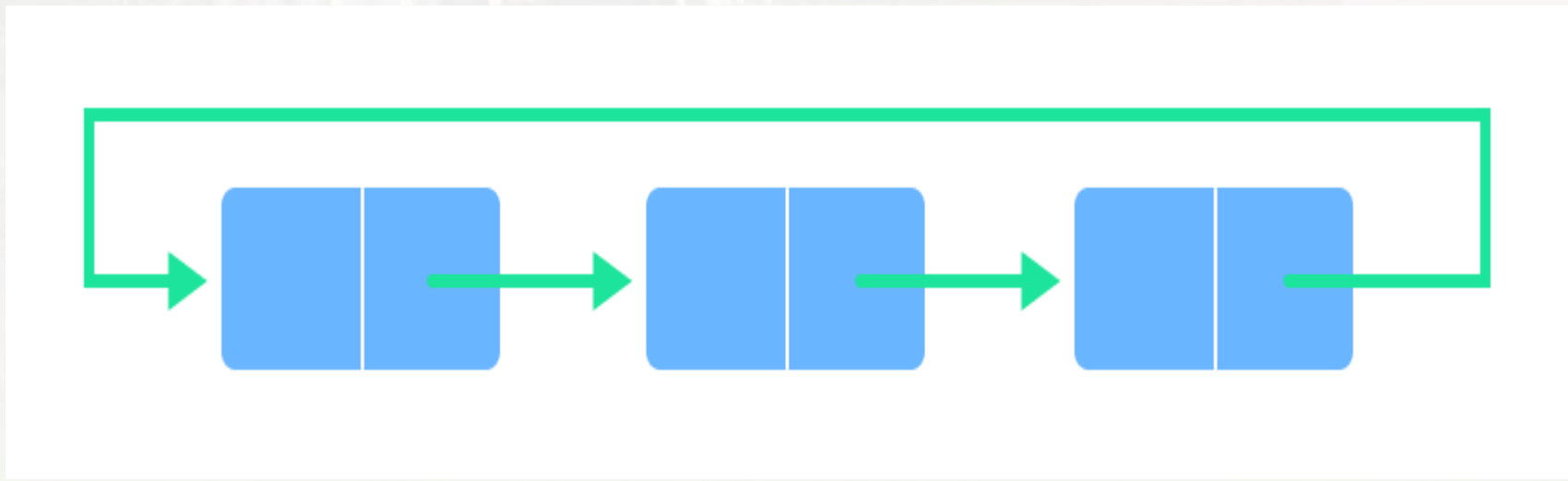
Simple Queue

In a simple queue, insertion takes place at the rear and removal occurs at the front. It strictly follows the FIFO (First in First out) rule.



Circular Queue

In a circular queue, the last element points to the first element making a circular link.



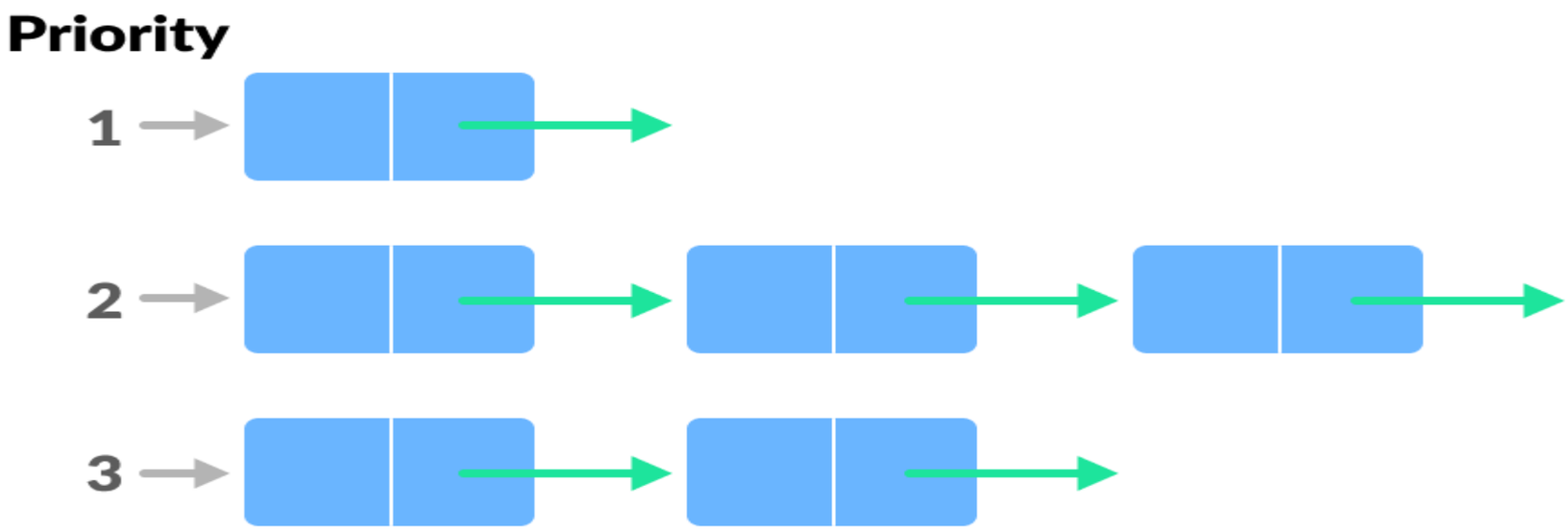
Types of Queue

Circular Queue :

The main advantage of a circular queue over a simple queue is better memory utilization. If the last position is full and the first position is empty, we can insert an element in the first position. This action is not possible in a simple queue.

Priority Queue

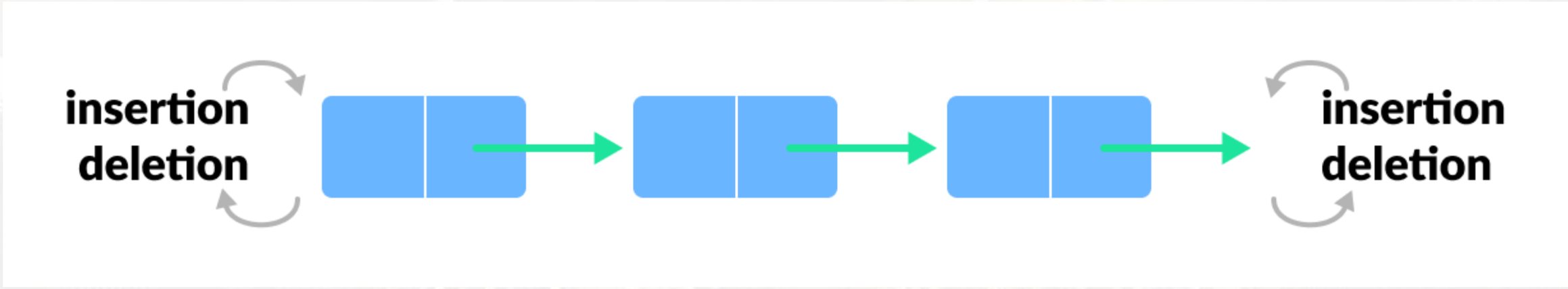
A priority queue is a special type of queue in which each element is associated with a priority and is served according to its priority. If elements with the same priority occur, they are s



Types of Queue

Deque (Double Ended Queue):

In a double ended queue, insertion and removal of elements can be performed from either from the front or rear. Thus, it does not follow the FIFO (First In First Out) rule.



Array representation and implementation of queue

- An array representation of queue require three entities :

1. An array to hold queue element
2. A variable to hold index of the front element
3. A variable to hold index of the rear element

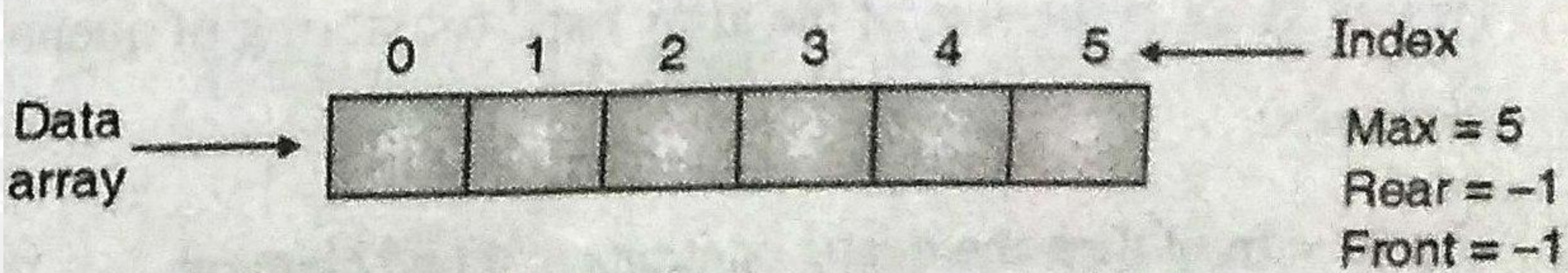
A queue datatype may be defined formally as follows :

```
# define MAX 30
typedef struct queue
{
        int data[MAX];
   int front, rear;
} queue;
```

Example of Queue using array : https://www.tutorialspoint.com/cplusplus-program-to-implement-queue-using-array

Array representation and implementation of queue

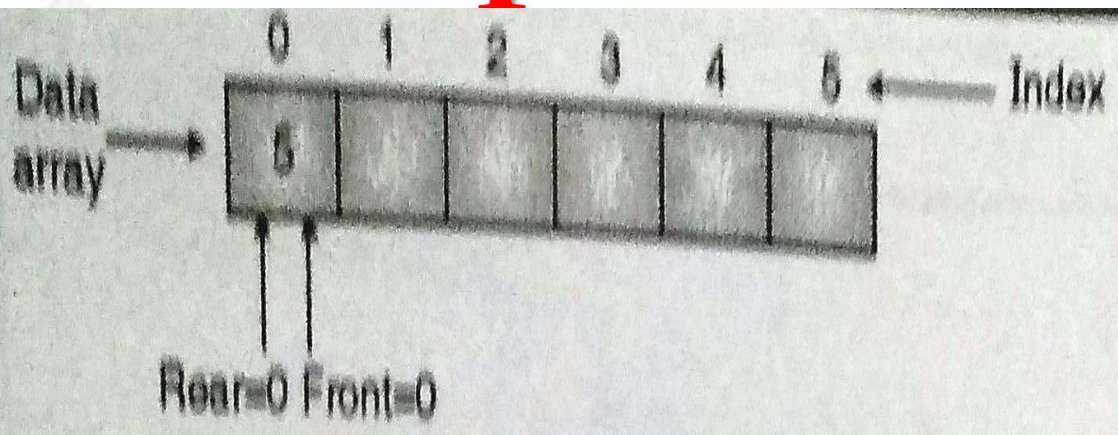
During initialization of a queue, its front and rear are set to –1.



(dsa11.2)**Fig. 6.1.2 : An empty queue after initialization**

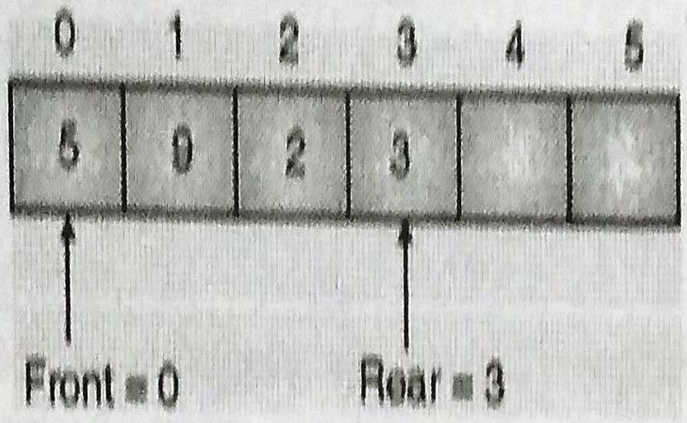
Fig. 6.1.3 shows the status of a queue after insertion of the element '5'.

Array representation and implementation of queue

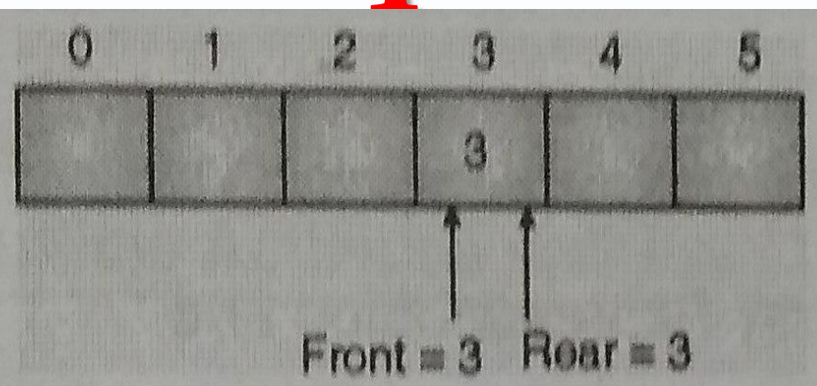


(dsa11.5)**Fig. 6.1.3 : A queue after insertion of the first element '5'**

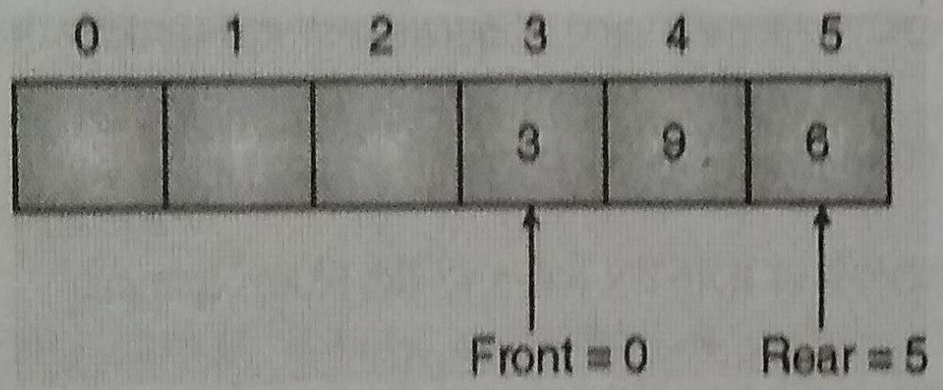
- On subsequent insertions, front remains at the same place, where rear advances.



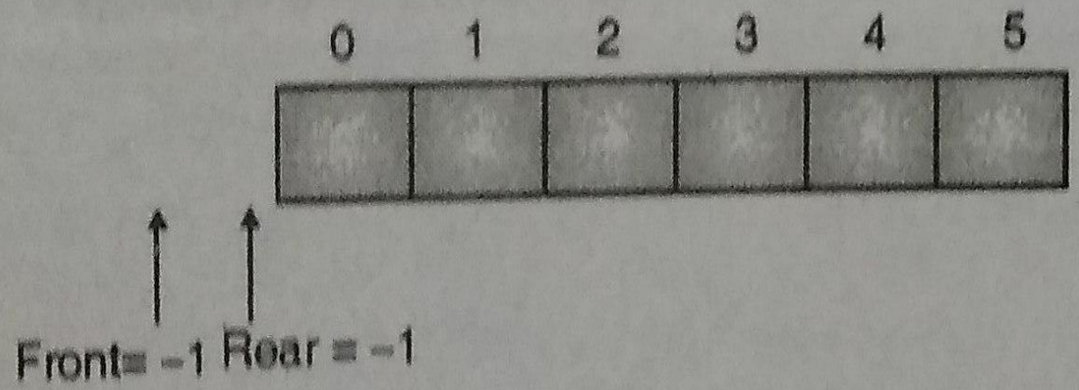
(dsa11.4)**Fig. 6.1.4 : A queue after insertion of four elements**



(dsa11.5)**Fig. 6.1.5 : A queue after 3 successive deletions**



(dsa11.6)**Fig. 6.1.6 : A queue after 2 successive insertions**



(dsa11.7)**Fig. 6.1.7 : A queue after 3 successive deletions**

STACK VS QUEUE

#	STACK	QUEUE
1	Objects are inserted and removed at the same end.	Objects are inserted and removed from different ends.
2	In stacks only one pointer is used. It points to the top of the stack.	In queues, two different pointers are used for front and rear ends.
3	In stacks, the last inserted object is first to come out.	In queues, the object inserted first is first deleted.
4	Stacks follow Last In First Out (LIFO) order.	Queues following First In First Out (FIFO) order.
5	Stack operations are called push and pop.	Queue operations are called enqueue and dequeue.
6	Stacks are visualized as vertical collections.	Queues are visualized as horizontal collections.
7	Collection of dinner plates at a wedding reception is an example of stack.	People standing in a file to board a bus is an example of queue.

Queue Example

Example 6.2.1

Consider the following queue of characters, implemented as array of six memory locations :-

Front = 2, Rear = 3

Queue : –, A, D, –, –, –

Where '–' denotes empty cell. Describe the queue as the following operations take place

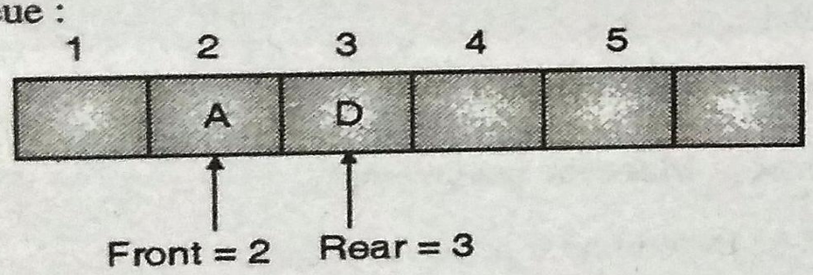
(i) Add 'S' (ii) Add 'J'

(iii) Delete two letters

(iv) Shift towards left to bring all free spaces to the right side

(v) Insert M, H, I and delete on letter.

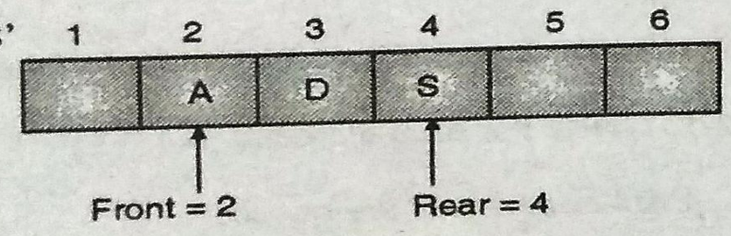
Initial queue :



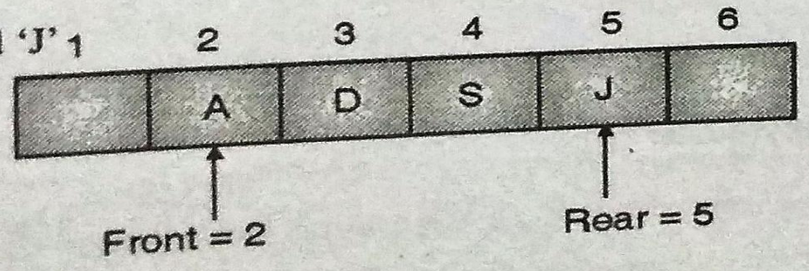
(a)

(a)

(i) Add 'S'

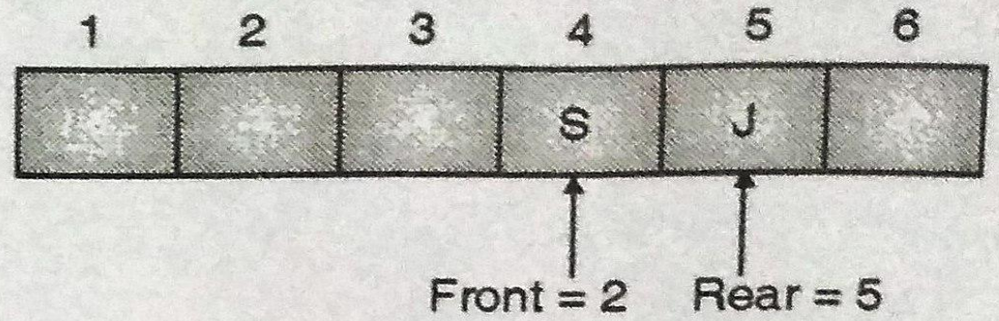


(ii) Add 'J'



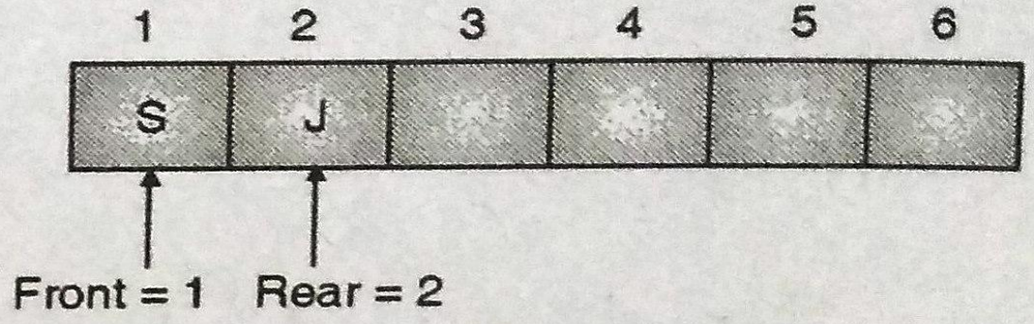
(c)

(iii) Delete two letters



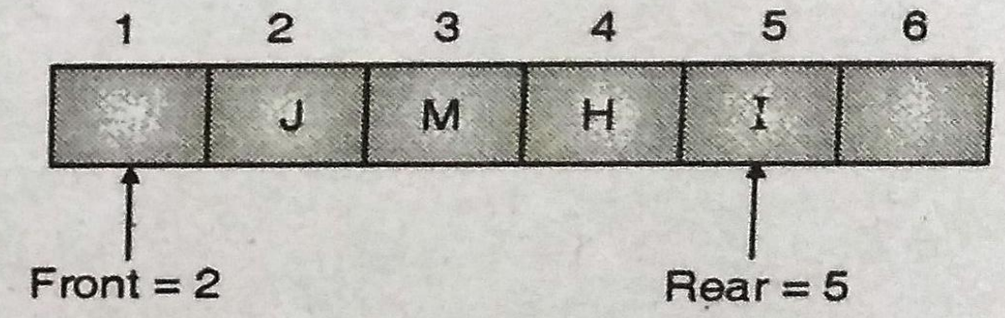
(d)

(iv) Shift towards left to bring all free spaces to the right.



(e)

(v) Insert M, H, I and delete one letter.



(f)

(dsa11.11)**Fig. Ex. 6.2.1**

Queue Example

Q. 5.2.2 Consider the following queue, where queue is a circular queue having 6 memory cells.

Front = 2, Rear = 4.

Queue : __, A, C, D,__,__

Describe queue as following operation take place :

F is added to the queue

Two letters are deleted

R is added to the queue

S is added to the queue

One letter is deleted **(4 Marks)**

Soln. :

	A	C	D		
	F		R		

F is added

	A	C	D	F	
	F			R	

Two letters are deleted

			D	F	
			F	R	

R is added to queue

			D	F	R
			F		R

S is added to queue

S			D	F	R
R			F		

One letter is deleted

S				F	R
R				F	

Queue Example

Example 3.2.5

A linear queue using array has a size of 3. Perform the following on this queue and show the sequence of steps with necessary diagrams indicating values of front, rear and contents of queue :

(i) insert 10

(ii) insert 20

(iii) insert 30

(iv) delete an element

(v) insert 40

(vi) delete an element

(vii) delete an element

(viii) delete an element

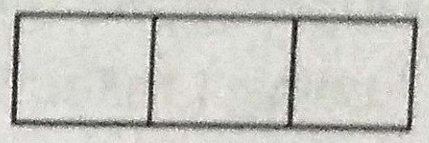
(ix) insert 50

Queue Example

Solution :

Operation — **Queue**

Initial

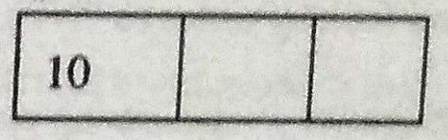


↑

rear, front

= –1 = –1

Insert 10

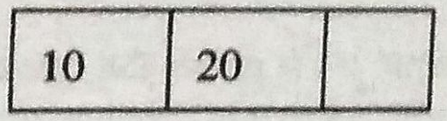


↑ ↑

rear front

= 0 = 0

Insert 20

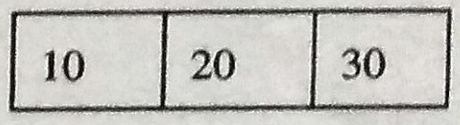


↑ ↑

front rear

= 0 = 1

Insert 30

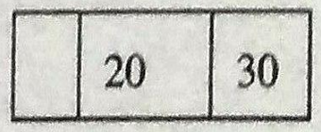


↑ ↑

front rear

= 0 = 2

Delete an element

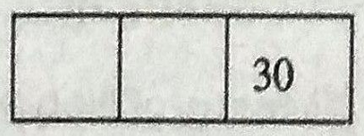


↑ ↑

Front rear

= 1 = 2

Delete an element

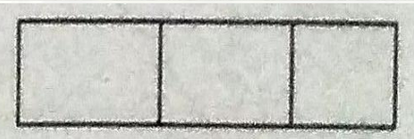


↑ ↑

front rear

= 2 = 2

Delete an element



↑ ↑

front, rear

= –1 = –1

Insert 50

↑ ↑

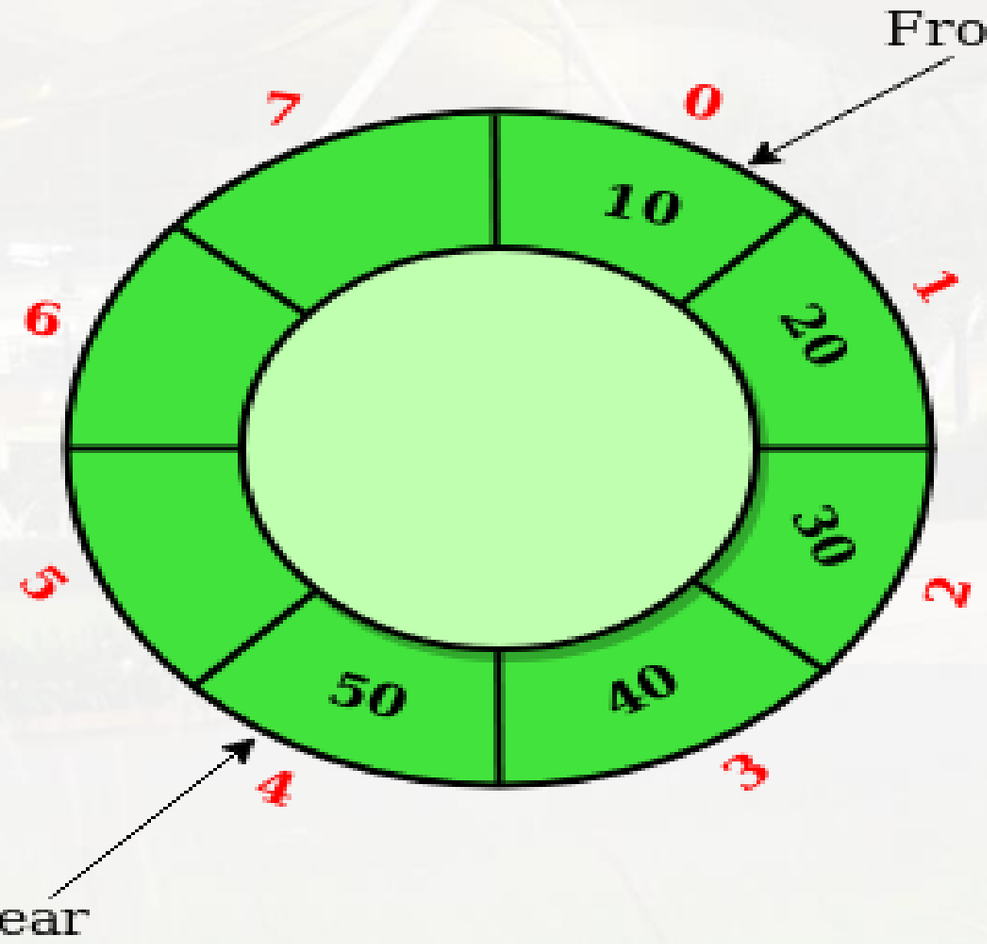
front rear

= 0 = 0

Circular Queue

A Circular Queue is an extended version of a normal queue where the last element of the queue is connected to the first element of the queue forming a circle.

The operations are performed based on FIFO (First In First Out) principle. It is also called ‘Ring Buffer’.



In a normal Queue, we can insert elements until queue becomes full. But once queue becomes full, we can not insert the next element even if there is a space in front of queue.

Operation on Circular Queue

1. **Front:** Get the front item from the queue.
2. **Rear**: Get the last item from the queue.
3. **enQueue(value)** This function is used to insert an element into the circular queue. In a circular queue, the new element is always inserted at the rear position.

Check whether the queue is full – [i.e., the rear end is in just before the front end in a circular manner].

If it is full then display Queue is full.

If the queue is not full then, insert an element at the end of the queue.

4. deQueue() This function is used to delete an element from the circular queue. In a circular queue, the element is always deleted from the front position.

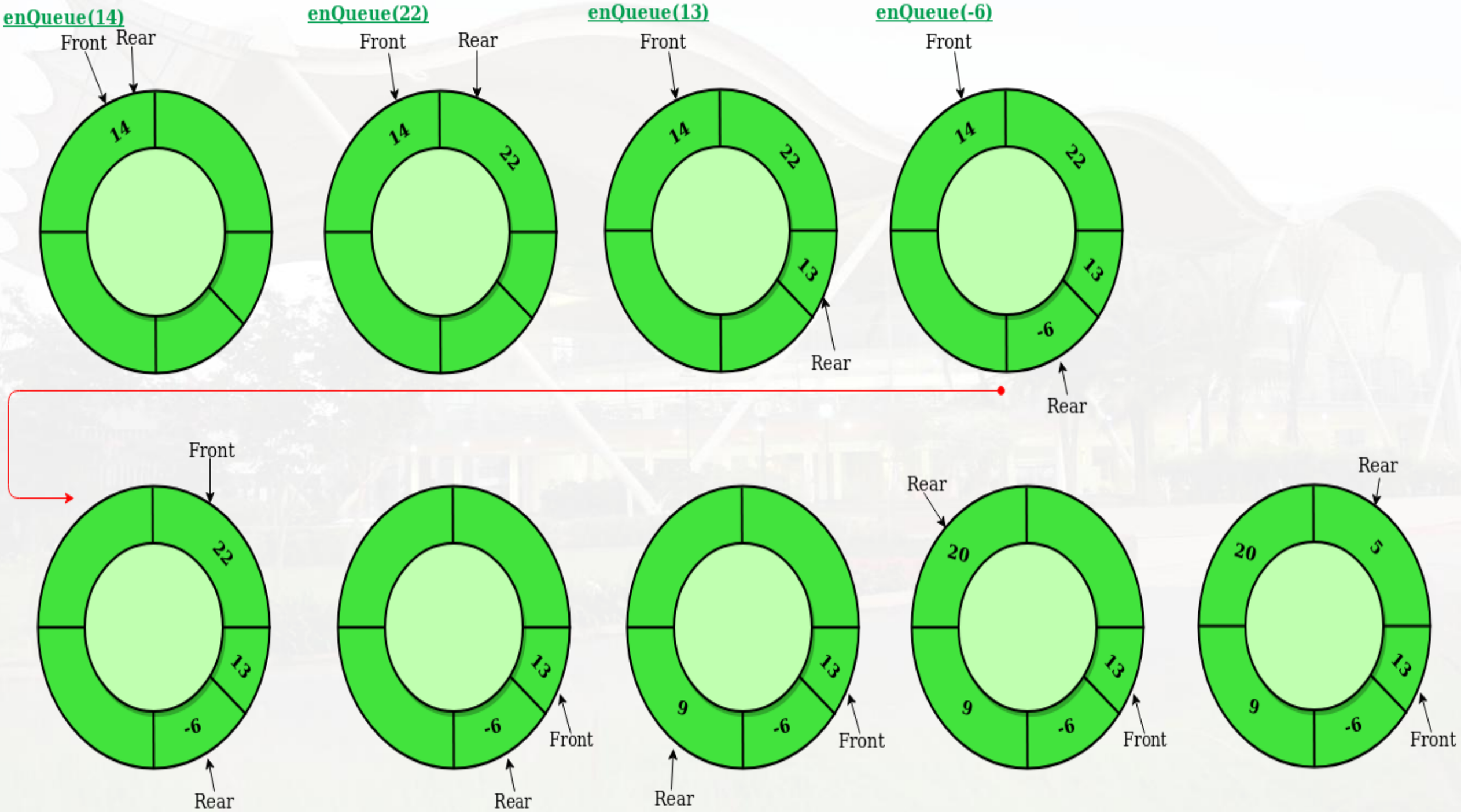
Check whether the queue is Empty.

If it is empty then display Queue is empty.

If the queue is not empty, then get the last element and remove it from the queue.

Operation on Circular Queue

Illustration of Circular Queue Operations:



Operation on Circular Queue

A circular queue can be implemented using two data structures:

- Array
- Linked List

Implement Circular Queue using Array

1. Initialize an array queue of size n, where n is the maximum number of elements that the queue can hold.
2. Initialize two variables front and rear to -1.
3. **Enqueue:** To enqueue an element x into the queue, do the following:

- Increment rear by 1.

    If rear is equal to n, set rear to 0.

- If front is -1, set front to 0.
- Set queue[rear] to x.

4. **Dequeue:** To dequeue an element from the queue, do the following:

- Check if the queue is empty by checking if front is -1.

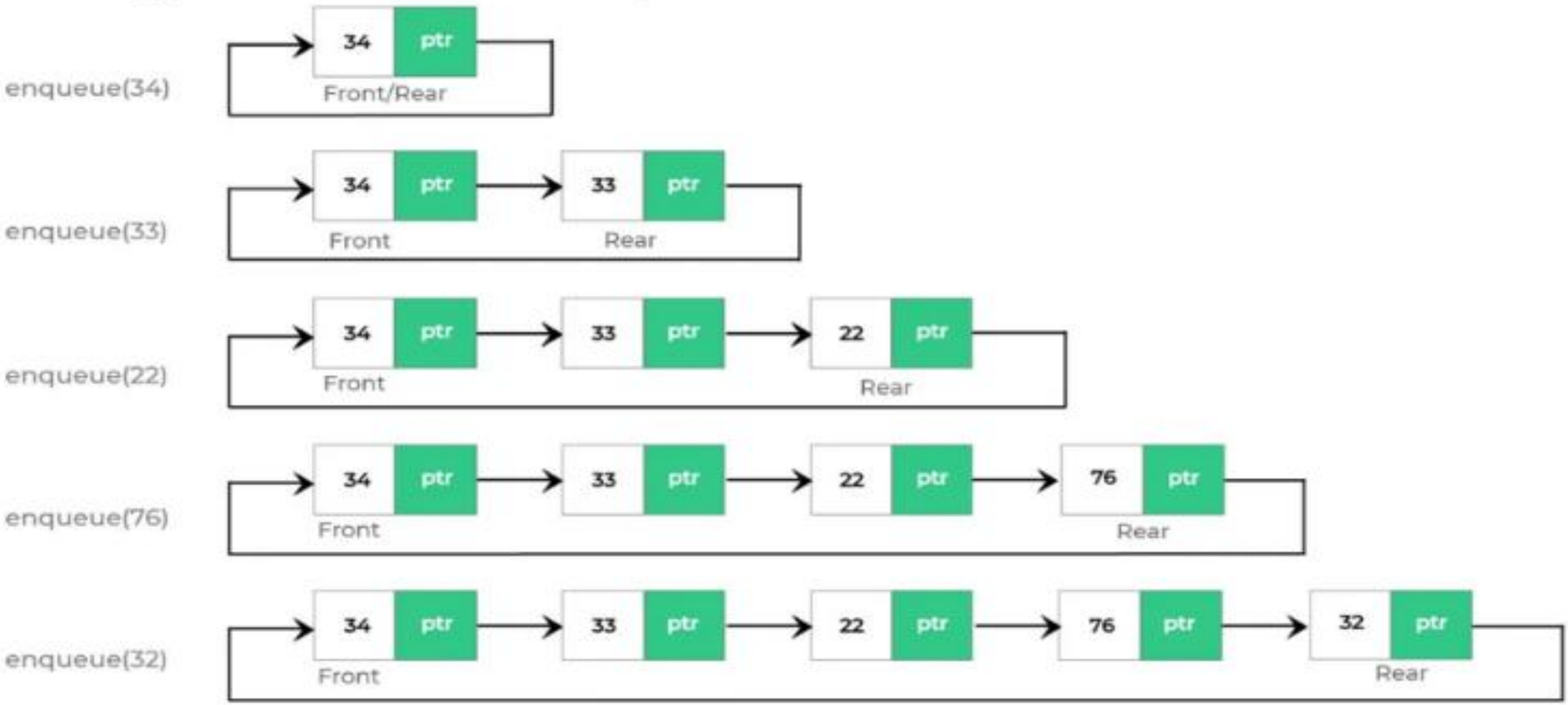
    If it is, return an error message indicating that the queue is empty.

- Set x to queue[front].
- If front is equal to rear, set front and rear to -1.
- Otherwise, increment front by 1 and if front is equal to n, set front to 0.
- Return x.

Implement Circular Queue using Linked

1. Asd

Adding the elements into Queue



Removing the elements from Queue

Implement Circular Queue using Linked

```
void enqueue (int data)
  {
   Node *newNode = new Node ();
   newNode->data = data;
   newNode->next = nullptr;
   if (front == nullptr)
     {
      front = rear = newNode;
      rear->next = front;
     }
   else
     {
      rear->next = newNode;
      rear = newNode;
      rear->next = front;
     }
   cout << data << " has been enqueued." << endl;
}
```

Implement Circular Queue using Linked

```
void dequeue ()
  {
   if (front == nullptr)
     {
      cout << "Queue is empty." << endl;
     }
   else if (front == rear)
     {
      Node *temp = front;
      front = rear = nullptr;
      delete temp;
     }
  else
     {
      Node *temp = front;
      front = front->next;
      rear->next = front;
      delete temp;
     }
}
```

Implement Circular Queue using Linked

```
void display ()
 {
  if (front == nullptr)
    {
     cout << "Queue is empty." << endl;
    }
  else
    {
     Node *temp = front;
    cout << "Circular Queue: ";
    do
      {
        cout << temp->data << " ";
        temp = temp->next;
      }
    while (temp != front);
    cout << endl;
   }
}
```

Circular Queue

Consider a circular queue of size 5 having initial status as :

Front	Rear	Circular queue				
		0	1	2	3	4
1	2		P	Q		

Show the value of front, rear and the contents of circular queue after every step in tabular form for the following operations : (1) R is added (2) Delete 2 letters (3) S, T, U are added (4) Three letters are deleted (5) V is added

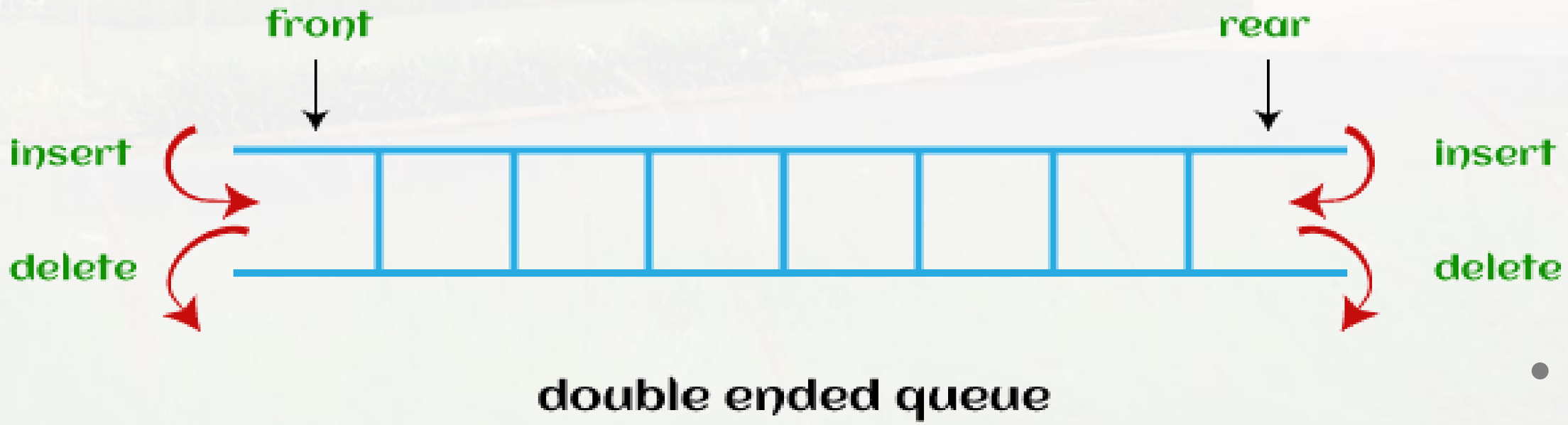
Operation	Front	Rear	Circular queue
Initial	1	2	0: ; 1: P; 2: Q; 3: ; 4:
R is added	1	3	0: ; 1: P; 2: Q; 3: R; 4:
Delete 2 letters	3	3	0: ; 1: ; 2: ; 3: R; 4:
S,T,U are added	3	1	0: T; 1: U; 2: ; 3: R; 4: S
Three letters are deleted	1	1	0: ; 1: U; 2: ; 3: ; 4:
V is added	1	2	0: ; 1: U; 2: V; 3: ; 4:

DeQueue

Deque (Double Ended Queue):

In a double ended queue, insertion and removal of elements can be performed from either from the front or rear. Thus, it does not follow the FIFO (First In First Out) rule.

- It means that we can insert and delete elements from both front and rear ends of the queue.
- Deque can be used as a palindrome checker means that if we read the string from both ends, then the string would be the same.
- The representation of the deque is shown in the below image -



double ended queue

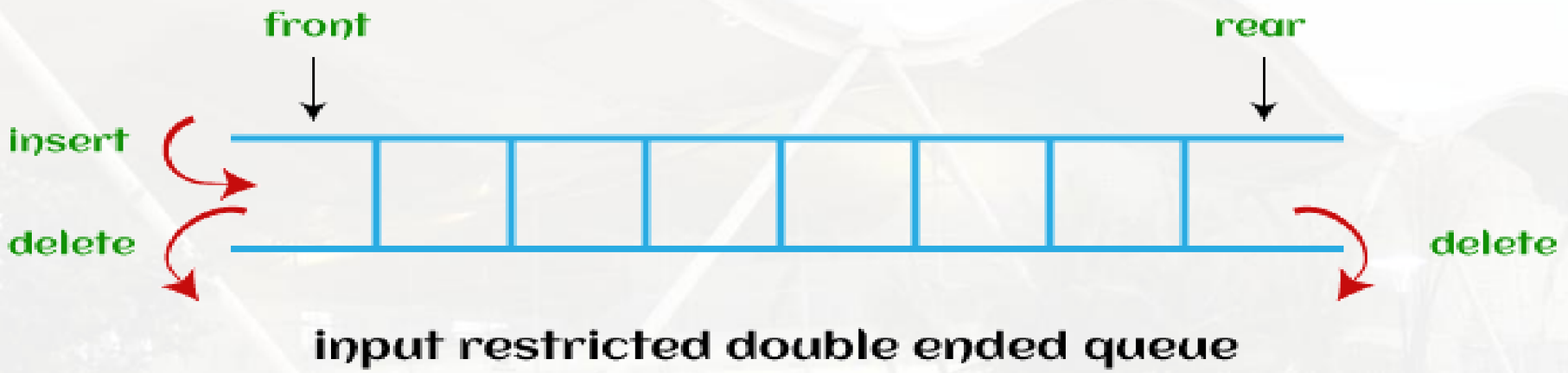
Operation of DeQueue

1. **Count() :** This function is used to count the total number of elements in the deque in data structure. This can be done by iterating over the whole deque in data structure.
2. **addFront() :** This function is used to insert an element at the front end of the deque in data structure.
3. **addRear() :** This function is used to insert element at the rear end of the deque in data structure
4. **delFront():** This function is used to delete elements from the front end of the deque in data structure
5. **delRear():** This function is used to delete elements from the rear end of the deque in data structure
6. **display():** This function is used to get or peek all the elements from the deque in data structure

Types of Queue

There are two types of deque that are discussed as follows –

Input restricted deque - As the name implies, in input restricted queue, insertion operation can be performed at only one end, while deletion can be performed from both ends.



input restricted double ended queue

Output restricted deque - As the name implies, in output restricted queue, deletion operation can be performed at only one end, while insertion can be performed from both ends.

Output restricted double ended queue

Advantages of Deque

1. **Dynamic Size:** Deques can grow or shrink dynamically.
2. **Efficient Operations:** Deques provide efficient O(1) time complexity for inserting and removing elements from both ends.
3. **Versatile:** Deques can be used as stacks (LIFO) or queues (FIFO), or as a combination of both.
4. **No Reallocation:** Deques do not require reallocation of memory when elements are inserted or removed.
5. **Thread Safe:** Deques can be thread-safe if used with proper synchronization.

Disadvantages of Deque

1. **Memory Overhead:** Deques have higher memory overhead compared to other data structures due to the extra pointers used to maintain the double-ended structure.
2. **Synchronization:** Deques can cause synchronization issues if not used carefully in multi-threaded environments.
3. **Complex Implementation:** Implementing a deque can be complex
4. **Not All Platforms:** Deques may not be supported by all platforms
5. **Not Suitable for Sorting:** Deques are not designed for sorting or searching, as these operations require linear time.
6. **Limited Functionality:** Deques have limited functionality compared to other data structures such as arrays, lists, or trees.

Applications of Deque

1. Deque can be used as both **stack and queue**, as it supports both operations.
2. Deque can be used as a **palindrome checker** means that if we read the string from both ends, the string would be the same.
3. It is used in **job scheduling** algorithms..
4. **Graph traversal :** Deques can be used to implement Breadth-First Search (BFS) on a graph.
5. In a **web browser's history**, recently visited URLs are added to the front of the deque and the URL at the back of the deque is removed after some specified number of operations of insertions at the front.

DeQueue as an ADT

```
#define MAX 30 /* A queue with maximum of 30 elements */
typedef struct DQ
{
        int data [MAX];
        int rear, front;
} DQ;
```

☞ **Operations on a dequeue**

i) initialize( ) : Make the queue empty.

ii) empty( ) : Determine if queue is empty.

iii) full( ) : Determine if queue is full.

iv) enqueueF( ) : Insert an element at the front end of the queue.

v) enqueueR( ) : Insert an element at the rear end of the queue.

vi) dequeueR( ) : Delete the rear element.

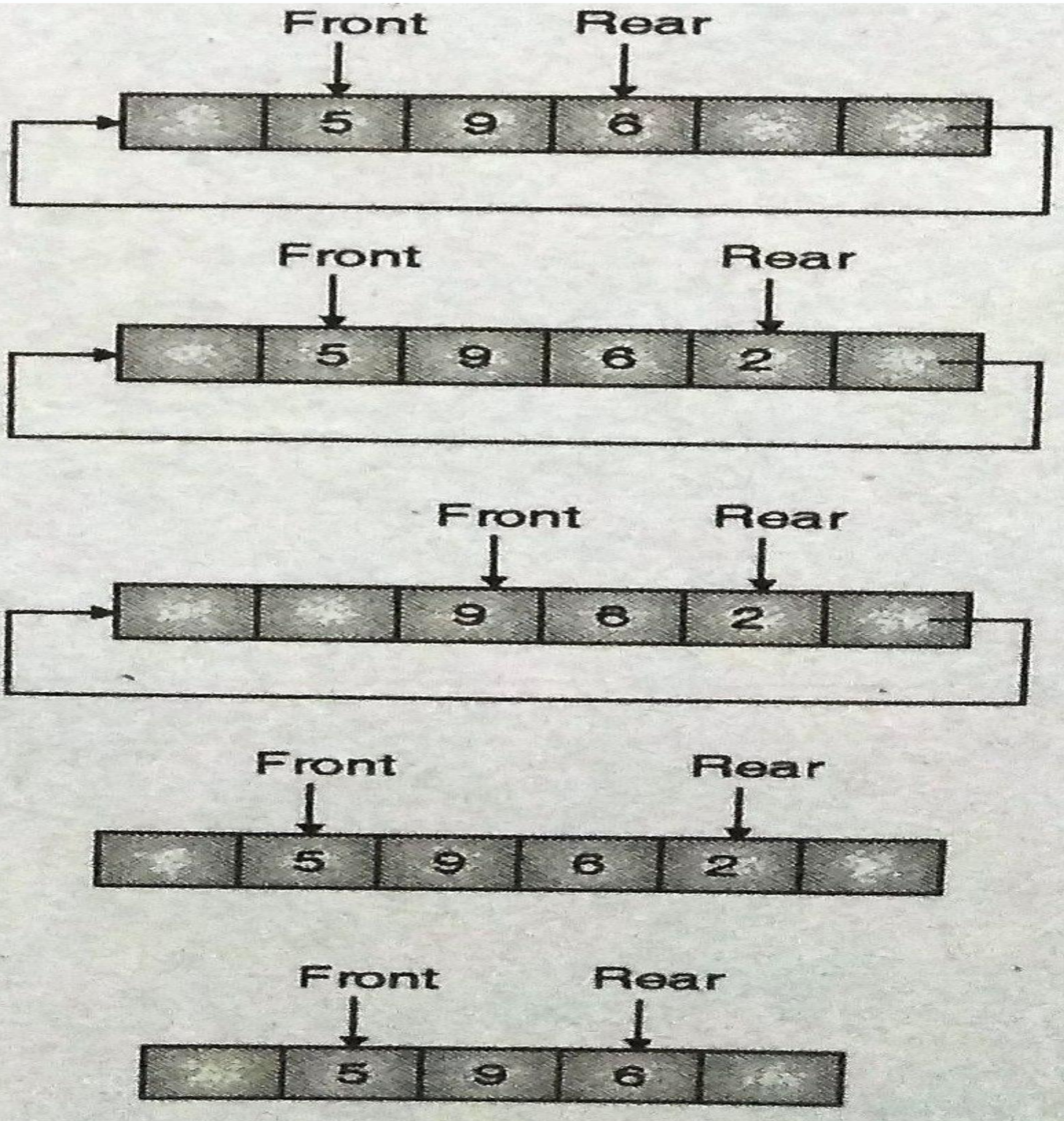
vii) dequeueF( ) : Delete the front element.

viii) print( ) : Print elements of the queue.

☞ **Prototype of functions used for various operations on queue**

- void intialize (DQ *p);
- int empty (DQ *p);
- function returns 1 or 0, depending on whether the queue pointed by p is empty or not.
- int full (DQ *p);
- function returns 1 or 0, depending on whether the queue pointed by p is full or not.
- void enqueueF (DQ *p, int x);
- void enqueueR (DQ *p, int x);
- int deleteR (DQ *p);
- int deleteF (DQ *p);
- void print (DQ *p);
- enqueueR( ) and enqueueF( ) will cause an overflow if the queue is full.
- dequeueR( ) and dequeueF( ) will cause an underflow if the queue is empty.

Implementation of DeQueue using linked list



(dsa11.34)**Fig. 6.4.2 : Various operations on dequeue and movement of front and rear in clock-wise or anti-clockwise direction**

Implementation of DeQueue using linked list

1) To check Empty and Full queue for dequeue :

```
bool Deque::isFull() {
  return ((front == 0 && rear == size - 1) ||
      front == rear + 1);
}

bool Deque::isEmpty() {
  return (front == -1);
}
```

Implementation of DeQueue using linked list

2) Insert at beginning(front) :

```
void Deque::insertfront(int key) {
  if (isFull()) {
    cout << "Overflow\n"
         << endl;
    return;
  }

  if (front == -1) {
    front = 0;
    rear = 0;
  }

  else if (front == 0)
    front = size - 1;

  else
    front = front - 1;

  arr[front] = key;
}
```

3) Insert at end(rear) :

```
void Deque ::insertrear(int key) {
  if (isFull()) {
    cout << " Overflow\n " << endl;
    return;
  }

  if (front == -1) {
    front = 0;
    rear = 0;
  }

  else if (rear == size - 1)
    rear = 0;

  else
    rear = rear + 1;

  arr[rear] = key;
}
```

Implementation of DeQueue using linked list

4) Delete at beginning(front) :

```
void Deque ::deletefront() {
  if (isEmpty()) {
    cout << "Queue Underflow\n"
        << endl;
    return;
  }

  if (front == rear) {
    front = -1;
    rear = -1;
  } else if (front == size - 1)
    front = 0;

  else
    front = front + 1;
}
```

5) Delete at end(rear) :

```
void Deque::deleterear() {
  if (isEmpty()) {
    cout << " Underflow\n"
        << endl;
    return;
  }

  if (front == rear) {
    front = -1;
    rear = -1;
  } else if (rear == 0)
    rear = size - 1;
  else
    rear = rear - 1;
}
```

Dequeu operation : https://youtu.be/MfQHAVPkkTo

Part-2 : https://youtu.be/TKpEbqAVDBE

Priority Queue

- A priority queue is an abstract data type that behaves similarly to the normal queue except that each element has some priority, i.e., the element with the highest priority would come first in a priority queue. The priority of the elements in a priority queue will determine the order in which elements are removed from the priority queue.
- The priority queue supports only comparable elements, which means that the elements are either arranged in an ascending or descending order.
- For example, suppose we have some values like 1, 3, 4, 8, 14, 22 inserted in a priority queue with an ordering imposed on the values is from least to the greatest. Therefore, the 1 number would be having the highest priority while 22 will be having the lowest priority.

Example : https://youtu.be/1KRbIVZZa48

Priority Queue

Characteristics of a Priority queue :

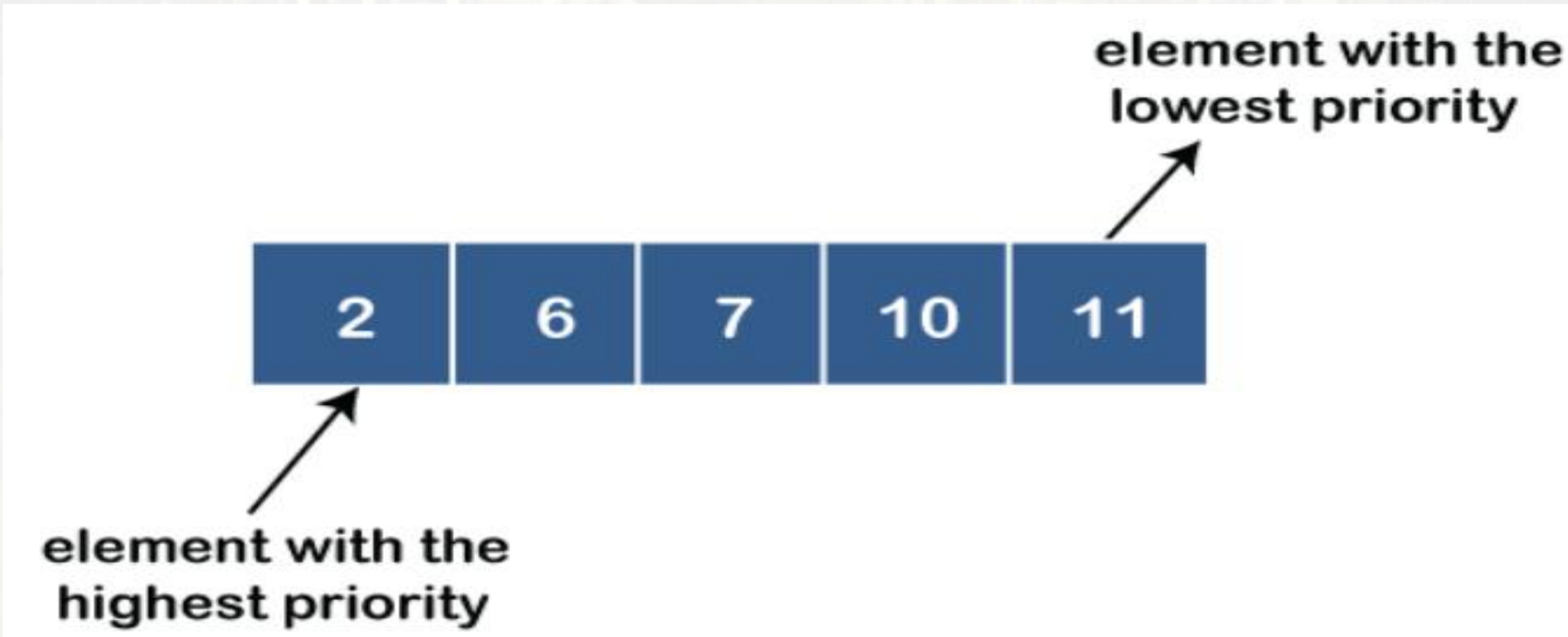
- A priority queue is an extension of a queue that contains the following characteristics:
- Every element in a priority queue has some priority associated with it.
- An element with the higher priority will be deleted before the deletion of the lesser priority.
- If two elements in a priority queue have the same priority, they will be arranged using the FIFO principle.

Types of Priority Queue

There are two types of priority queue:

1. Ascending order priority queue:
2. Descending order priority queue:

Ascending order priority queue: In ascending order priority queue, a lower priority number is given as a higher priority in a priority. For example, we take the numbers from 1 to 5 arranged in an ascending order like 1,2,3,4,5; therefore, the smallest number, i.e., 1 is given as the highest priority in a priority queue.

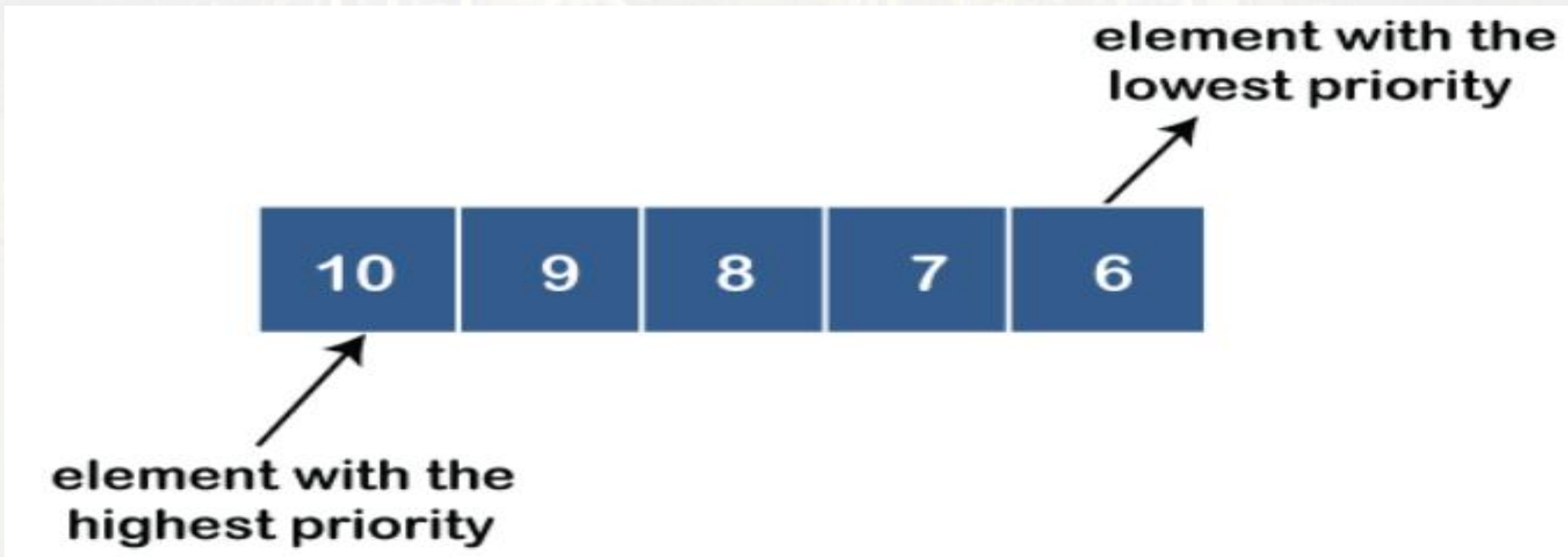


Types of Priority Queue

There are two types of priority queue:

1. Ascending order priority queue:
2. Descending order priority queue:

Descending order priority queue: In descending order priority queue, a higher priority number is given as a higher priority in a priority. For example, we take the numbers from 1 to 5 arranged in descending order like 5, 4, 3, 2, 1; therefore, the largest number, i.e., 5 is given as the highest priority in a priority queue.



Priority Queue as an ADT

```
# define MAX 30 /* A queue with maximum of 30 elements */
        typedef struct pqueue
        { int data [MAX];
         int front, rear;
        } pqueue;
```

☞ **Operations on a priority queue**

i) initialize( ) : Make the queue empty.

ii) empty( ) : Determine if the queue is empty.

iii) full( ) : Determine if the queue is full.

iv) enqueue( ) : Insert an element as per its priority.

v) dequeue( ) : Delete the front element (front element will have the highest priority)

vi) print( ) : Print elements of the queue.

☞ **Prototype of functions used for various operations on the queue**

- `void initialize(pqueue *p);`
- `int empty(pqueue *p);`
- `int full(pqueue *p);`
- `void enqueue(pqueue *p, int x);`

Priority Queue as an ADT

- int dequeue(pqueue *p);
- void print(pqueue *p);
- enqueue( ) operation will cause an overflow if the queue is full.
- Dequeue( ) operation will cause an underflow if the queue is empty.

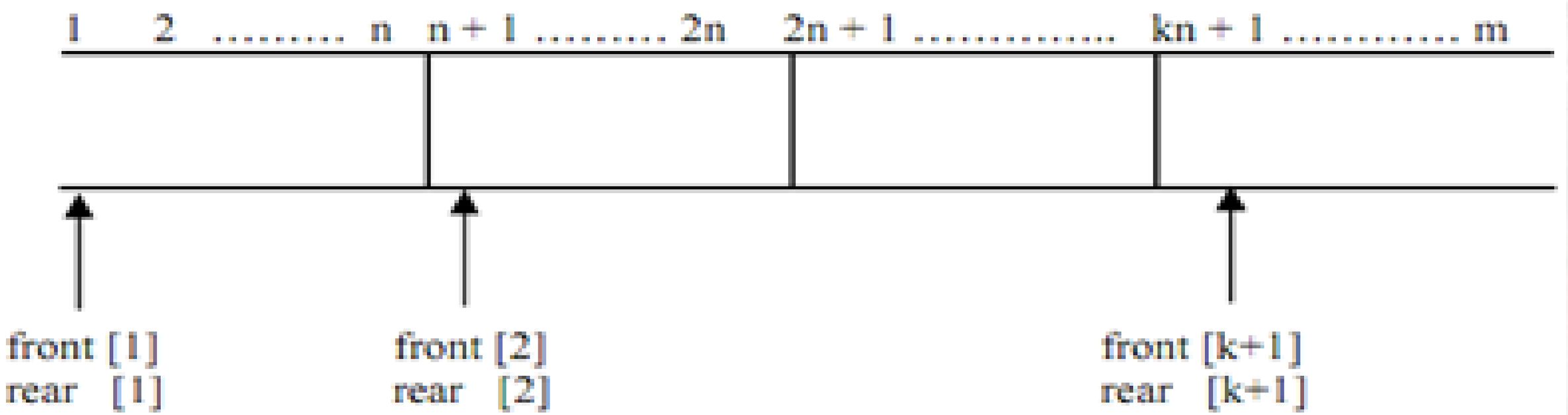
Priority as ADT : https://youtu.be/dFBoOS38dAk

Applications of Priority Queue

1. It is used in the Dijkstra's shortest path algorithm.
2. It is used in prim's algorithm
3. It is used in data compression techniques like Huffman code.
4. It is used in heap sort.
5. It is also used in operating system like priority scheduling, load balancing and interrupt handling.

Multiple Queue using array

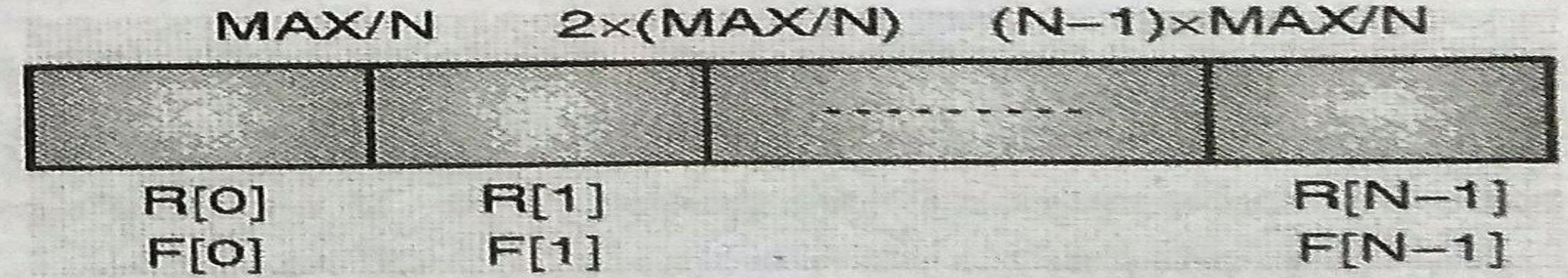
Multi queue is data structure in which multiple queues are maintained. This type of data structures are utilized for process scheduling. We might use one dimensional array or multidimensional array to illustrated a multiple queue.



Multiple Queue using array

Several queues can be stored in a single array. Suppose A[ ] is an array of size MAX and N number of queues are to be stored in the array. MAX is a predefined constant.

We can divide the available memory A [0.. MAX-1] into N segments. Each queue can be allocated one of these segments. For each Queue i, as shown in the Fig. 6.6.1, R[i] and F[i] represent the indices of rear and front of queue i.



(dsa11.36)**Fig. 6.6.1**

Initial values of R[i] and F[i] is calculated as given below.

For (i = 0; i < N; i++)

R[i] = F[i] = (MAX/N)*i;

Insert ( ) and Delete ( ) can be performed on any of the queues. Both insert ( ) and Delete ( ) have an additional parameter, queue number.

Applications of Queue

1. Queues are widely used as waiting lists for a single shared resource like printer, disk, CPU.
2. Queues are used in asynchronous transfer of data (where data is not being transferred at the same rate between two processes) for eg. pipes, file IO, sockets.
3. Queues are used as buffers in most of the applications like MP3 media player, CD player, etc.
4. Queue are used to maintain the play list in media players in order to add and remove the songs from the play-list.
5. Queues are used in operating systems for handling interrupts.

Application of Queue

Application of Queue :

1. Fcfs Schedulling
2. Queue at Cashier counter

Applications of deque

1. Pallindrome checker.

Applications of circular queue

1. **Traffic light** functioning is the best example for circular queues. The colors in the traffic light follow a circular pattern.

2. Round-robin schedulling

Application of Priority Queue :

1. Prims & Kruskals Algorithms
2. Dijkstra's Shortest Path Algorithm

Application of Queue

Example of RR : https://youtu.be/hy5dn9mK36I

Process schedulling : https://youtu.be/THqcAa1bbFU

Priority schedulling : https://youtu.be/YzBBJYfwdi8

https://youtu.be/VSMAjMfJ6KQ

FCFS Schedulling : https://youtu.be/cNdEQKw4apM

Josphous PROBLEM : https://youtu.be/uCsD3ZGzMgE

References

1. https://www.javatpoint.com/ds-types-of-queues
2. https://www.programiz.com/dsa/types-of-queue
3. https://www.scaler.com/topics/dequeue-in-data-structure/
4. https://www.simplilearn.com/tutorials/data-structure-tutorial/dequeue-in-data-structure
5. https://www.expertsmind.com/questions/implementation-of-multiple-queues-30152374.aspx
6. https://www.codewhoop.com/queue/queue-using-linked-list.html
7. https://prepinsta.com/cpp-program/circular-queue-using-linked-list/

References

1. https://www.geeksforgeeks.org/introduction-to-backtracking-data-structure-and-algorithm-tutorials/
2. https://www.tutorialspoint.com/cplusplus-program-to-implement-stack-using-array
3. https://www.helpmestudybro.com/program-to-implement-infix-to-postfix-conversion/
4. https://www.upgrad.com/blog/recursion-in-data-structure/
5. https://aits-tpt.edu.in/wp-content/uploads/2018/08/DS-unit-2.1.pdf
6. https://www.javatpoint.com/applications-of-stack-in-data-structure
7. https://www.boardinfinity.com/blog/postfix-expression/#:~:text=%22Stack%2Dbased%20postfix%20evaluation%22,result%20back%20on%20the%20stack.
8. https://www.tutorialspoint.com/cplusplus-program-to-implement-stack-using-linked-list
9. https://www.javatpoint.com/conversion-of-prefix-to-postfix-expression

Linked List

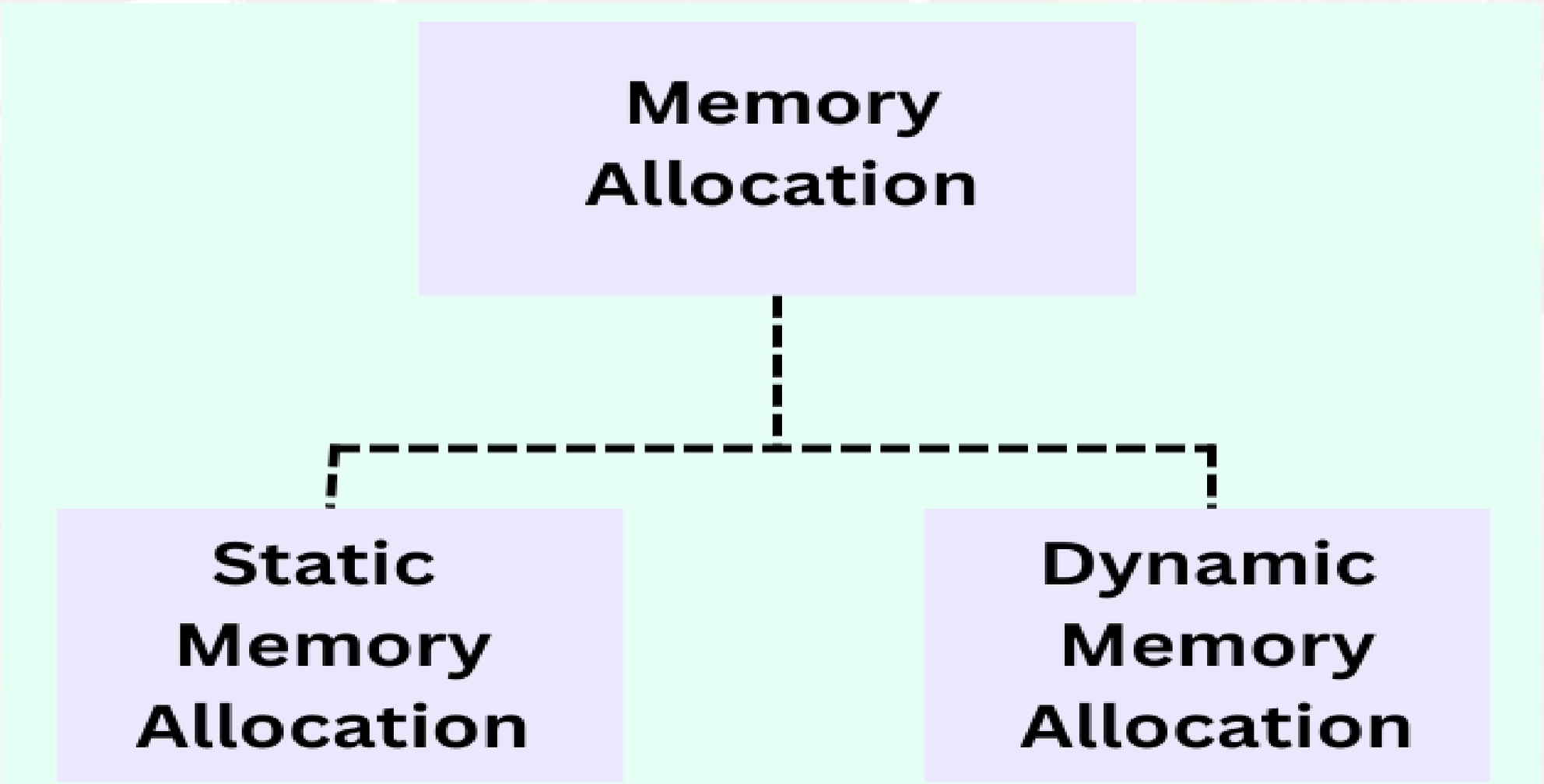
Introduction to Static and Dynamic Memory Allocation, Linked List: Introduction of Linked Lists, Realization of linked list using dynamic memory management, operations, Linked List as ADT, **Types of Linked List:** singly linked, linear and Circular Linked Lists, Doubly Linked List, Doubly Circular Linked List, Primitive Operations on Linked List- Create, Traverse, Search, Insert, Delete, Sort, Concatenate. Polynomial Manipulations-Polynomial addition. Generalized Linked List (GLL) concept, Representation of Polynomial using GLL.

Static & Dynamic Memory Allocation

Memory Allocation

There are two types of memory allocations, which are given as follows:

1. Static memory Allocation (Stack memory)
2. Dynamic memory Allocation (Heap memory)



Static Memory Allocation

Static memory allocation is also known as Compile-time memory allocation because the memory is allocated during compile time.

- In this type of memory allocation, the memory that the program can use is fixed i.e. we can not allocate or deallocate memory during the program's execution.
- In many applications, it is not possible to predict how much memory will be needed by the program at run time.

Static Memory Allocation

Properties of Static Memory allocation

1. Memory allocation is done during compile time.
2. Stack Memory is used here.
3. Memory cannot be changed while executing a program.
4. The static memory allocation is fast and saves running time.
5. It is less efficient as compared to Dynamic memory allocation.
6. The allocation process is simple.

Static Memory Allocation

Advantages of Static Memory allocation :

1. The memory is allocated during compile time.
2. It is easy to use.
3. It uses a stack Data Structure.
4. The execution time is efficiently controlled.

Disadvantages of Static Memory allocation :

1. This allocation method leads to memory wastage.
2. Memory cannot be changed while executing a program.
3. Exact memory requirements must be known.
4. If memory is not required, it cannot be freed.

Dynamic Memory Allocation

Dynamic memory allocation is also known as **Runtime memory allocation** because the memory is allocated during runtime or program execution.

- The allocation and release of the memory space can be done using the library functions of stdlib.h header file.
- These functions allocate memory from a memory area called heap and deallocate this memory whenever not required so that it can be used for some other purpose.

Dynamic Memory Allocation

Properties of Dynamic Memory allocation

1. Memory is allocated at runtime.
2. Memory can be allocated and released at any time.
3. Heap memory is used here.
4. Dynamic memory allocation is slow.
5. It is more efficient as compared to Static memory allocation
6. The allocation process is complicated.
7. Memory can be resized dynamically or reused.

Dynamic Memory Allocation

Advantages of Dynamic Memory allocation :

1. This allocation method has no memory wastage.
2. The memory allocation is done at run time.
3. Memory size can be changed based on the requirements of the dynamic memory allocation.
4. If memory is not required, it can be freed.

Disadvantages of Dynamic Memory allocation :

1. It requires more execution time due to execution during runtime.
2. The compiler does not help with allocation and deallocation. Programmer needs to both allocate and deallocate the memory.

Dynamic Memory Allocation

The library functions of the stdlib.h header file, which helps to allocate memory dynamically are.

1. malloc() – Memory Allocation(assigned memory anywhere)
2. calloc() – Contiguous Allocation (block of memory assigned)
3. realloc() – Resize the memory block
4. free() – Deallocate Memory

FUNCTION DYNAMIC MEMORY ALLOCATION

Function	Use of Function
malloc()	Allocates requested size of bytes and returns a pointer first byte of allocated space
calloc()	Allocates space for an array elements, initializes to zero and then returns a pointer to memory
free()	deallocate the previously allocated space
realloc()	Change the size of previously allocated space

COMPARE Malloc( ) AND Calloc( ) MEMORY

What are the differences between malloc( ) and calloc( )?

malloc()	calloc()
malloc() accepts one argument of type **size_t** with the following prototype: `void *malloc(size_t size);`	calloc() accepts two arguments of type **size_t** with the following prototype: `void *calloc(size_t nitems,size_t size);`
malloc() can allocate one block of memory.	calloc() can allocate multiple blocks of memory.
malloc() does not clear the allocated memory.	calloc() clears the allocated memory. i.e., it places zeros in allocated bytes.
malloc() function allocates memory as chained link.	calloc() function expects the total expected memory to be continuous.

Static Vs Dynamic Memory Allocation

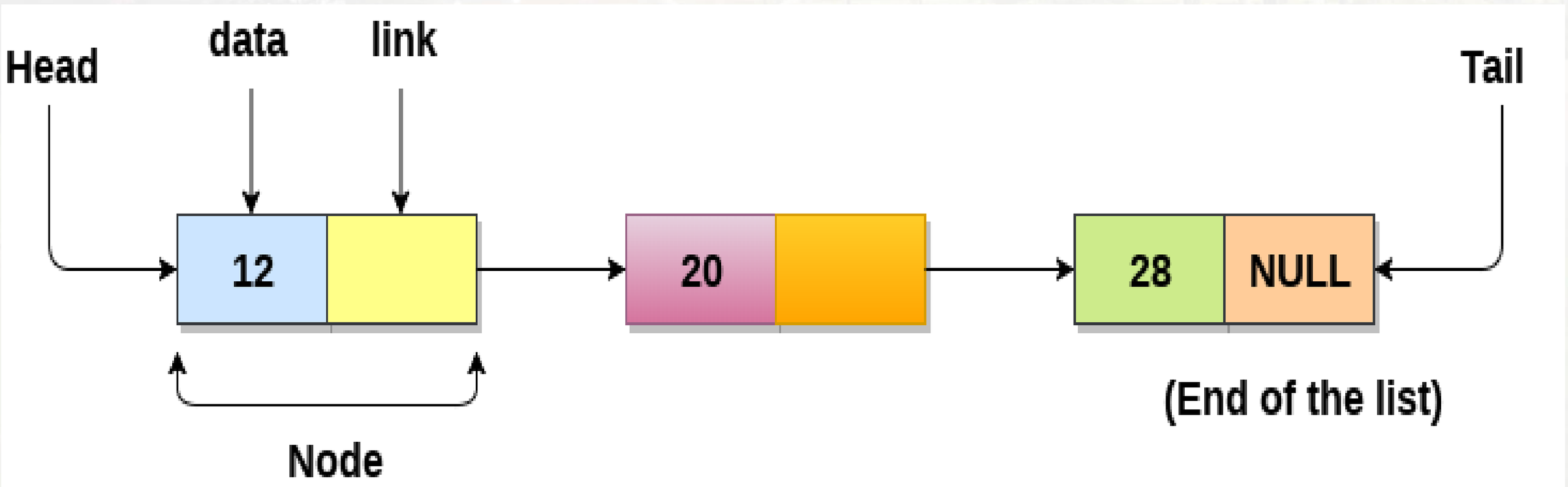
Static Memory Allocation	Dynamic Memory Allocation
1.It occurs at the time of compilation.	It occurs dynamically at the run time.
2. Here, the compiler allocates the memory.	Here, the programmer allocates and deallocates the memory.
3. The size is fixed and known at the time of compilation.	The size can vary during execution.
4. The memory allocation is in the stack.	The memory allocation is in the heap.
5. Here, we can access the memory by the variable names.	Here, we can access the memory with the use of pointers.
6. Lower risk of memory leaks since the compiler takes care of it.	Higher risk of memory leaks since the programmer may forget to deallocate the memory.

Linked List

Introduction to Linked List

Linked List can be defined as collection of objects called nodes that are randomly stored in the memory.

- A node contains two fields i.e. data stored at that particular address and the pointer which contains the address of the next node in the memory.
- The last node of the list contains pointer to the null.

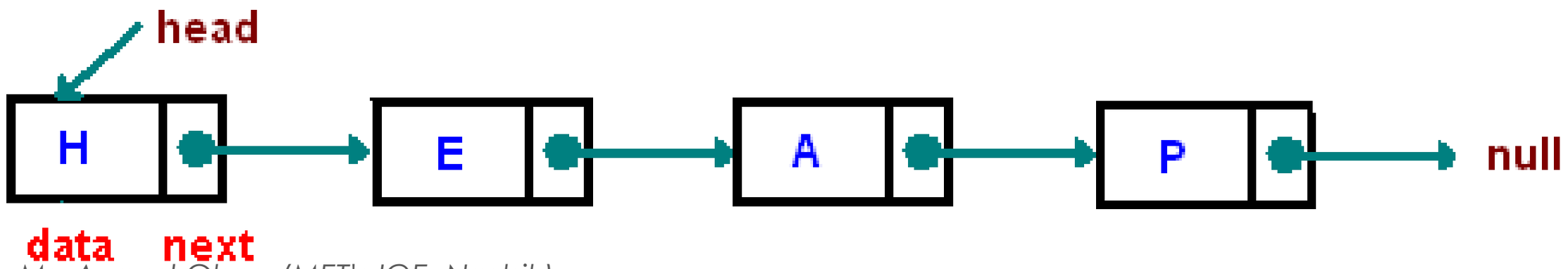


INTRODUCTION OF LINKED LIST

“ Linked List is a very commonly used linear data structure which consists of group of **nodes** in a sequence.

Each node holds its own **data** and the **address of the next node** hence forming a chain like structure.”

Linked Lists are used to create trees and graphs.



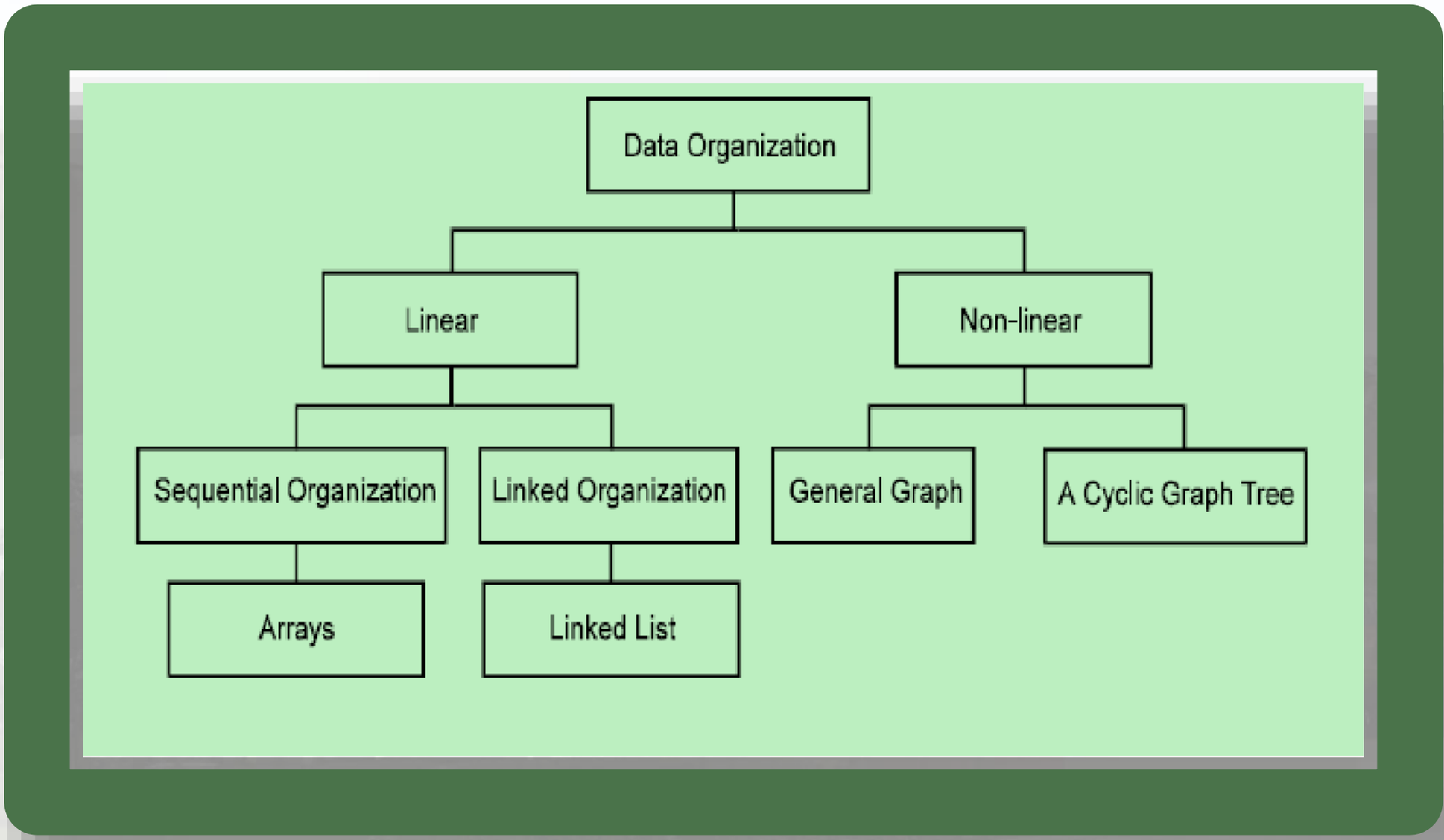


Fig: 1 Data Organization

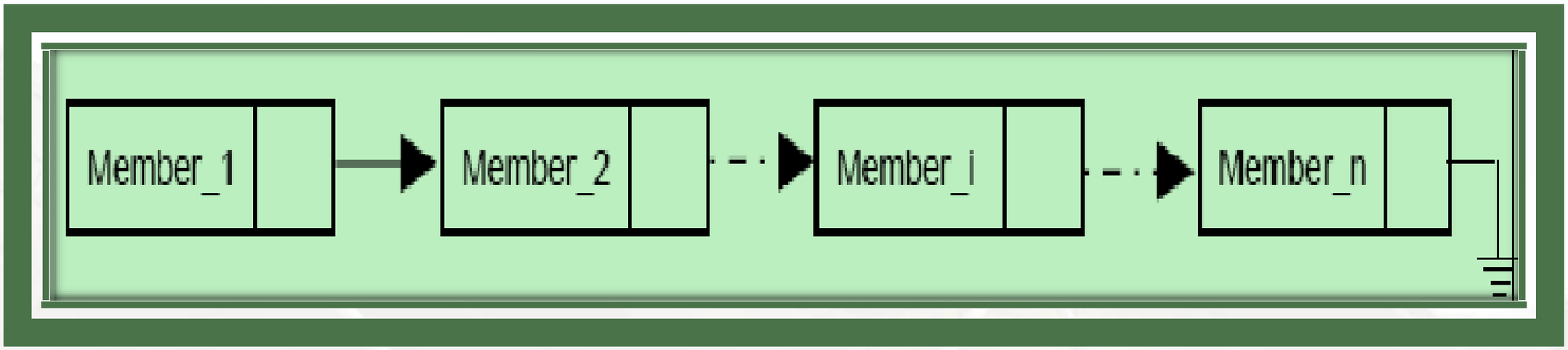


Fig: 2 (a): A linked list of n elements

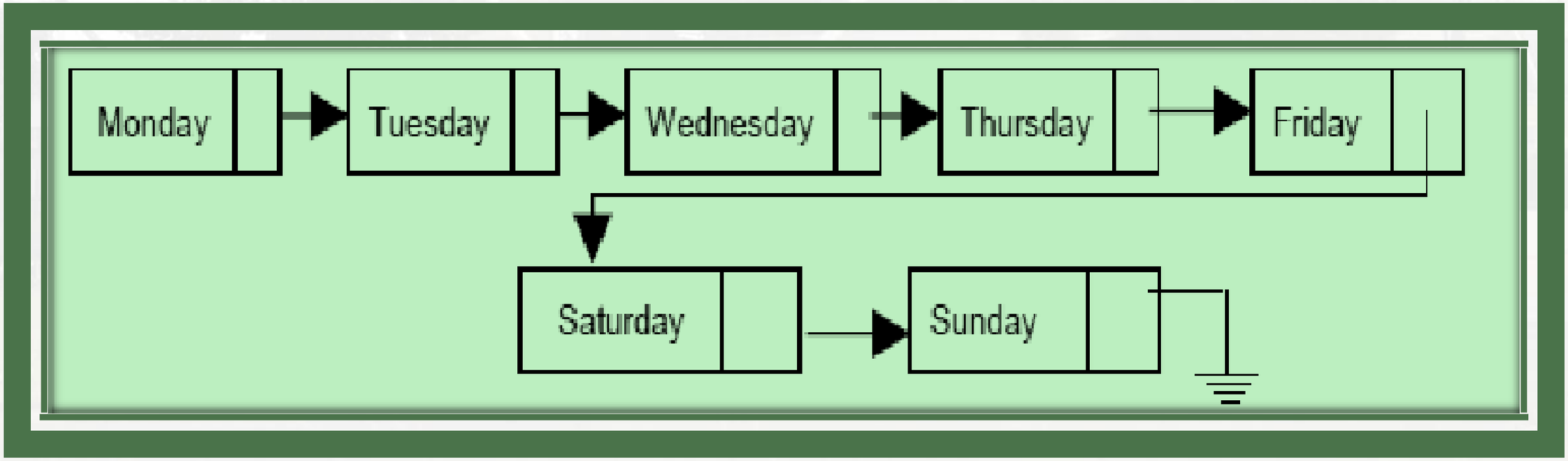


Fig: 2 (b) : A linked list of weekdays

INTRODUCTION OF LINKED LIST

Advantages of Linked Lists

- They are a dynamic in nature which allocates the memory when required.
- Insertion and deletion operations can be easily implemented.
- Stacks and queues can be easily executed.
- Linked List reduces the access time.

Disadvantages of Linked Lists

- The memory is wasted as pointers require extra memory for storage.
- No element can be accessed randomly; it has to access each node sequentially.
- Reverse Traversing is difficult in linked list.

INTRODUCTION OF LINKED LIST

Applications of Linked Lists

1. Linked lists are used to implement stacks, queues, graphs, etc.

2. Linked lists let you insert elements at the beginning and end of the list.

3. In Linked Lists we don't need to know the size in advance.

TERMINOLOGIES OF LINKED LIST

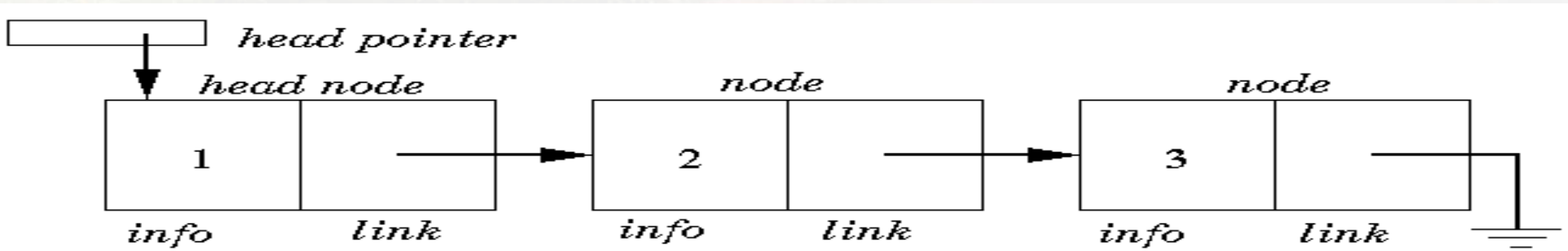
Node an item in a linked list. Each node contains a piece of list data and the location of the next node (item).

Link (of a node) the location of the next node.

head node first node in a linked list

head pointer points to the head node.

Null pointer there is no need in it (linked list is empty)



A Linked List

DIFFERENCE BETWEEN SEQUENTIAL AND LINKED ORGANIZATION

ARRAY	LINKED LISTS
1. Arrays are stored in contiguous location.	1. Linked lists are not stored in contiguous location.
2. Fixed in size.	2. Dynamic in size.
3. Memory is allocated at compile time.	3. Memory is allocated at run time.
4. Uses less memory than linked lists.	4. Uses more memory because it stores both data and the address of next node.
5. Elements can be accessed easily.	5. Element accessing requires the traversal of whole linked list.
6. Insertion and deletion operation takes time.	6. Insertion and deletion operation is faster.

LINKED LIST PRIMITIVE OPERATION

Basic Operations

Following are the basic operations supported by a list.

Insertion − Adds an element at the beginning of the list.

Deletion − Deletes an element at the beginning of the list.

Display − Displays the complete list.

Search − Searches an element using the given key.

Delete − Deletes an element using the given key.

Types of Linked List

Types of Linked Lists

There are 3 different implementations of Linked List available, they are:

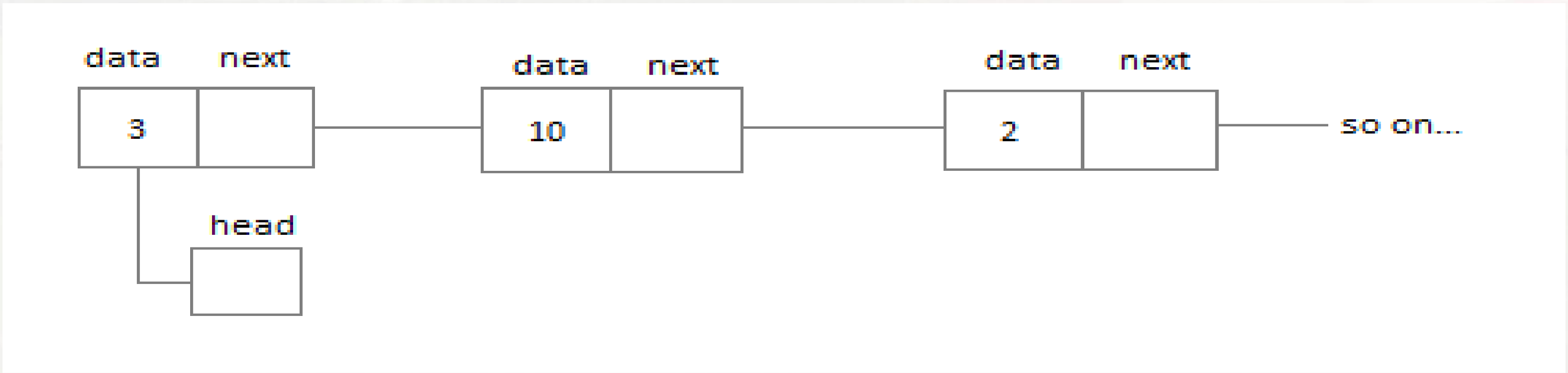
1. Singly Linked List
2. Doubly Linked List
3. Circular Linked List

Types of Linked List

Singly Linked List

Singly linked lists contain nodes which have a data part as well as an address part i.e. next, which points to the next node in the sequence of nodes.

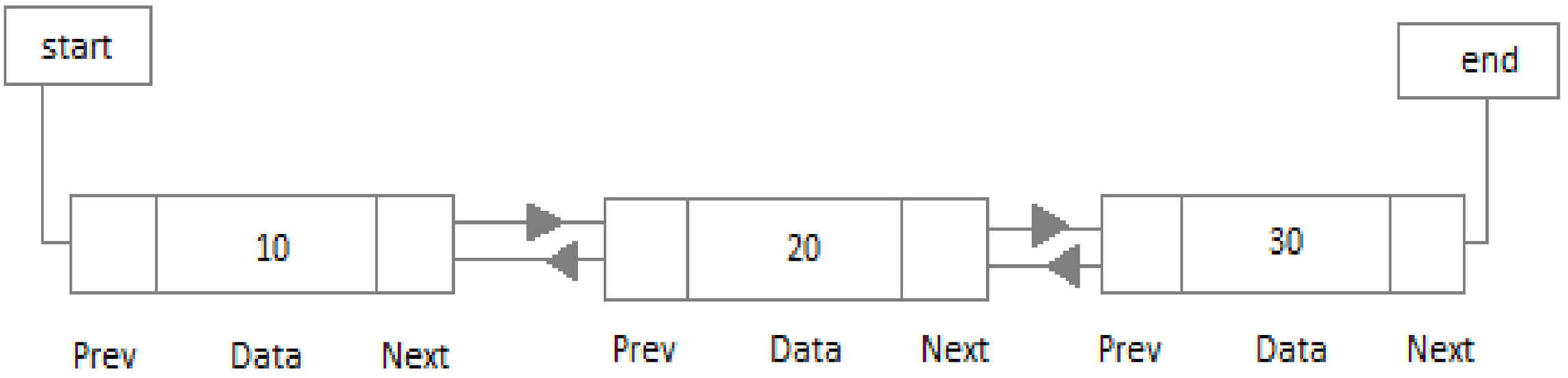
The operations we can perform on singly linked lists are insertion, deletion and traversal.



Types of Linked List

Doubly Linked List

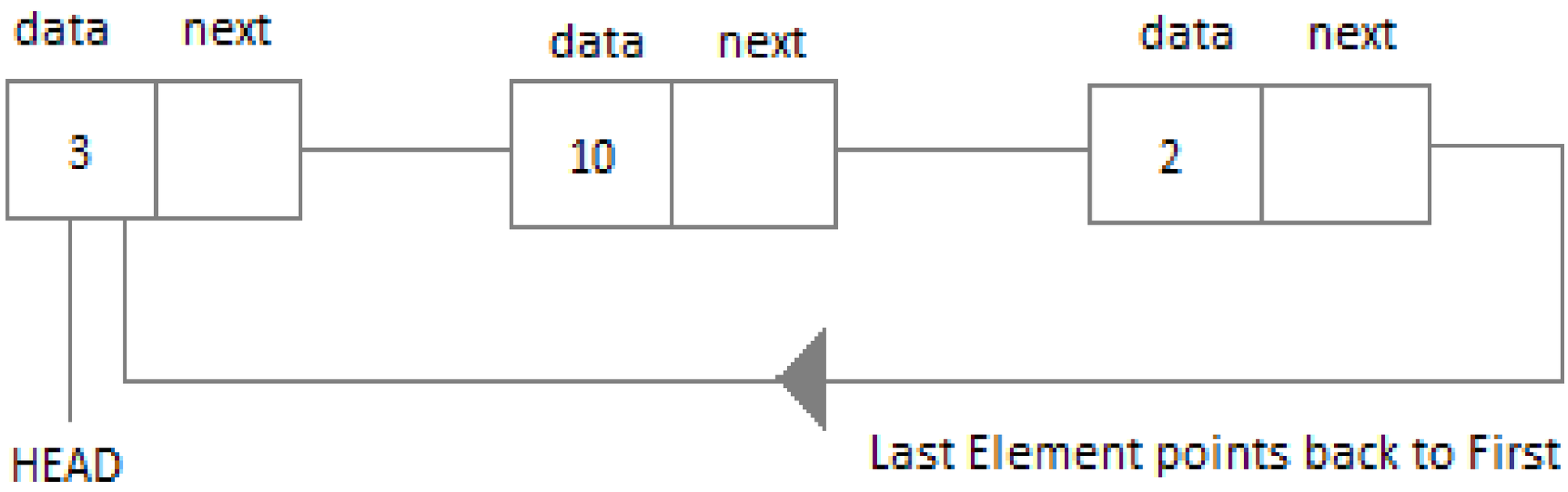
In a doubly linked list, each node contains a data part and two addresses, one for the previous node and one for the next node.



Types of Linked List

Circular Linked List

In circular linked list the last node of the list holds the address of the first node hence forming a circular chain..



DYNAMIC MEMORY MANAGEMENT

" Dynamic Memory Allocation means memory which can be allocated or deallocated as per requirement at run time"

Importance of memory allocation :

- No need to initially occupy large amount of memory.
- Memory can be allocated or deallocated as per need.
- It avoid wastage of memory.
- We can free the memory by de-allocating it using free( ).

Representation of Linked list

Realization stands for Representation of Linked List

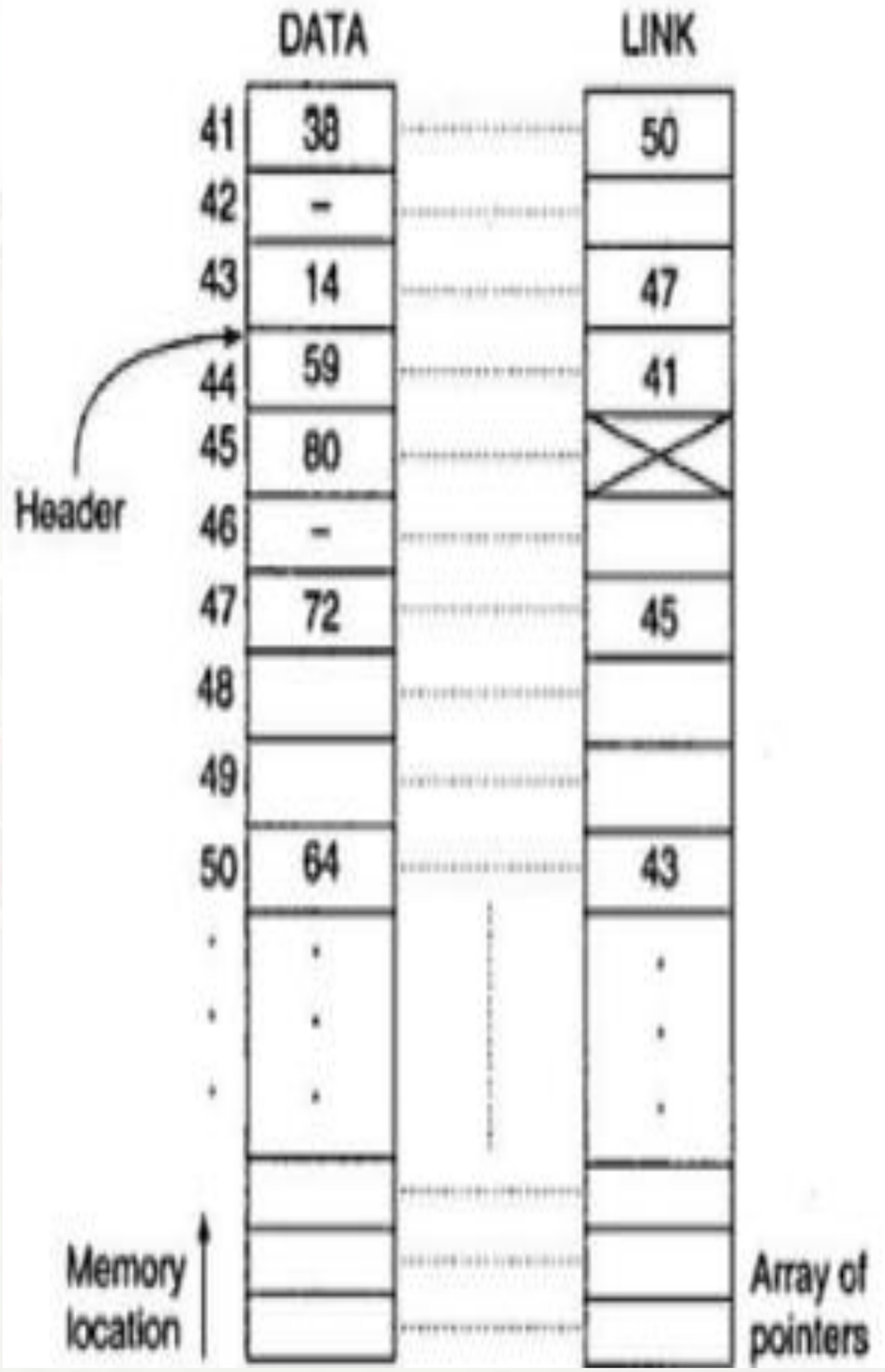
A linked list can be represented in two ways :

1. Static representation of linked list
2. Dynamic representation of linked list

Representation of Linked list

Static Representation of linked list :

In static representation of a single linked list, two arrays are maintained: one array for data and the other for links. The static representation of the linked list in above Figure is shown in below Figure. Two parallel arrays of equal size are allocated which should be sufficient to store the entire linked list. Nevertheless this contradicts the idea of the linked list (that is non-contagious location of elements). But in some programming languages, for example, ALGOL, FORTRAN, BASIC, etc. such a representation is the only representation to manage a linked list.



Representation of Linked list

Dynamic Representation of linked list :

The efficient way of representing a linked list is using the free pool of storage. In this method, there is a memory bank (which "is nothing but a collection of free memory spaces) and a memory manager (a program, in fact). During the creation of a linked list, whenever a node is required the request is placed to 2 the memory manager; the memory manager will then search the memory bank for the block requested and, if found, grants the desired block to the caller. Again, there is also another program called the garbage collector; it plays whenever a node is no more in use; it returns the unused node to the memory bank. It may be noted that memory bank is basically a list of memory' spaces which is available to a programmer. Such a memory management is known as dynamic memory management. The dynamic representation of linked list uses the dynamic memory management policy.

Representation of Linked list

Dynamic Representation of linked list :

☞ **Importance of dynamic memory allocation**

- No need to initially occupy large amount of memory.
- Memory can be allocated as well as de-allocated as per necessity.
- It avoids the memory shortage as well as memory wastage.

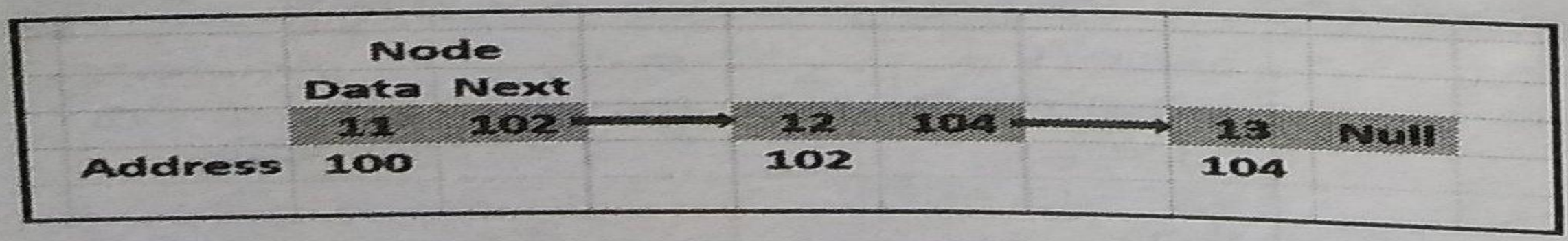


Fig. 3.6.2 : Representation of linear linked list

- In Fig. 3.6.2 we can observe that a linked list is made up of number of nodes. Every node contains two parts : data and next.

Linked list Abstract Data Type

"**Linked List is an *Abstract Data Type (ADT)*** that holds a collection of **Nodes,**the nodes can be accessed in a sequential way. **Linked List doesn't provide a random access to a Node.**"

Usually, those Nodes are connected to the next node and/or with the previous one, this gives the *linked* effect. When the Nodes are connected with only the *next* pointer the list is called *Singly Linked List* and when it's connected by the *next and previous* the list is called *Doubly Linked List.*

Linked list Abstract Data Type

ADT — Interface

The Linked List interface can be implemented in different ways, is important to have operations to insert a new node and to remove a Node:

ADT Operation :

Create – Creation of linked list

Traversal – it can be traversed

Search – Data can be compare with data to be search.

Destroy – Free the memory

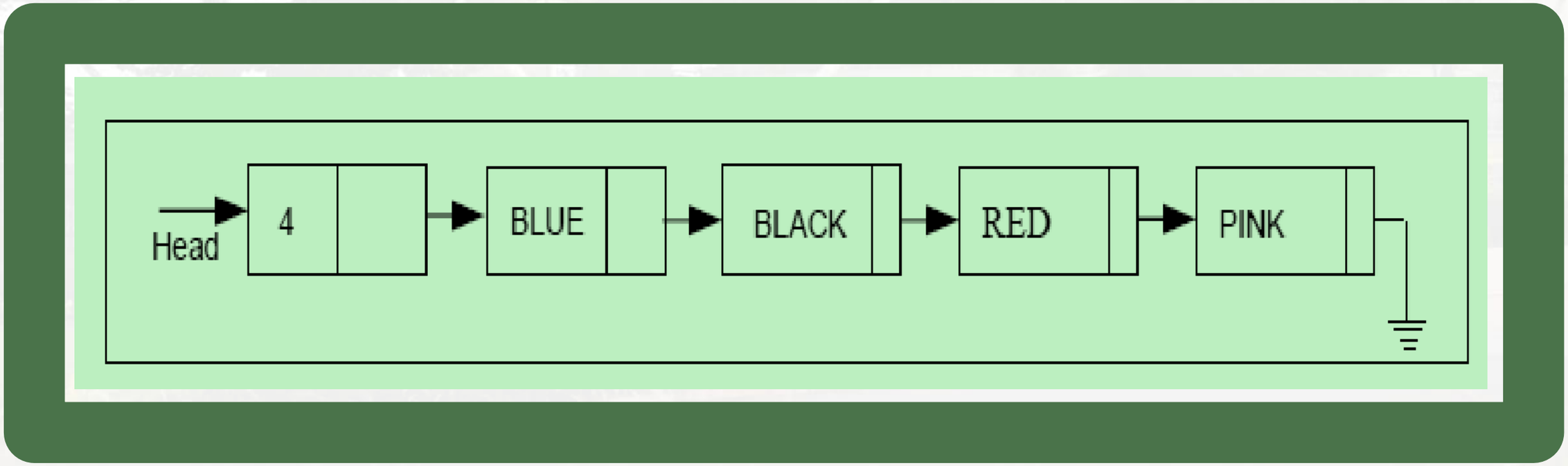
TYPES OF LINKED LIST

There are 3 different implementations of Linked List available, they are:

1. Linear/Singly Linked List
2. Doubly Linked List
3. Circular Linked List

LINEAR/SINGLY LINKED LIST

" A linked list in which every node has one link field, to provide information about where the next node of list is, is called as ***singly linked list*** *"*



LINEAR/SINGLY LINKED LIST

Singly linked list is a basic linked list type. Singly linked list is a collection of nodes linked together in a sequential way where each node of singly linked list contains a data field and an address field which contains the reference of the next node. Singly linked list can contain multiple data fields but should contain at least single address field pointing to its connected next node.

ADVANTAGES OF SINGLY LINKED LIST

1. Singly linked list is probably the most easiest data structure to implement.
2. Insertion and deletion of element can be done easily.
3. Insertion and deletion of elements doesn't requires movement of all elements when compared to an array.
4. Requires less memory when compared to doubly, circular or doubly circular linked list.
5. Can allocate or deallocate memory easily when required during its execution.
6. It is one of most efficient data structure to implement when traversing in one direction is required.

DISADVANTAGES OF SINGLY LINKED LIST

1. It uses more memory when compared to an array.
2. Since elements are not stored sequentially hence requires more time to access each elements of list.
3. Traversing in reverse is not possible in case of Singly linked list when compared to Doubly linked list.
4. Requires O(n) time on appending a new node to end. Which is relatively very high when compared to array or other linked list.

SINGLY LINKED LIST OPERATION

1. Creation https://youtu.be/Ovhj6qDSF9M
2. Insertion https://youtu.be/dU0KxLgNM9E
3. Deletion https://youtu.be/VCRGaWXGOKU?list=PLhb7SOmGNUc5AZurO-im4t_RDr-ymjz0d
4. Searching https://youtu.be/lWEBpaVPoJA?list=PLhb7SOmGNUc5AZurO-im4t_RDr-ymjz0d
5. Display
6. Reverse https://youtu.be/sYcOK51hl-A

DOUBLY LINKED LIST OPERATION

1. *Basic of DLL : https://youtu.be/zwN6s2RC0HM*

2. *ALL DLL Operation : https://youtu.be/ZlNKNSz88Nk*

3. *DLL IN C++ : https://youtu.be/zRs0NRpw7L8*

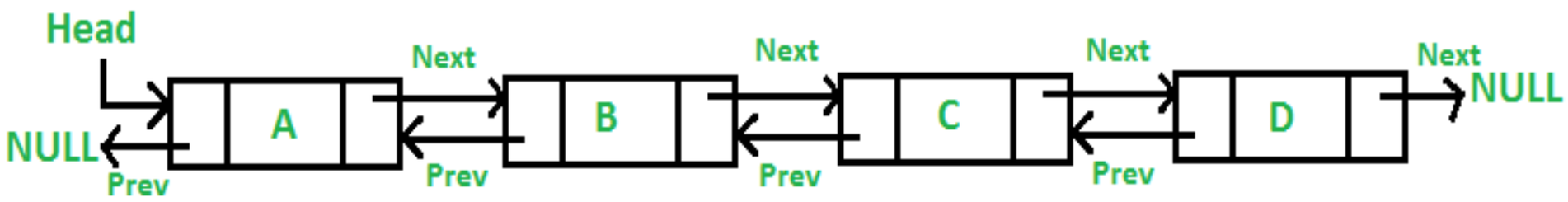
- *Mr. Anand Gharu(MET's IOE, Nashik)*

DOUBLY LINKED LIST

*In **doubly linked list**, each node has two link fields to store information about who is the next and also about who is ahead of the node*

*Hence each node has knowledge of its **successor and also its predecessor.***

*In doubly linked list, from every node the list can be traversed in **both the directions.***



DOUBLY LINKED LIST

Doubly Linked List is a variation of Linked list in which navigation is possible in both ways, either forward and backward easily as compared to Single Linked List. Following are the important terms to understand the concept of doubly linked list.

Link − Each link of a linked list can store a data called an element.

Next − Each link of a linked list contains a link to the next link called Next.

Prev − Each link of a linked list contains a link to the previous link called Prev.

LinkedList − A Linked List contains the connection link to the first link called First and to the last link called Last.

BASIC OPERATION OF DLL

Following are the basic operations supported by a list.

Insertion − Adds an element at the beginning of the list.

Deletion − Deletes an element at the beginning of the list.

Insert Last − Adds an element at the end of the list.

Delete Last − Deletes an element from the end of the list.

Insert After − Adds an element after an item of the list.

Delete − Deletes an element from the list using the key.

Display forward − Displays the complete list in a forward manner.

Display backward − Displays the complete list in a backward manner.

ADVANTAGES OF DOUBLY LINKED LIST

1) A DLL can be traversed in both forward and backward direction.

2) The delete operation in DLL is more efficient if pointer to the node to be deleted is given.

3) We can quickly insert a new node before a given node.

In singly linked list, to delete a node, pointer to the previous node is needed. To get this previous node, sometimes the list is traversed. In DLL, we can get the previous node using previous pointer.

DISADVANTAGES OF DOUBLY LINKED LIST

1) Every node of DLL Require extra space for an previous pointer.

It is possible to implement DLL with single pointer though

2) All operations require an extra pointer previous to be maintained. For example, in insertion, we need to modify previous pointers together with next pointers. For example in following functions for insertions at different positions, we need 1 or 2 extra steps to set previous pointer.

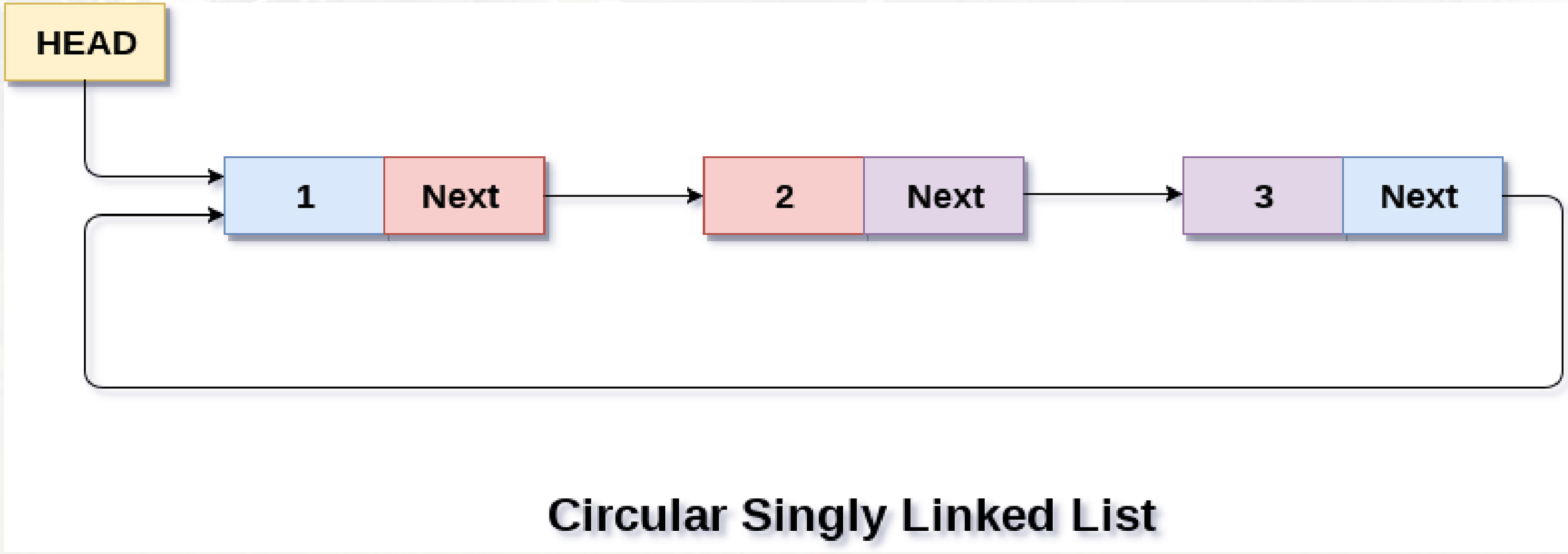
APPLICATION OF DOUBLY LINKED LIST

Doubly Linked List Applications :

1. Redo and undo functionality in software.
2. Forward and backward navigation in browsers.
3. For navigation systems where forward and backward navigation is required.

SINGLY CIRCULAR LINKED LIST

"Circular Linked List is a variation of Linked list in which the first element points to the last element and the last element points to the first element. Both Singly Linked List and Doubly Linked List can be made into a circular linked list".



Circular Singly Linked List

CIRCULAR LINKED LIST

Circular linked list are mostly used in task maintenance in operating systems. There are many examples where circular linked list are being used in computer science including browser surfing where a record of pages visited in the past by the user, is maintained in the form of circular linked lists and can be accessed again on clicking the previous button.

BASIC OPERATION OF CLL

Following are the important operations supported by a circular list.

insert − Inserts an element at the start of the list.

delete − Deletes an element from the start of the list.

display − Displays the list.

ADVANTAGES OF CIRCULAR LINKED LIST

1. Some problems are **circular** and a **circular** data structure would be more natural when used to represent it.

2. The entire **list** can be traversed starting from any node (traverse means visit every node just once)

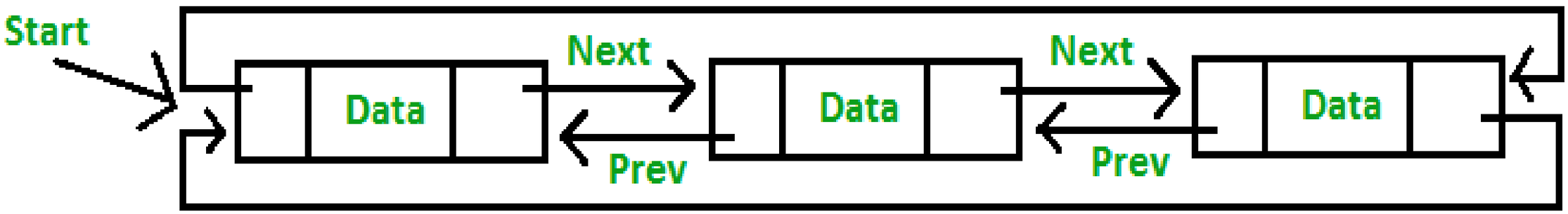
3. fewer special cases when coding(all nodes have a node before and after it)

DISADVANTAGES OF CIRCULAR LINKED LIST

1. **Circular list** are complex as compared to **singly linked lists**.
2. Reversing of **circular list** is a complex as compared to **singly** or doubly **lists**.
3. If not traversed carefully, then we could end up in an infinite loop.

DOUBLY CIRCULAR LINKED LIST

“Circular Doubly Linked List has properties of both doubly linked list and circular linked list in which two consecutive elements are linked or connected by previous and next pointer and the last node points to first node by next pointer and also the first node points to last node by previous pointer”.



BASIC OPERATION OF DOUBLY CIRCULAR LL

SN	Operation	Description
1	Insertion at beginning	Adding a node in circular doubly linked list at the beginning.
2	Insertion at end	Adding a node in circular doubly linked list at the end.
3	Deletion at beginning	Removing a node in circular doubly linked list from beginning.
4	Deletion at end	Removing a node in circular doubly linked list at the end.

ADVANTAGES OF CIRCULAR LINKED LIST

1. List can be traversed from both the directions i.e. from head to tail or from tail to head.
2. Jumping from head to tail or from tail to head is done in constant time O(1).
3. Circular Doubly Linked Lists are used for implementation of advanced data structures like Fibonacci Heap.

DISADVANTAGES OF CIRCULAR LINKED LIST

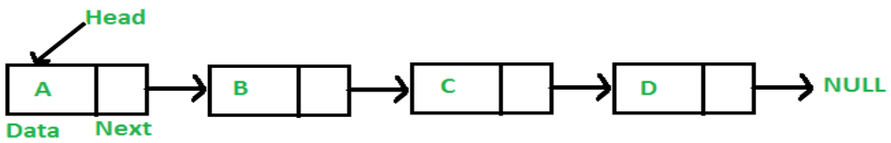
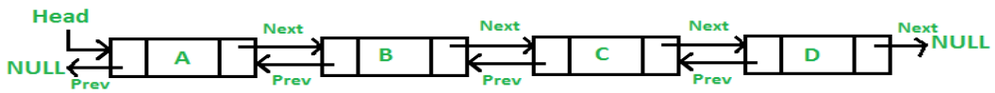
1. It takes slightly extra memory in each node to accommodate previous pointer.
2. Lots of pointers involved while implementing or doing operations on a list. So, pointers should be handled carefully otherwise data of the list may get lost.

Applications of Circular doubly linked list

Managing songs playlist in media player applications.

Managing shopping cart in online shopping

Singly vs Doubly Linked List

Singly linked list (SLL)	Doubly linked list (DLL)
SLL nodes contains 2 field -data field and next link field.	DLL nodes contains 3 fields -data field, a previous link field and a next link field.
	
In SLL, the traversal can be done using the next node link only. Thus traversal is possible in one direction only.	In DLL, the traversal can be done using the previous node link or the next node link. Thus traversal is possible in both directions (forward and backward).
The SLL occupies less memory than DLL as it has only 2 fields.	The DLL occupies more memory than SLL as it has 3 fields.
Complexity of insertion and deletion at a given position is O(n).	Complexity of insertion and deletion at a given position is O(n / 2) = O(n) because traversal can be made from start or from the end.
Complexity of deletion with a given node is O(n), because the previous node needs to be known, and traversal takes O(n)	Complexity of deletion with a given node is O(1) because the previous node can be accessed easily

Parameter	Singly Linked List	Doubly Linked List	Circular Linked List
Node Structure	Node contains two parts: data and link to next node.	Node contains three parts: data and links to previous and next nodes.	Node contains two parts: data and link to next node.
Traversing	Only forward traversing is allowed.	Forward and backward both traversing is allowed.	Only forward traversing is allowed but can jump from last node to first.
Memory	It uses less memory per node (single pointer).	It uses more memory per node(two pointers).	It uses less memory per node (single pointer).
Use	Singly linked list can mostly be used for stacks.	Doubly linked list can be used to implement stacks, heaps, binary trees.	Can be used to implement round robin method.
Complexity	Complexity of Insertion and Deletion at known position is O (n).	Complexity of Insertion and Deletion at known position is O (1).	At known, Position the Complexity of Insertion is O(n) an deletion is O(1).

ADVANTAGES OF GARBAGE COLLECTION

Advantages of Garbage Collection

- GC eliminates the need for the programmer to deallocate memory blocks explicitly
- Garbage collection helps ensure program integrity.
- Garbage collection can also dramatically simplify programs.

Link List Operations/ Programming

- *Mr. Anand Gharu(MET's IOE, Nashik)*

Singly Linked List

- **Structure of the singly linked list :**

```
struct Node
{
   int data;
   struct Node *next;
};
```

- **Insertion of a node**

```
void insertStart(int data)
    {
        Node* newNode = new Node(data);
        newNode->next = head;
        head = newNode;
        cout << data << " Inserted" << endl;
    }
```

Singly Linked List

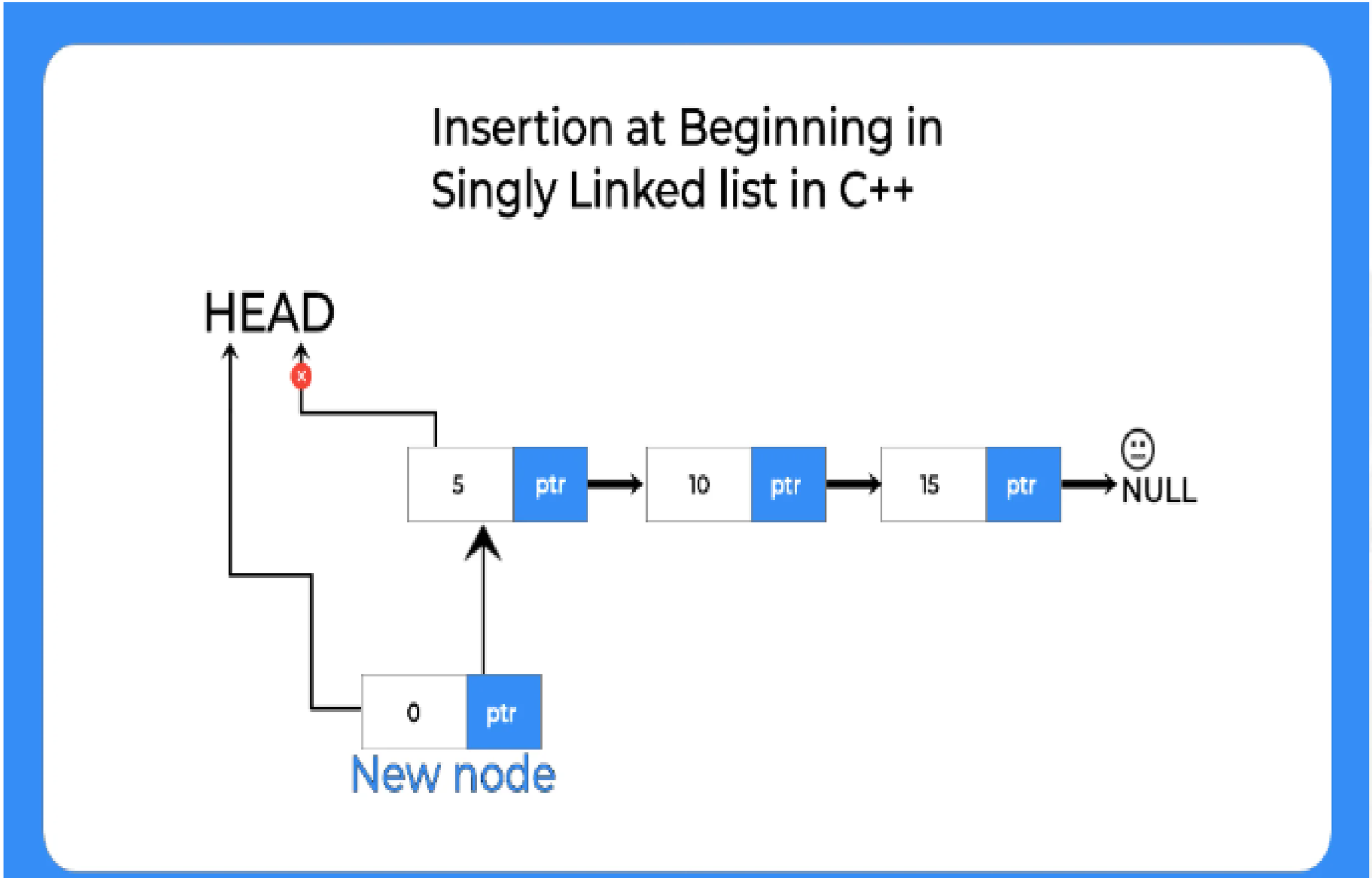
- **Insertion of a node**

Algorithm for insertion in a sorted linked list in C++

1. IF (*HEAD==NULL || (*HEAD)->DATA>=NEWNODE->DATA
2. NEWNODE->NEXT=*HEAD
3. *HEAD=NEWNODE
4. RETURN AND END IF
5. WHILE(CURRENT->NEXT!=NULL && CURRENT->NEXT->DATADATA)
6. CURRENT=CURRENT->NEXT
7. NEWNODE->NEXT=CURRENT->NEXT
8. CURRENT->NEXT=NEWNODE

Singly Linked List

- **Insertion of a node at beginning :**



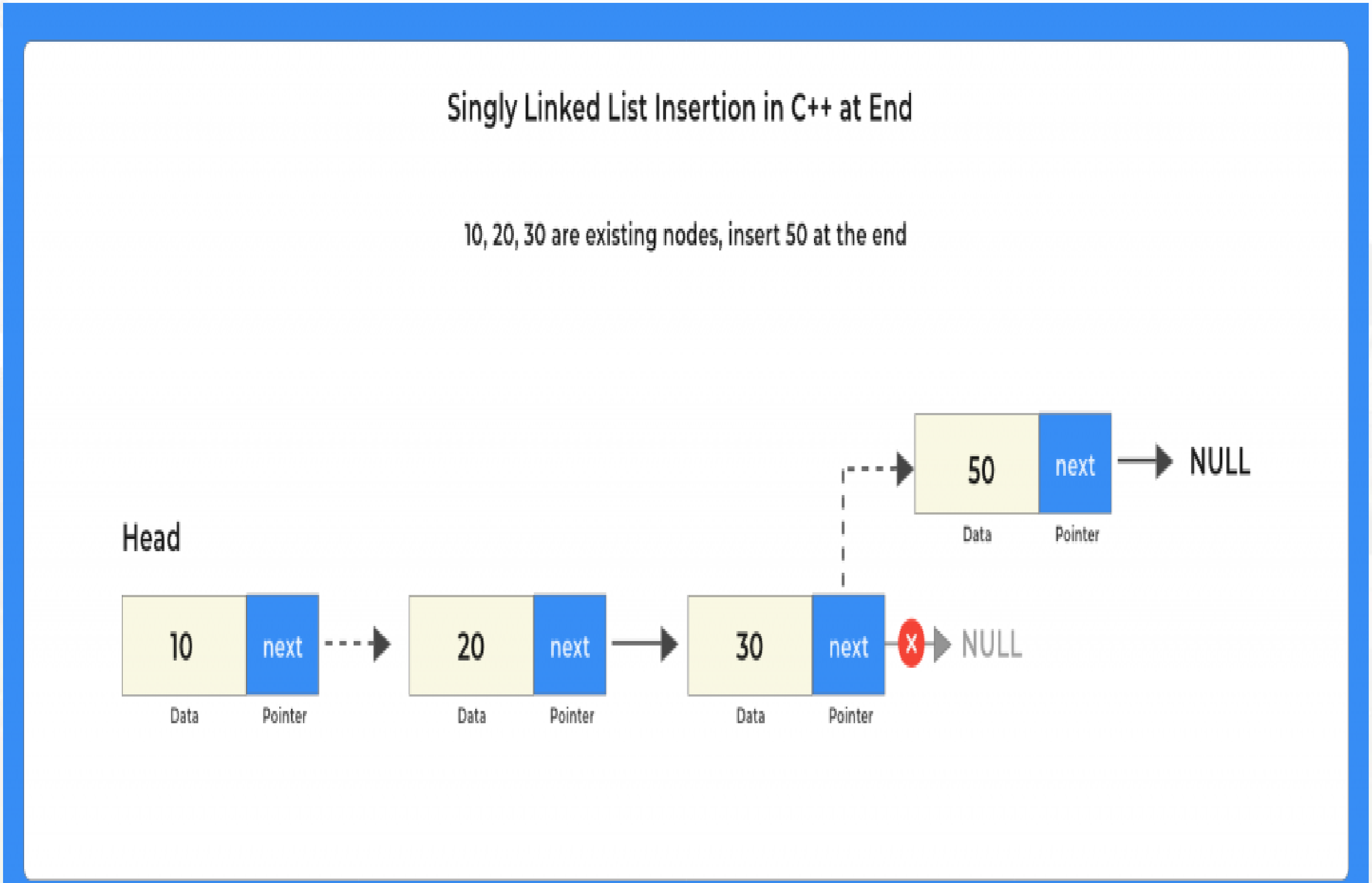
Singly Linked List

- **Insertion of a node at beginning :**

```
void insertFront(Node** head, int data){

    Node* new_node = new Node();

    // assign data
    new_node->data = data;

    // change the next node of this newNode
    // to current head of Linked List
    new_node->next = *head;

    // new_node should become the new head of Linked List
    *head = new_node;

    cout << "Inserted Item: " << new_node->data << endl;
}
```

Singly Linked List

- **Insertion of a node at end :**



Singly Linked List

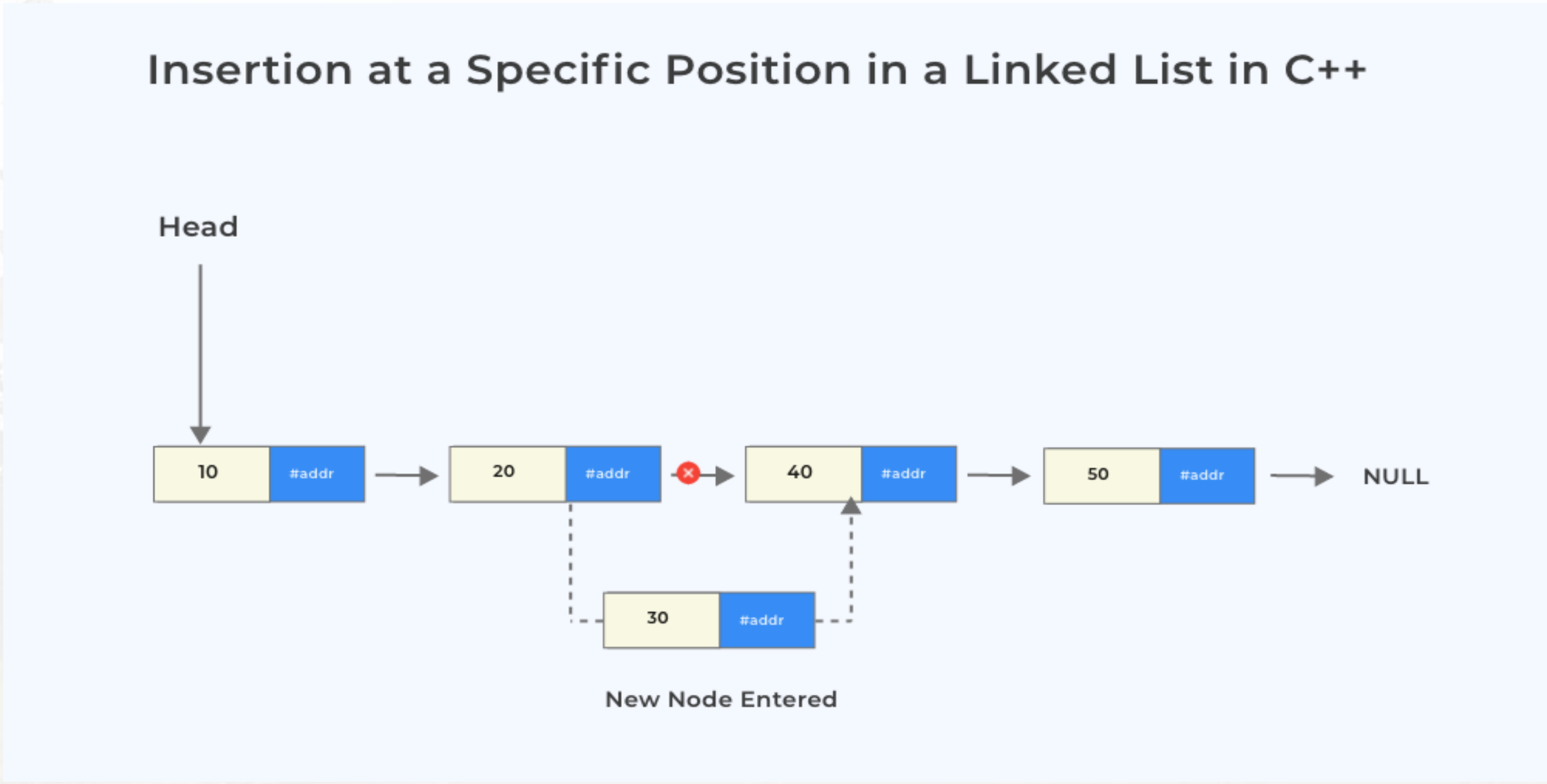
- **Insertion of a node at end :**

```
void insertEnd(Node** head, int data)
{

    Node* freshNode = new Node();

    freshNode->data = data;
    // since this will be the last node so it will point to NULL
    freshNode->next = NULL;

    //need this if there is no node present in linked list at all
    if(*head==NULL)
    {
        *head = freshNode;
        cout << freshNode->data << " inserted" << endl;
        return;
    }

    struct Node* temp = *head;

    // traverse to the last node of Linked List
    while(temp->next!=NULL)
        temp = temp->next;

    // assign last node's next to this freshNode
    temp->next = freshNode;
    cout << freshNode->data << " inserted" << endl;
}
```

Mr. Anand Ghare (MITesOL, Udemy)

Singly Linked List

- **Insertion of a node at position :**



Program Link : *https://prepinsta.com/cpp-program/insert-a-node-at-a-specific-position-in-a-linked-list/*

- *Mr. Anand Gharu(MET's IOE, Nashik)*

Singly Linked List

- **Insertion at position(in between):**

```
struct node *newNode;
newNode = malloc(sizeof(struct node));
newNode->data = 4;

struct node *temp = head;

for(int i=2; i < position; i++) {
   if(temp->next != NULL) {
      temp = temp->next;
   }
}
newNode->next = temp->next;
temp->next = newNode;
```

Singly Linked List

- **Insertion at position(in between):**

```
// Method to insert at a given position
void insertPosition(int pos, int data, Node **head)
{
   Node *new_node = new Node();
   new_node->data = data;
   new_node->next = nullptr;

   // Invalid positions
   if (pos < 1 || pos > listSize + 1) // Use listSize instead of size
   {
      cout << "Invalid" << endl;
      return;
   }

   // Inserting as the first node
   if (pos == 1)
   {
      new_node->next = *head;
      *head = new_node;
      listSize++; // Update the global size variable
   }
```

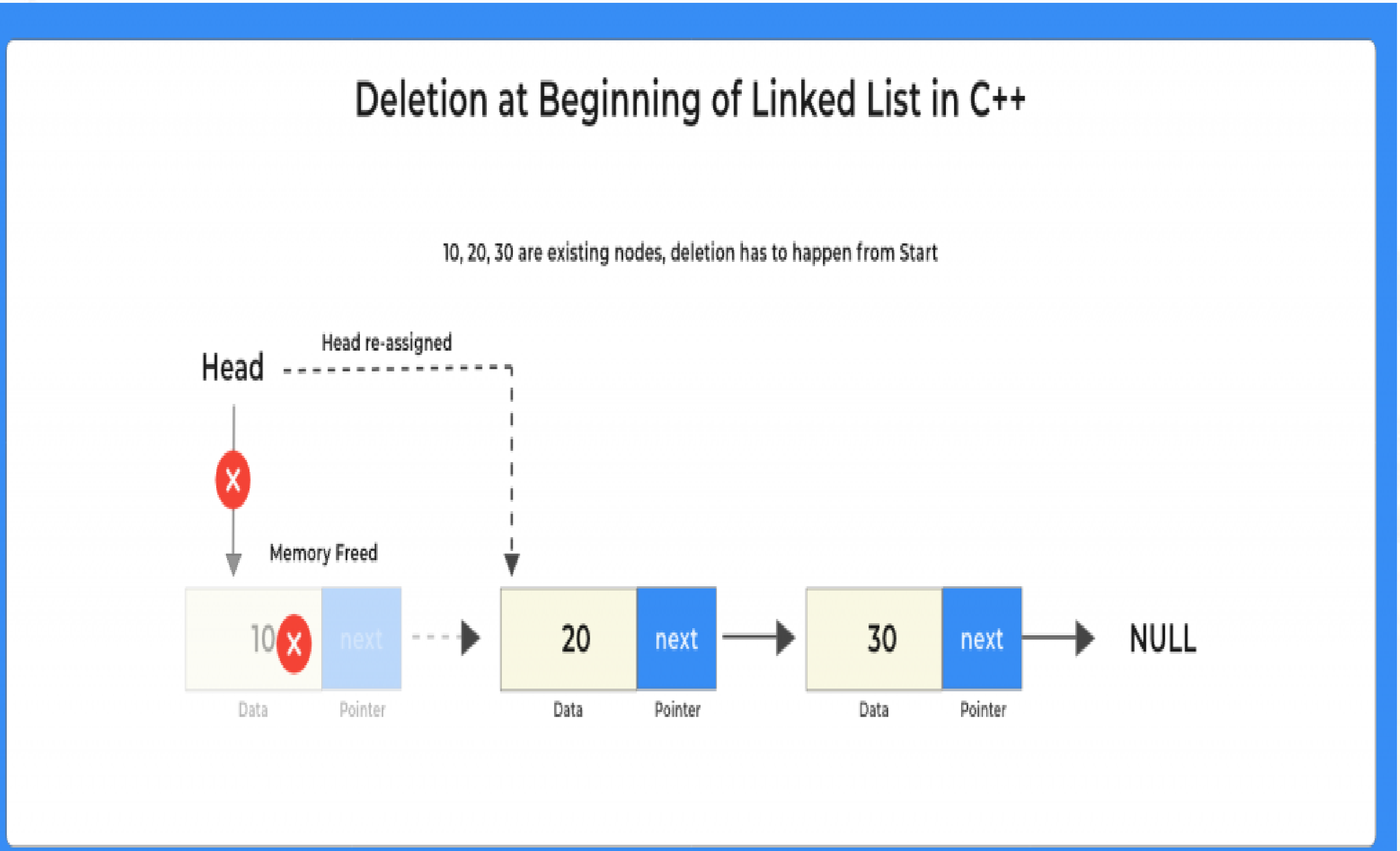
Singly Linked List

- **Insertion at position(in between):**

```
    else
    {
        Node *temp = *head;

        // Traverse to the current (pos-1)th node
        while (--pos > 1)
        {
            temp = temp->next;
        }

        new_node->next = temp->next;
        temp->next = new_node;
        listSize++; // Update the global size variable
    }
}
```

Singly Linked List

- **Deletion of a node at beginning :**



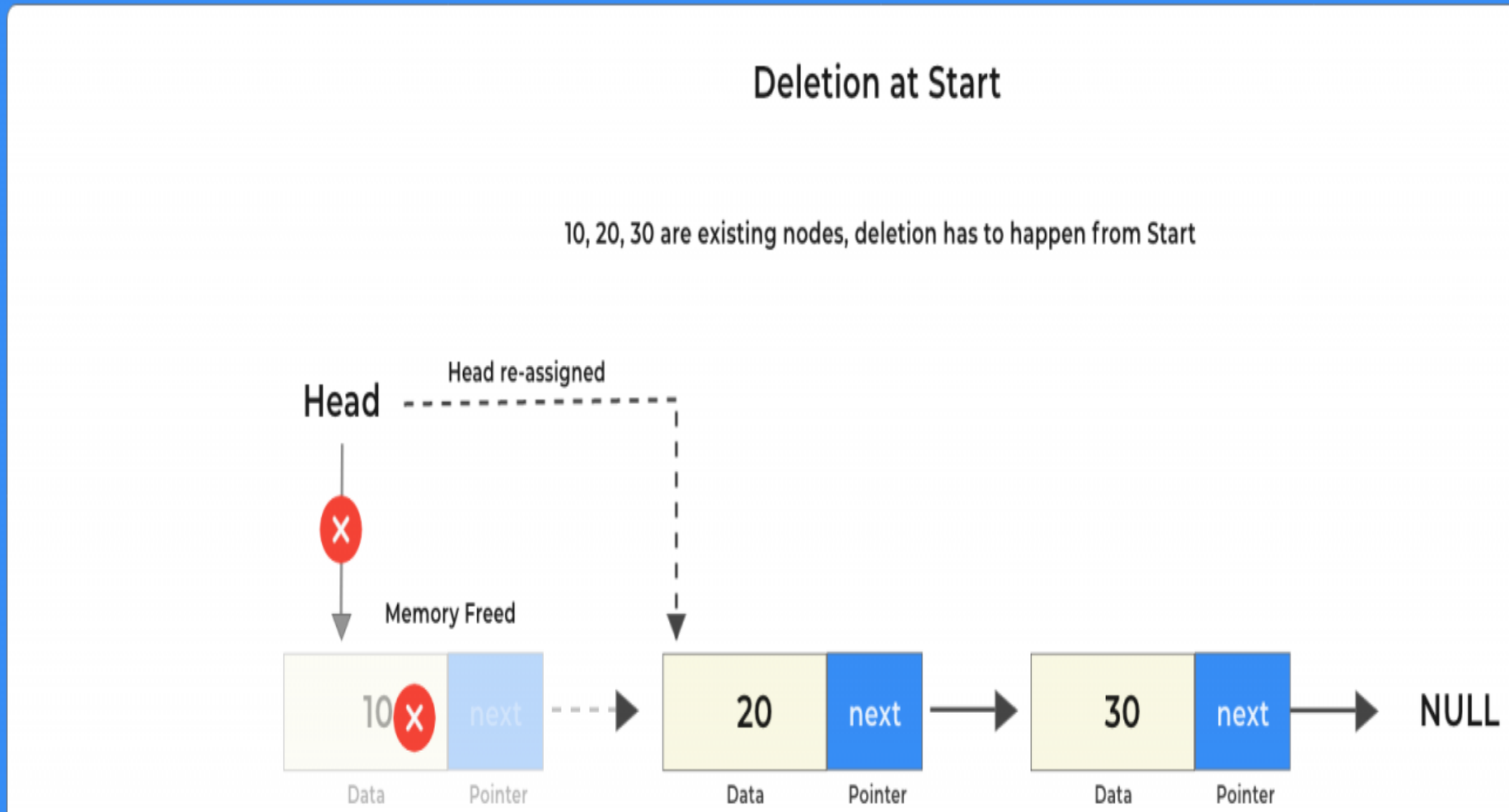
Singly Linked List

- **Deletion of a node at beginning :**

```
void deleteStart()
    {
        if (head == nullptr)
        {
            cout << "Linked List Empty, nothing to delete" << endl;
            return;
        }

        Node* temp = head;
        head = head->next;

        cout << temp->data << " deleted" << endl;
        delete temp;
    }
```

Singly Linked List

- **Deletion of a node**



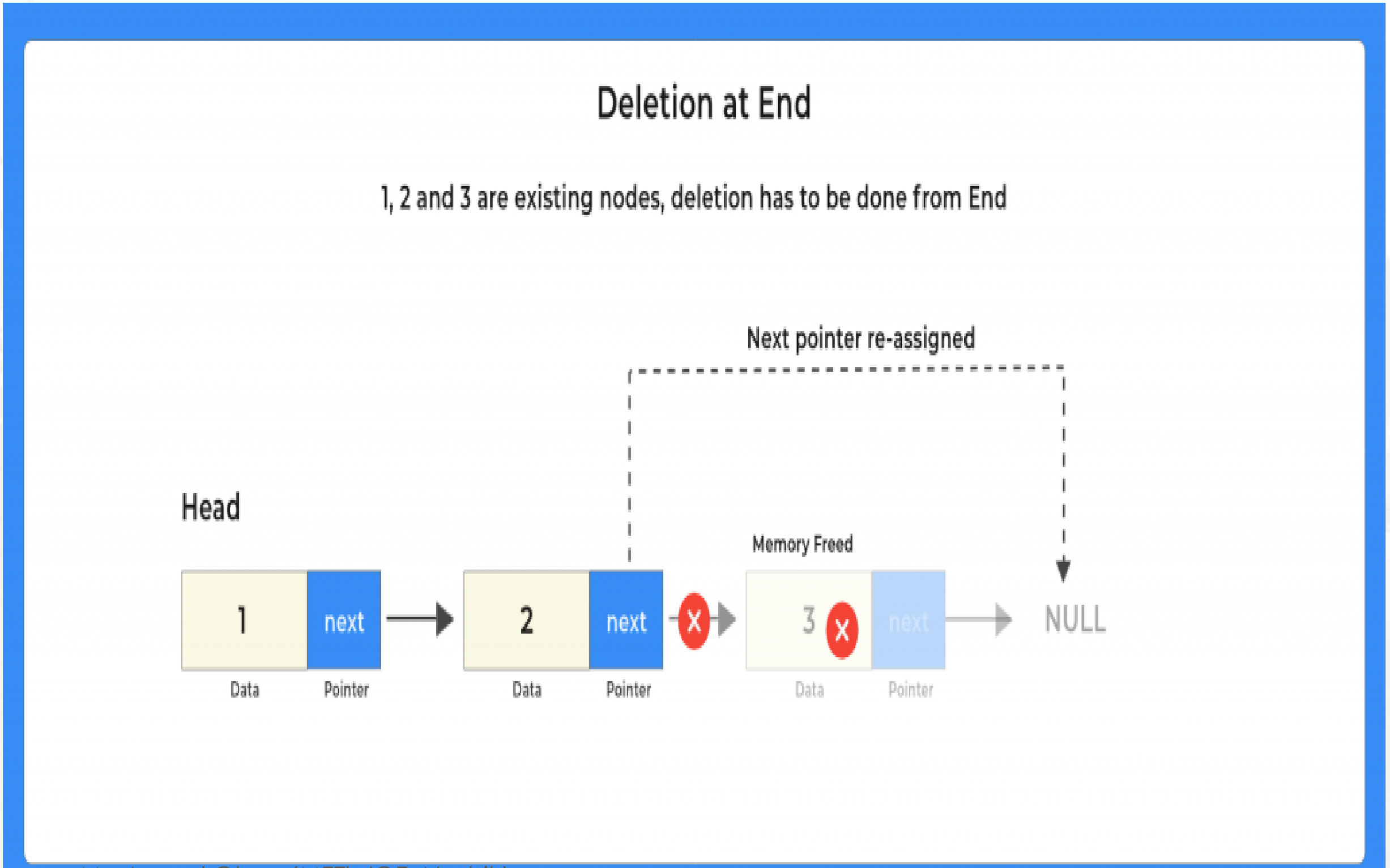
Singly Linked List

- **Deletion of a node**

```
void deleteStart()
    {
        if (head == nullptr)
        {
            cout << "Linked List Empty, nothing to delete" << endl;
            return;
        }

        Node* temp = head;
        head = head->next;

        cout << temp->data << " deleted" << endl;
        delete temp;
    }
```

Singly Linked List

- **Deletion of a node at end :**



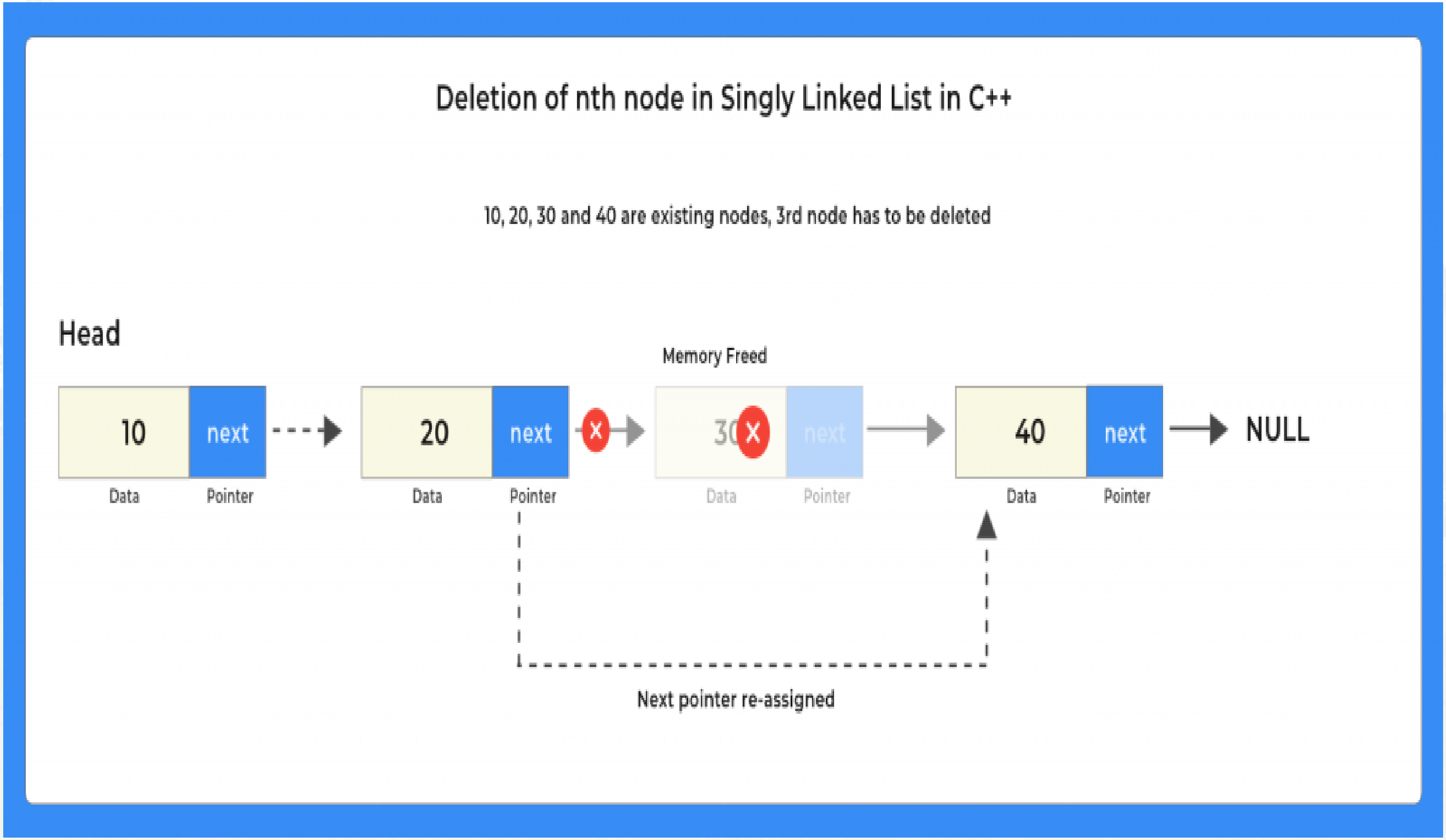
Singly Linked List

- **Deletion of a node at end :**

```
void deleteEnd (Node ** head)
{
  Node *tempNode = *head;
  Node *previous;

  // Can't delete from empty Linked List
  if (*head == NULL)
    {
      cout << ("\nEmpty List, can't delete");
      return;
    }

// if Linked List has only 1 node
if (tempNode->next == NULL)
  {
    cout << "\nValue Deleted: " << (*head)->data;
    *head = NULL;
    return;
  }
// else traverse to the last node
while (tempNode->next != NULL)
  {
    // store previous link node as we need to change its
    previous = tempNode;
    tempNode = tempNode->next;
  }

  // Curr assign 2nd last node's next to Null
  previous->next = NULL;

  cout << "\nValue Deleted: " << tempNode->data;
  // delete the last node
  free (tempNode);
  // 2nd last now becomes the last node
}
```

Singly Linked List

- **Deletion of a node at nth position :**



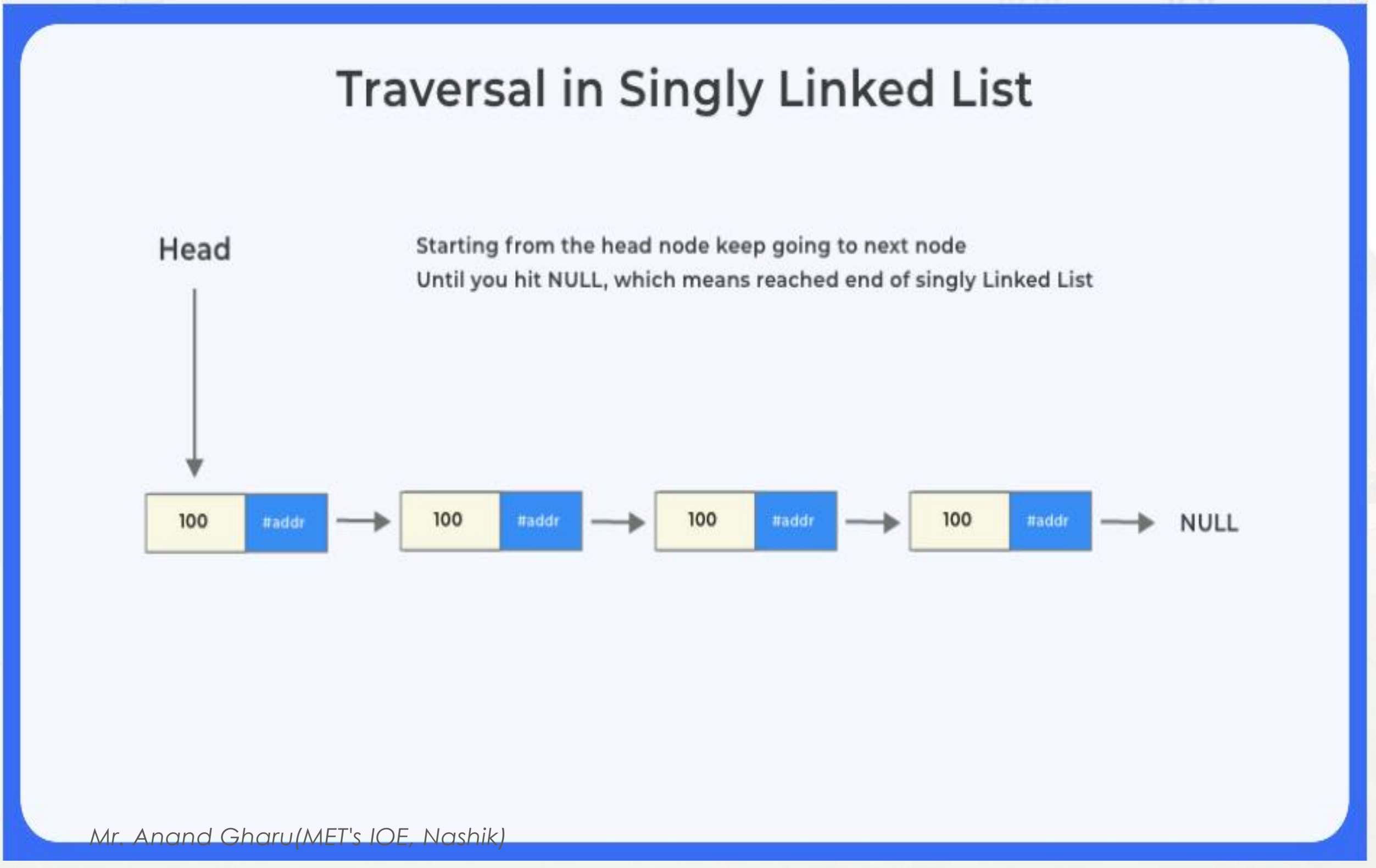
Singly Linked List

- **Deletion of a node at nth position:**

```
void deleteNthNode (int n, Node ** head)
{
  Node *temp = *head;
  Node *prevNode;

  int len = calcLen (*head);

  if (n < 1 || n > len)
    {
      cout << "Invalid" << endl;
      return;
    }

// delete the 1st node
if (n == 1)
  {
    *head = (*head)->next;
    cout << temp->data << " deleted" << endl;
    delete (temp);
    return;
  }

// traverse to the n'th node
while (--n)
  {
    prevNode = temp;
    temp = temp->next;
  }

// change prevNode node's next node to nth node's next node
  prevNode->next = temp->next;

  // delete this nth node
  cout << temp->data << " deleted" << endl;;
  delete (temp);
}
```

Singly Linked List

- **Traversal in Singly List :**



Singly Linked List

- **Traversal/Display in Singly List :**

```
void display()
    {
        cout << "Linked List: ";

        Node* current = head;
        while (current != nullptr)
        {
            cout << current->data << " ";
            current = current->next;
        }

        cout << endl;
    }
```

Singly Linked List

- **Traversal in Singly List :**

```
void display()
    {
        cout << "Linked List: ";

        Node* current = head;
        while (current != nullptr)
        {
            cout << current->data << " ";
            current = current->next;
        }

        cout << endl;
    }
```

Singly Linked List

- **Reverse Singly Linked List :**

Reverse A Linked List In C++

```
ListNode* reverseList(ListNode* head) {
        ListNode *prev = NULL, *cur=head, *tmp;
        while(cur){
            tmp = cur->next;
            cur->next = prev;
            prev = cur;
            cur = tmp;
        }
        return prev;
    }
```

Reverse Singly Linked List

```
void reverseList() {
   //1. If head is not null create three nodes
   //    prevNode - pointing to head,
   //    tempNode - pointing to head,
   //    curNode - pointing to next of head
   if(head != NULL) {
      Node* prevNode = head;
      Node* tempNode = head;
      Node* curNode = head->next;

      //2. assign next of prevNode as null to make the
      //    first node as last node of the reversed list
      prevNode->next = NULL;

      while(curNode != NULL) {
         //3. While the curNode is not null adjust links
         //    (unlink curNode and link it to the reversed list
         //    from front and modify curNode and prevNode)
         tempNode = curNode->next;
         curNode->next = prevNode;
         prevNode = curNode;
         curNode = tempNode;
      }

      //4. Make prevNode (last node) as head
      head = prevNode;
   }
}
```

Sort Singly Linked List

In this program, we need to sort the nodes of the given singly linked list in ascending order.

Original list:

Sorted list:

Sort Singly Linked List

```
void sortList() {
   //Node current will point to head
   struct node *current = head, *index = NULL;
   int temp;

   if(head == NULL) {
      return;
   }
   else {
      while(current != NULL) {
         //Node index will point to node next to current
         index = current->next;

         while(index != NULL) {
            //If current node's data is greater than index's node data, swap the data between them
            if(current->data > index->data) {
               temp = current->data;
               current->data = index->data;
               index->data = temp;
            }
            index = index->next;
         }
         current = current->next;
      }
   }
```

Doubly Linked List

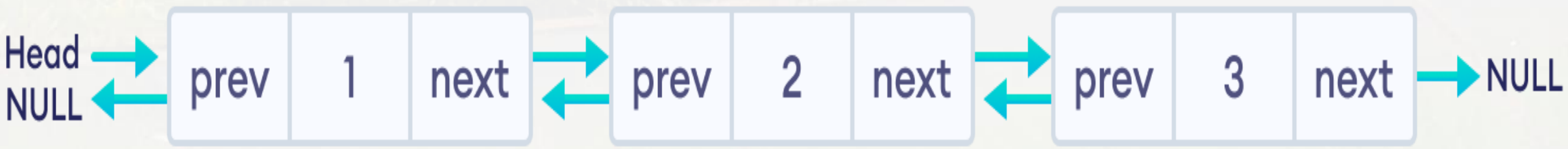
A doubly linked list is a type of linked list in which each node consists of 3 components:

1. *prev - address of the previous node
2. data - data item
3. *next - address of next node

Here, the single node is represented as

```c
struct node {
    int data;
    struct node *next;
    struct node *prev;
}
```

Representation of Doubly Linked List



Doubly Linked List

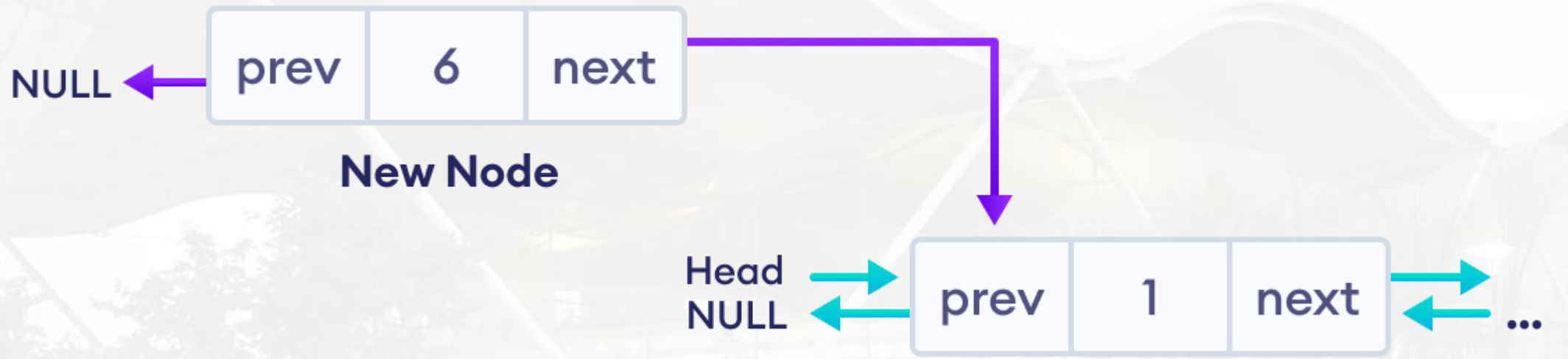
Initialization of Doubly Linked List

```
/* Initialize nodes */
struct node *head;
struct node *one = NULL;
struct node *two = NULL;
struct node *three = NULL;

/* Allocate memory */
one = malloc(sizeof(struct node));
two = malloc(sizeof(struct node));
three = malloc(sizeof(struct node));

/* Assign data values */
one->data = 1;
two->data = 2;
three->data = 3;

/* Connect nodes */
one->next = two;
one->prev = NULL;

two->next = three;
two->prev = one;

three->next = NULL;
three->prev = two;

/* Save address of first node in head */
head = one;
```

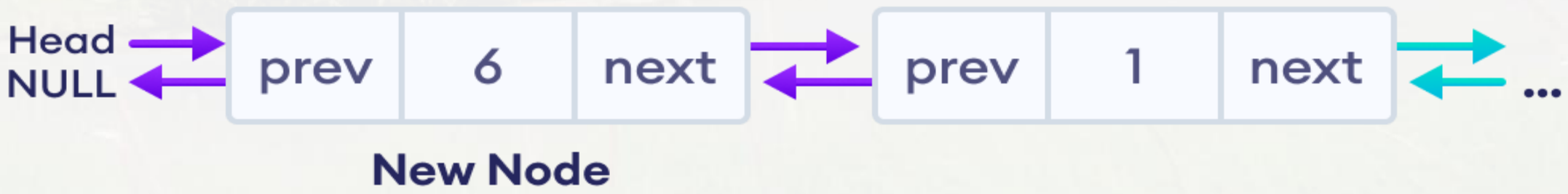
Doubly Linked List

Insertion at beginning in Doubly Linked List

- **Set prev and next pointers of new node :**



- **Make new node as head node :**



Doubly Linked List

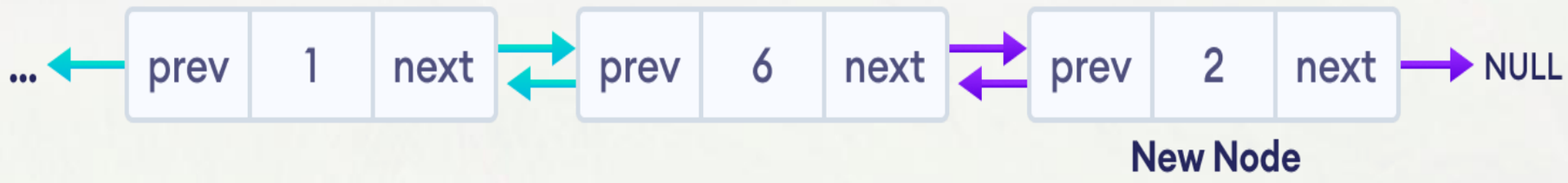
Insertion at beginning in Doubly Linked List

```
// insert node at the front
void insertFront(struct Node** head, int data) {

    // allocate memory for newNode
    struct Node* newNode = new Node;

    // assign data to newNode
    newNode->data = data;

    // point next of newNode to the first node of the doubly linked list
    newNode->next = (*head);

    // point prev to NULL
    newNode->prev = NULL;

    // point previous of the first node (now first node is the second node) to newNode
    if ((*head) != NULL)
        (*head)->prev = newNode;

    // head points to newNode
    (*head) = newNode;
}
```

Doubly Linked List

Insertion at end in Doubly Linked List

```
// insert a newNode at the end of the list
void insertEnd(struct Node** head, int data) {
    // allocate memory for node
    struct Node* newNode = new Node;

    // assign data to newNode
    newNode->data = data;

    // assign NULL to next of newNode
    newNode->next = NULL;

    // store the head node temporarily (for later use)
    struct Node* temp = *head;

    // if the linked list is empty, make the newNode as head node
    if (*head == NULL) {
        newNode->prev = NULL;
        *head = newNode;
        return;
    }

    // if the linked list is not empty, traverse to the end of the linked list
    while (temp->next != NULL)
        temp = temp->next;

    // now, the last node of the linked list is temp

    // point the next of the last node (temp) to newNode.
    temp->next = newNode;

    // assign prev of newNode to temp
    newNode->prev = temp;
}
```



Doubly Linked List

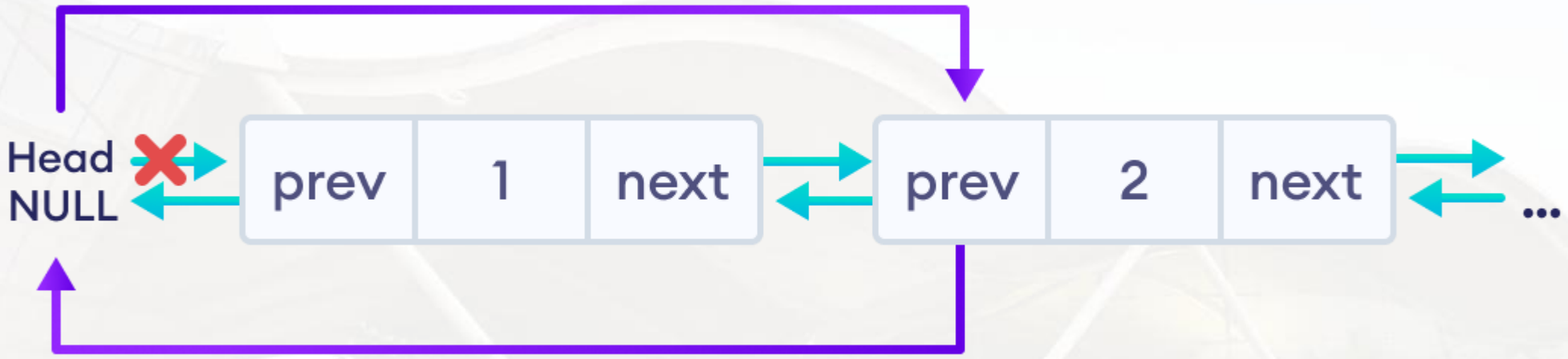
Insertion at specific position in Doubly Linked List

```
// insert a node after a specific node
void insertAfter(struct Node* prev_node, int data) {

    // check if previous node is NULL
    if (prev_node == NULL) {
        cout << "previous node cannot be NULL";
        return;
    }

    // allocate memory for newNode
    struct Node* newNode = new Node;

    // assign data to newNode
    newNode->data = data;

    // set next of newNode to next of prev node
    newNode->next = prev_node->next;

    // set next of prev node to newNode
    prev_node->next = newNode;

    // set prev of newNode to the previous node
    newNode->prev = prev_node;

    // set prev of newNode's next to newNode
    if (newNode->next != NULL)
        newNode->next->prev = newNode;
}
```

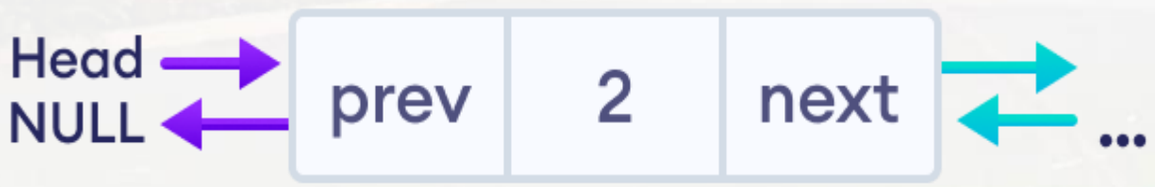
Doubly Linked List

Deletion first node in Doubly Linked List :

- Reset value node after the del_node (i.e. node two)




```
if (*head == del_node)
    *head = del_node->next;

if (del_node->prev != NULL)
    del_node->prev->next = del_node->next;

free(del);
```

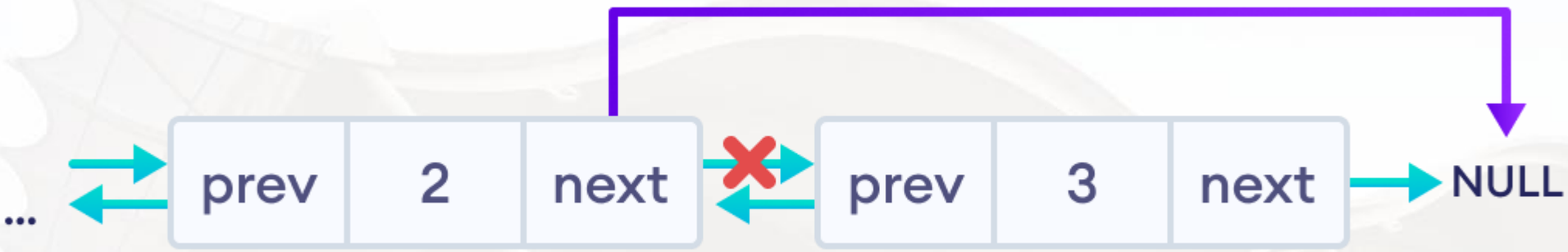



Free the space of the first node

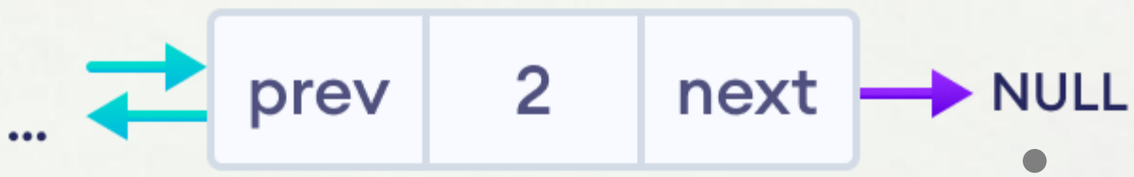
Doubly Linked List

Deletion last node in Doubly Linked List :

- Reset value node after the del_node (i.e. node two)




```
if (del_node->prev != NULL)
    del_node->prev->next = del_node->next;
```



Doubly Linked List

Deletion nth(inner) node in Doubly Linked List :

```
if (del_node->next != NULL)
    del_node->next->prev = del_node->prev;

if (del_node->prev != NULL)
    del_node->prev->next = del_node->next;
```

Doubly Linked List

Display Doubly Linked List :

```
void display() {
    struct Node* ptr;
    ptr = head;
    while(ptr != NULL) {
        cout<< ptr->data <<" ";
        ptr = ptr->next;
    }
}
```

References

1. https://www.codingninjas.com/studio/library/static-and-dynamic-memory-allocation.
2. https://www.studytonight.com/data-structures/introduction-to-linked-list
3. https://www.javatpoint.com/singly-linked-list
4. https://www.studytonight.com/data-structures/introduction-to-linked-list
5. https://prepinsta.com/cpp-program/for-insertion-in-a-sorted-linked-list/
6. https://prepinsta.com/cpp-program/for-insertion-in-a-sorted-linked-list/
7. *https://prepinsta.com/cpp-program/insert-a-node-at-a-specific-position-in-a-linked-list/*
8. *https://www.interviewbit.com/blog/reverse-a-linked-list/*
9. *https://www.alphacodingskills.com/cpp/ds/cpp-reverse-the-linked-list.php*
10. *https://www.javatpoint.com/program-to-sort-the-elements-of-the-singly-linked-list*
11. *https://www.geeksforgeeks.org/adding-two-polynomials-using-linked-list/*
12. *https://www.programiz.com/dsa/doubly-linked-list#insertion-at-end*
13. *https://www.programiz.com/dsa/doubly-linked-list*

Thanks .....!!!!

Prof. ANAND GHARU

ASSISTANT PROFESSOR

Blog : anandgharu.wordpress.com